Sustav za upravljanje hotelom

Projektni rad

Tim-6
Ivona Vlašić | Eugen Bučko
Marko Telešman | Alma Reka

Fakultet informatike Pula / Informatika / Baze podataka II

Mentor: Doc. Dr. sc. Goran Oreški
Asistent: Dr. sc. Romeo Šajina

Pula, 19. siječnja 2026.

Fakultet informatike u Puli

Sadržaj

1 Opis poslovnog procesa **4**

2 ER-Dijagram **5**

3 Tablice (Eng. Tables) **8**

3.1 Temeljni podaci i Akteri sustava . . . 8
3.1.1 Tablica: DRZAVA . . . 8
3.1.2 Tablica: GRAD . . . 8
3.1.3 Tablica: VRSTA_DOKUMENTA . . . 9
3.1.4 Tablica: STATUS_GOSTA . . . 9
3.1.5 Tablica: ODJEL . . . 10
3.1.6 Tablica: POZICIJA_ZAPOSLENIKA . . . 10
3.1.7 Tablica: ZAPOSLENIK . . . 11
3.1.8 Tablica: GOST . . . 12
3.2 Definiranje ponude i resursa . . . 14
3.2.1 Tablica: TIP_SOBE . . . 14
3.2.2 Tablica: SOBA . . . 14
3.2.3 Tablica: CJENIK_SOBA . . . 15
3.2.4 Tablica: USLUGA . . . 16
3.2.5 Tablica: RESTORAN_STOL . . . 16
3.2.6 Tablica: ARTIKL . . . 17
3.2.7 Tablica: NORMATIV . . . 18
3.3 Glavni proces: Rezervacije . . . 19
3.3.1 Tablica: PROMOCIJA . . . 19
3.3.2 Tablica: REZERVACIJA . . . 19
3.3.3 Tablica: REZERVACIJA_GOST . . . 20
3.3.4 Tablica: LOG_REZERVACIJE . . . 21
3.4 Operativno poslovanje (tokom boravka) . . . 23
3.4.1 Tablica: RESTORAN_NARUDZBA . . . 23
3.4.2 Tablica: RESTORAN_STAVKA . . . 23
3.4.3 Tablica: CISCENJE_DNEVNI_NALOG . . . 24
3.4.4 Tablica: SERVIS_DNEVNI_NALOG . . . 25
3.5 Financije i Zaključenje boravka . . . 26
3.5.1 Tablica: RACUN . . . 26
3.5.2 Tablica: STAVKA_RACUNA . . . 26
3.5.3 Tablica: RECENZIJA . . . 27

4 Eugen B. **28**

4.1 Procedure (Eng. Procedures) . . . 28
4.1.1 Procedura: Otkazivanje rezervacije . . . 28
4.1.2 Procedura: Brisanje podataka iz loga kojima je prošlo više od 90 dana . . . 29
4.2 Okidač (Eng. Trigger) . . . 31
4.2.1 Okidač: Sprječavanje preklapanja rezervacija . . . 31
4.2.2 Okidač: Promocija se smije unijeti u rezervaciju samo ako je aktivna i vremenski važeća . . . 32

4.2.3 Okidač: Provjera valjanosti promocije prilikom ažuriranja rezervacije 34
4.2.4 Okidač: Kontrola vremena prijave (Check-in) striktno od 14:00 . . . 35
4.2.5 Okidač: Automatski unos svih novih rezervacija u "log_rezervacija" 36
4.2.6 Okidač: Automatski unos svih ažuriranja postojećih rezervacija u "log_rezervacija" . . . 37
4.2.7 Okidač: Automatski unos svih brisanja postojećih rezervacija u "log_rezervacija" . . . 38
4.2.8 Okidač: Automatski unos besplatnog čišćenja ako je gost boravio duže do tri dana . . . 39
4.3 Upiti (Eng. Query) . . . 41
4.3.1 Prateći upit uz okidač - prikaz aktinih rezervacija za provjeru zauzetosti (Eugen B.) . . . 41
4.3.2 Prateći upit uz okidač: Upit za prikaz statusa i statistike korištenja promocija . . . 42
4.3.3 Prateći upit uz okidač: Prikaz statusa na aktivnim rezervacijama . 43
4.3.4 Upit: Prikaz aktivnih rezervacija i statusa prijave gostiju . . . 44
4.3.5 Upit: Prikaz povijesti promjena statusa rezervacija s detaljima . . . 45
4.3.6 Upit: Prikaz zadnje promjene i trenutnog stanja za specifičnu rezervaciju . . . 47
4.3.7 Upit: Pregled obrisanih rezervacija i uklonjenih podataka . . . 48
4.3.8 Upit: Upit za provjeru ispravnosti automatskog popusta na čišćenje 49

5 Ivona V. 52
5.1 Procedure (Eng. Procedures) . . . 52
5.1.1 Procedura 1: SP_OCISCENA_SOBA . . . 52
5.1.2 Procedura 2: SP_DODAJ_GOSTA_NA_REZERVACIJU . . . 52
5.1.3 Procedura 3: SP_PRIMIJENI_PROMOCIJU . . . 53
5.2 Pogledi (Eng. Views) . . . 55
5.2.1 Pogled: TRENUTNO_STANJE_SOBA . . . 55
5.2.2 Pogled: RACUN_PO_REZERVACIJI . . . 56
5.2.3 Pogled: LOW_STOCK_ALERT . . . 57
5.2.4 Pogled: NAJBOLJI_GOSTI_POTROSNJA . . . 58
5.2.5 Pogled: EVIDENCIJA_CIJENA_SOBA . . . 59
5.2.6 Pogled: NEPLACENI_RACUNI . . . 60

6 Marko T. 61
6.1 Okidači . . . 61
6.1.1 Okidač 1: Osiguravanje ispravnog kronološkog slijeda rezervacije . . 61
6.1.2 Okidač 2: Zabrana prijevremenog Check-ina prije datuma rezervacije 62
6.1.3 Okidač 3: Automatizacija statusa sobe nakon odlaska gosta . . . 63
6.1.4 Okidač 4: Blokada prodaje soba koje su u kvaru ili čišćenju . . . 63
6.2 Procedure . . . 65
6.2.1 Procedura 1: PROC_OTVORI_NARUDZBU . . . 65
6.2.2 Procedura 2: PROC_DODAJ_STAVKU . . . 66
6.2.3 Procedura 3: PROC_ZAPOCNI_PRIPREMU . . . 67
6.2.4 Procedura 4: PROC_POSLUZI_STAVKU . . . 69
6.2.5 Procedura 5: PROC_NAPLATI_NARUDZBU . . . 71
6.2.6 Procedura 6: PROC_KREIRAJ_REZERVACIJU . . . 74
6.2.7 Procedura 7: PROC_KREIRAJ_GOSTA . . . 76

6.2.8 Procedura 8: PROC_REZERVACIJA_CHECK_IN 77
6.2.9 Procedura 9: PROC_REZERVACIJA_CHECK_OUT 77
6.2.10 Procedura 10: PROC_DODAJ_USLUGU_NA_SOBU 78
6.2.11 Procedura 11: PROC_PRIJAVI_KVAR 80
6.3 Funkcije 82
6.3.1 Funkcija 1: IZRACUNAJ_MARZU_JELA 82
6.4 Pogledi 84
6.5 Pogledi (Views) 84
6.5.1 Pogled 1: PREGLED_SVIH_RACUNA 84
6.5.2 Pogled 2: PREGLED_ZALIHA_SKLADISTE 85
6.5.3 Pogled 3: PREGLED_SVIH_REZERVACIJA 85
6.5.4 Pogled 4: VIEW_MENI_RESTORAN 86
6.5.5 Pogled 5: VIEW_KUHINJA_DISPLAY 86
6.5.6 Pogled 6: VIEW_OTVORENE_NARUDZBE_TOTAL 87
6.5.7 Pogled 7: VIEW_SOBE_ZA_CISCENJE 88
6.5.8 Pogled 8: VIEW_AKTIVNE_REZERVACIJE 88
6.5.9 Pogled 9: VIEW_DODATNE_USLUGE 89
6.5.10 Pogled 10: VIEW_AKTIVNI_KVAROVI 89
6.5.11 Pogled 11: VIEW_OSOBLJE_ODRZAVANJA 90
6.5.12 Pogled 12: VIEW_STATUS_SOBA_BOJE 91
6.5.13 Pogled 13: VIEW_SVE_SOBE_DROPDOWN 91
6.5.14 Pogled 14: VIEW_STAVKE_NARUDZBE 92
6.5.15 Pogled 15: VIEW_DETALJI_RACUNA 93
6.5.16 Pogled 16: VIEW_RECEPTURE_DETALJNO 93
6.5.17 Pogled 17: VIEW_ZAPOSLENICI_RECEPCIJA 94
6.5.18 Pogled 18: VIEW_ZAPOSLENICI_KONOBARI 94
6.5.19 Pogled 19: VIEW_ZAPOSLENICI_DOMACINSTVO 95
6.5.20 Pogled 20: VIEW_ZAPOSLENICI_ODRZAVANJE 95

7 Alma **97**
7.1 A. Upravljanje Recenzijama 97
7.2 B. Rezervacije i Provjera Dostupnosti 99
7.3 C. Automatizacija Računa (Billing) 103
7.4 D. Financije i Porezi (Reporting) 110

1. Opis poslovnog procesa

Poslovni proces koji podržava ova baza podataka odnosi se na cjelovito upravljanje hotelskim poslovanjem, obuhvaćuje ciklus boravka gosta od rezervacije do odlaska, uz integrirano praćnje popratnih usluga, resursa i financija.

Proces započinje definiranjem temelja poslovanja. To uključuje uspostavu organizacijske strukture hotela (odijeli i zaposlenici s definiranim ulogama) te precizno definiranje smještajnih kapaciteta. Unose se tipovi soba i konkretne sobe, a ključan element je formiranje sezonskih cjenika koji određuju cijene noćenja i boravišnih pristrojbi za različite periode godine. Paralelno se definiraju dodatne usluge hotela (npr. restoran) i usposavlja se veza sa skladišnim poslovanjem putem normativa, koji točno određuju utrošak artikala za svaku pruženu uslugu.

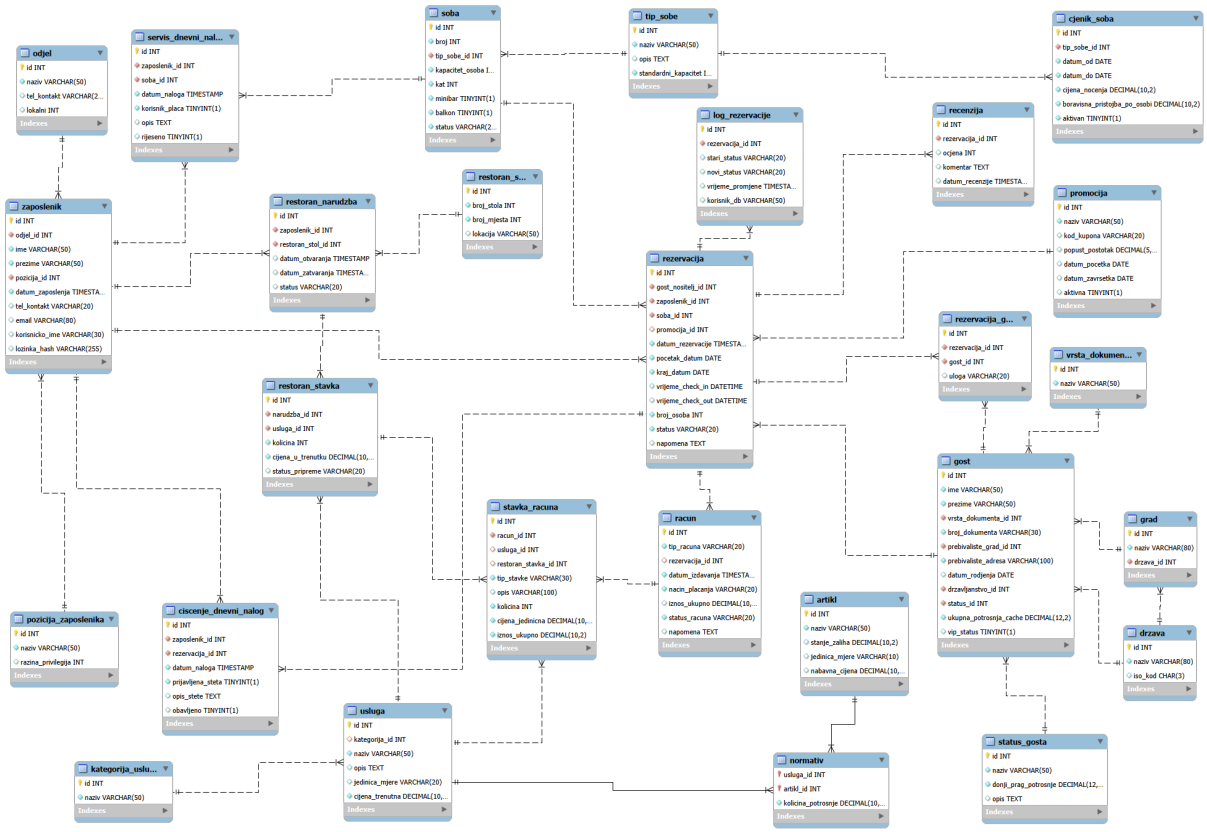
Operativni dio procesa fokusiran je na putovanje gosta. Započinje registracijom podataka o gostu i kreiranjem rezervacije, pri čemu sustav provjerava dostupnost kapaciteta i omogućuje primjenu eventualnih promotivnih akcija. Dolaskom gosta (check-in), soba se zauzima, a boravak aktivira.

Tijekom boravka, gost koristi različite hotelske usluge. Narudžbe u restoranu bilježe se na stolove, a sustav automatski, temeljem definiranih normativa, razdužuje skladište za utrošene artikle, osiguravajući točno stanje zaliha u realnom vremenu. Isto vremeno se odvija proces održavanja, generiraju se dnevni nalozi za čišćenje soba, a u slučaju kvarova kreiraju se servisni nalozi koji privremeno blokiraju sobu za daljnje iznajmljivanje dok se problem ne riješi. Sve ključne promjene na rezervacijama se bilježe u logovima radi sigurnosti i sljedivosti.

Završna faza procesa je odlazak gosta (check-out) i financijsko zaključenje. Sustav objedinjuje sve troškove, od cijene smještaja izračunate prema važećim cjenicima do svih konzumiranih dodatnih usluga, u jedan finalni račun. Plaćanjem računa boravak se zaključuje, soba se oslobađa za čišćenje, a ukupna potrošnja gosta se ažurira, što može rezultirati automatskim promaknućem u viši status (npr. VIP) za buduće pogodnosti. Proces završava s mogučnošću da gost ostavi recenziju svog boravka.

Ovaj poslovni proces dizajniran je kako bi centralizirao i automatizirao sve ključne hotelske operacije, povezujući recepciju, restoran, skladište i menadžment u jedinstven i učinkovit sustav koji osigurava točnost podataka i visoku razinu usluge.

2. ER-Dijagram



Tablica 1: Kardinaliteti i odnosi u bazi podataka

Odnos (Entiteti)	**Tip**	**Objašnjenje**
Geografija		
Država - Grad	1:N	Jedna država može imati više gradova povezanih s njom, ali svaki pojedini grad pripada točno jednoj državi.
Država - Gost	1:N	Jedna država može biti zemlja državljanstva za više gostiju, dok je svaki gost državljanin točno jedne države.
Vrsta dokumenta - Gost	1:N	Jedna vrsta dokumenta može biti korištena kao identifikacijski dokument za više gostiju, a svaki gost je povezan s točno jednom vrstom dokumenta s kojim se identificira.
Organizacija i ljudi		
Odjel - Zaposlenik	1:N	U jednom odjelu može raditi više zaposlenika, dok je svaki zaposlenik raspoređen u točno jedan odjel.
Zaposlenik - Rezervacija	1:N	Jedan zaposlenik (recepcioner) može kreirati ili biti odgovoran za više rezervacija, a svaka pojedina rezervacija je povezana s točno jednim zaposlenikom koji ju je obradio.
		Nastavlja se na idućoj stranici...

Tablica 1 – nastavak s prethodne stranice

Odnos (Entiteti)	Tip	Objašnjenje
Zaposlenik - Restoran narudžba	1:N	Jedan zaposlenik (konobar) može otvoriti više narudžbi, dok je svaka narudžba povezana s točno jednim zaposlenikom koji ju je otvorio.
Zaposlenik - Čišćenje dnevni nalog	1:N	Jedan zaposlenik (sobarica) može biti zadužen za više dnevnih naloga za čišćenje, a svaki nalog je dodijeljen točno jednom zaposleniku.
Zaposlenik - Servis dnevni nalog	1:N	Jedan zaposlenik (domar) može biti zadužen za više servisnih naloga, dok je svaki servisni nalog dodijeljen točno jednom zaposleniku.
Smještajni kapaciteti		
Tip sobe - Soba	1:N	Jedan tip sobe (npr. standard dvokrevetna) može definirati karakteristike za više fizičkih soba u hotelu, a svaka fizička soba je definirana točno jednim tipom sobe.
Tip sobe - Cjenik soba	1:N	Za jedan tip sobe može postojati više zapisa u cjeniku (različite cijene za različite sezone ili datumske raspone), a svaki zapis u cjeniku se odnosi na točno jedan tip sobe.
Soba - Rezervacija	1:N	Jedna soba može biti predmet više rezervacija tijekom vremena (u različitim terminima), dok se svaka specifična rezervacija odnosi na točno jednu sobu u određenom terminu.
Soba - Servis dnevni nalog	1:N	Za jednu sobu se može kreirati više servisnih naloga (npr. popravak klime danas, zamjena žarulje sutra), a svaki servisni nalog je vezan za točno jednu sobu.
Usluge, restoran i skladište		
Kategorija usluge - Usluga	1:N	Jedna kategorija usluge (npr. hrana) može obuhvaćati više različitih usluga (npr. pizza, tjestenina), dok svaka usluga pripada točno jednoj kategoriji.
Usluga - Restoran stavka	1:N	Jedna kategorija usluge može obuhvaćati više različitih usluga, dok svaka usluga pripada točno jednoj kategoriji.
Restoran narudžba - Restoran stavka	1:N	Za jednim restoranskim stolom može se tijekom vremena otvoriti više narudžbi, a svaka narudžba je vezana za točno jedan stol za kojim je napravljena.
Artikl - Normativ | Usluga - Normativ	M:N	**Realizirano preko vezne tablice 'normativ'.** Iz perspektive artikla: jedan artikl (npr. mlijeko) može biti sastojak u više različitih usluga. Iz perspektive usluge: jedna usluga (npr. kava s mlijekom) može se sastojati od više različitih artikala.
Rezervacije i marketing		
Gost - Rezervacija	1:N	Jedan gost može biti nositelj (glavni gost) za više različitih rezervacija, dok svaka rezervacija ima točno jednog gosta nositelja.

Nastavlja se na idućoj stranici...

Tablica 1 – nastavak s prethodne stranice

Odnos (Entiteti)	**Tip**	**Objašnjenje**
Gost - Rezervacija | Rezervacija - Gost	M:N	**Realizirano preko vezne tablice 'rezervacija_gost'.** Iz perspektive gosta: jedan gost može biti pridružen više različitih rezervacija (npr. kao član obitelji). Iz perspektive rezervacije: jedna rezervacija može uključivati više gostiju.
Promocija - Rezervacija	1:N	Jedna promocija (npr. ljetni popust) može biti primijenjena na više različitih rezervacija, dok svaka rezervacija može imati primijenjenu najviše jednu promociju (ili nijednu).
Rezervacija - Log rezervacije	1:N	Za jednu rezervaciju se može generirati više zapisa u logu (svaki put kad se promijeni njen status), dok se svaki zapis u logu odnosi na točno jednu rezervaciju čiju povijest prati.
Financije		
Rezervacija - Račun	1:N	Za jednu rezervaciju se može izdati više računa (npr. predujam, konačni račun, storno račun), a svaki račun je vezan za točno jednu rezervaciju na koju se odnosi.
Račun - Stavka računa	1:N	Jedan račun može sadržavati više stavki (npr. noćenje, boravišna pristojba, večera), dok svaka stavka pripada točno jednom računu.
Usluga - Stavka računa	1:N	Jedna usluga se može pojaviti kao stavka na više različitih računa, dok svaka stavka na računu referencira točno jednu uslugu.
Održavanje i feedback		
Rezervacija - Čišćenje dnevni nalog	1:N	Tijekom trajanja jedne rezervacije može se kreirati više dnevnih naloga za čišćenje sobe (npr. svaki dan boravka), a svaki nalog je vezan za točno jednu rezervaciju.
Rezervacija - Recenzija	1:N	**Fizički:** jedna rezervacija može imati više recenzija. **Logički:** Za jednu rezervaciju bi trebala biti dozvoljena najviše jedna recenzija.

3. Tablice (Eng. Tables)

3.1 Temeljni podaci i Akteri sustava

3.1.1 Tablica: DRZAVA

```sql
CREATE TABLE drzava (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(80) NOT NULL,
    iso_kod CHAR(3) UNIQUE
);
```

Isječak koda 1: Kreiranje tablice DRZAVA

Svrha:

Tablica DRZAVA služi kao središnji šifrarnik za pohranu popisa država. Njena osnovna funkcija je standarizacija geografskih podataka unutar sustava. Umjesto ručnog upisivanja naziva države, ostale tablice (poput GRAD ili GOST) referencirat će se na ovu tablicu. Ovo je ključno za točno određivanje državljanstva gostiju i lokacije prebivališta.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator svake države. Služi kao tehnički ključ za povezivanje s tablicom gradova i gostiju.
- naziv (VARCHAR (80), NOT NULL): Službeni naziv države. Obavezno polje koje se prikazuje korisnicima aplikacije prilikom odabira države.
- iso_kod (CHAR(3), UNIQUE): Međunarodni standardizirani kod države (npr. HRV, USA, DEU). Ovo polje je jedinstveno i korisno za eventualnu integraciju s vanjskim sustavima jer osigurava nedvosmislenu identifikaciju države.

3.1.2 Tablica: GRAD

```sql
CREATE TABLE grad (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(80) NOT NULL,
    drzava_id INT NOT NULL,
    CONSTRAINT fk_grad_drzava FOREIGN KEY (drzava_id) REFERENCES
        drzava(id)
);
```

Isječak koda 2: Kreiranje tablice GRAD

Svrha:

Tablica GRAD pohranjuje popis gradova i povezuje ih s odgovarajućim državama. Njena svrha je omogućiti precizno definiranje prebivališta gostiju te osigurati hijerarshijsku strukturu podataka (svaki grad mora pripadati jednoj državi). Ovi podaci su bitni za izvještaje i analizu demografije gostiju.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator svakog grada. Koristi se za povezivanje s tablicom GOST (mjesto prebivališta)

- naziv (VARCHAR(80), NOT NULL): Ime grada. Obavezno polje za prikaz lokacije.
- drzava_id (INT, NOT NULL, FOREIGN KEY): Vanjski ključ koji povezuje grad s tablicom DRZAVA. Ovo polje osigurava da se grad ne može unijeti bez pripadnosti postojećoj državi u sustavu (referncijalni integritet -> sprječava nastanak neispravnih podataka)

3.1.3 Tablica: VRSTA_DOKUMENTA

```sql
CREATE TABLE vrsta_dokumenta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL
);
```

Isječak koda 3: Kreiranje tablice VRSTA_DOKUMENTA

Svrha:

Tablica VRSTA_DOKUMENTA je jednostavan Šifrarnik koji definira koje vrste osobnih dokumenata hotel prihvaća za identifikaciju gostiju (npr. "Osobna iskaznica", "Putovnica"). Izdvajanjem ovih naziva u zasebnu tablicu osigurava se dosljednost - recepcionari ne moraju ručno upisivati, već biraju iz definiranog popisa.

Atributi:

- id (AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator vrste dokumenta. Koristi se kao vanjski ključ u tablici GOST za povezivanje gosta s dokumentom kojim se prijavio.
- naziv (VARCHAR(50), NOT NULL): Naziv dokumenta. Obavezno polje koje se prikazuje djelatnicima na recepciji.

3.1.4 Tablica: STATUS_GOSTA

```sql
CREATE TABLE status_gosta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL UNIQUE,
    donji_prag_potrosnje DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    opis TEXT
);
```

Isječak koda 4: Kreiranje tablice STATUS_GOSTA

Svrha:

Tablica STATUS_GOSTA definira razine lojalnosti ili kategorije gostiju. Njena svrha je omogućiti segmentaciju gostiju na temelju njihove potrošnje. Ovo je temelj za programe vjernosti, gdje sustav može automatski prepoznati važne goste i dodijeliti im posebne pogodnosi ili popuste.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator statusa

- naziv (VARCHAR(50), NOT NULL, UNIQUE): Naziv statusa. Polje je jedinstveno jer ne mogu postojati dva različita statusa s istim imenom.
- donji_prag_potrosnje (DECIMAL(12,2), NOT NULL): Iznos koji definira minimalnu potrosnju potrebnu za dobivanje ovog statusa. Ovo je ključno polje za automatizaciju, ako gost potroši više od ovog iznosa, sustav ga može "unaprijediti" u viši staus.
- opis (TEXT, NULL): Dodatno objašnjenje pogodnosti koje taj status donosi.

3.1.5 Tablica: ODJEL

```sql
CREATE TABLE odjel (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL,
    tel_kontakt VARCHAR(20),
    lokalni INT
);
```

Isječak koda 5: Kreiranje tablice ODJEL

Svrha:

Tablica ODJEL definira unutarnju strukturu organizacije (hotela). Služi za grupiranje zaposlenika u logičke cjeline kao što su "recepcija", "kuhinja" i sl. Ovakva podjela je nužna za dodjeljivanje zadataka, organizaciju smjena i usmjeravanje interne komunikacije.

Atributi

- id (INT ,AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator odjela. Koristi se za povezivanje zaposlenika s njihovim radnim mjestom.
- nazvi (VARCHAR(50), NOT NULL): Ime odjela, obavezno polje za raspoznavanje organizacijske jefinice
- tel_kontakt (VARCHAR(20), NULL): Službeni kontakt telefon odjela, koristan za vanjske pozive ili hitne slučajeve
- lokalni(INT,NULL): Broj internog lokala (ekstenzija). Praktično polje za brzu komunikaciju unutar hotela (102 za kuhinju)

3.1.6 Tablica: POZICIJA_ZAPOSLENIKA

```sql
CREATE TABLE pozicija_zaposlenika (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL UNIQUE,
    razina_privilegija INT
);
```

Isječak koda 6: Kreiranje tablice POZICIJA_ZAPOSLENIKA

Svrha:

Tablica POZICIJA_ZAPOSLENIKA definira radna mjesta (uloge) unutar organizacije. Osim samog naziva, ključna funkcija ove tablice je upravljanje sigurnošću sustava. Kroz razinu privilegija određuje se tko ima pristup kojim podacima.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator pozicije.
- naziv (VARCHAR(), NOT NULL, UNIQUE): Naziv radnog mjesa. Polje je jedinstveno kako bi se izbjeglo dupliciranje istih uloga.
- razina_privilegija (INT, NULL): Brojčana vrijednost koja označava razinu ovlasti u softveru. Ovo omogućuje implementaciju sigurnosnog modela gdje aplikacije automatski skriva ili pokazuje funkcionalnosti ovisno o ulozi zaposlenika.

3.1.7 Tablica: ZAPOSLENIK

```sql
CREATE TABLE zaposlenik (
    id INT AUTO_INCREMENT PRIMARY KEY,
    odjel_id INT NOT NULL,
    ime VARCHAR(50) NOT NULL,
    prezime VARCHAR(50) NOT NULL,
    pozicija_id INT NOT NULL,
    datum_zaposlenja TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tel_kontakt VARCHAR(20),
    email VARCHAR(80),
    korisnicko_ime VARCHAR(30) UNIQUE,
    lozinka_hash VARCHAR(255),
    CONSTRAINT fk_zaposlenik_odjel
        FOREIGN KEY (odjel_id) REFERENCES odjel(id),
    CONSTRAINT fk_zaposlenik_pozicija
        FOREIGN KEY (pozicija_id) REFERENCES pozicija_zaposlenika(
           id)
);
```

Isječak koda 7: Kreiranje tablice ZAPOSLENIK

Svrha:

Tablica ZAPOSLENIK predstavlja registar svih radnika u organizaciji. Ona ima dvostruku ulogu

1. HR evidencija: Čuva informacije o zaposlenicima
2. Kontrola pristupa (Login): Sadrži podatke potrebne za prijavu u sustav. Svi nalozi, rezervacije i računi bit će povezani s ID-jem zaposlenika, što omogućuje točno praćenje

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator zaposlenika. Ključan za povezivanje s rezervacijama, računima i nalozima
- odjel_id (INT, NOT NULL, FOREIGN KEY): Poveznica na tablicu odjel. Određuje kojem timu zaposlenik pripada.
- ime, prezime (VARCHAR(50), NOT NULL): Osobni podaci za identifikaciju zaposlenika.

- pozicija_id (INT, NOT NULL, FOREIGN KEY): Poveznica na tablicu pozicija_zaposlenika. Definira ulogu i ovlasti zaposlenika u sustavu.
- datum_zaposlenja (TIMESTAMP): Datum kada je osoba počela raditi. Korisno za evidenciju staža.
- tel_kontakt, email (VARCHAR): Kontakt podaci za komunikaciju unutar firme.
- korisnicko_ime (VARCHAR(30), UNIQUE): Ime s kojim se zaposlenik prijavljuje u aplikaciju. Mora biti jedinstveno.
- lozinka_hash (VARCHAR(255)): Ovo polje ne čuva lozinku u običnom tekstu, već njenu "kriptiranu" (hashiranu) verziju. To je sigurnosni standard, ako netko provali u bazu, neće vidjeti stvarne lozinke zaposlenika.

3.1.8 Tablica: GOST

```sql
CREATE TABLE gost (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ime VARCHAR(50) NOT NULL,
    prezime VARCHAR(50) NOT NULL,
    vrsta_dokumenta_id INT NOT NULL,
    broj_dokumenta VARCHAR(30) NOT NULL,
    prebivaliste_grad_id INT NOT NULL,
    prebivaliste_adresa VARCHAR(100) NOT NULL,
    datum_rodjenja DATE,
    drzavljanstvo_id INT NOT NULL,
    status_id INT NOT NULL DEFAULT 1,
    ukupna_potrosnja_cache DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    vip_status BOOLEAN DEFAULT 0,

    CONSTRAINT fk_gost_vrsta_dok FOREIGN KEY (vrsta_dokumenta_id)
       REFERENCES vrsta_dokumenta(id),
    CONSTRAINT fk_gost_drzavljanstvo FOREIGN KEY (drzavljanstvo_id)
        REFERENCES drzava(id),
    CONSTRAINT fk_gost_preb_grad FOREIGN KEY (prebivaliste_grad_id)
        REFERENCES grad(id),
    CONSTRAINT fk_gost_status FOREIGN KEY (status_id) REFERENCES
       status_gosta(id),

    UNIQUE (vrsta_dokumenta_id, broj_dokumenta),
    INDEX idx_gost_potrosnja (ukupna_potrosnja_cache DESC)
);
```

Isječak koda 8: Kreiranje tablice GOST

Svrtha:

Tablica GOST predstavlja središnji registar svih osoba koje su boravile ili će boraviti u objektu. Njena svrha nadilazi samo čuvanje imena, ona služi za:

1. Zakonsku evidenciju: Pohranjuje podatke o dokumentima nužne sa prijavu u sustav
2. CRM (upravljanje odnosima): Prati status lojalnosti i potrošnju kako bi se gostima pružila personalizirana usluga

3. Operativno poslovanje: povezuje gosta s rezervacijama i računima

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator gosta u bazi.
- ime, prezime (VARCHAR(50), NOT NULL): Osnovni podaci za identifikaciju i komunikaciju.
- vrsta_dokumenta_id (INT, FOREIGN KEY) i broj_dokumenta (VARCHAR): Par podataka koji identificira gosta. Ovi podaci su zakonski obavezni za prijavu boravka. Postoji UNIQUE ograničenje na ova dva polja zajedno, što sprječava da se isti dokument unese dva puta za različite osobe.
- prebivaliste_grad_id (INT, FOREIGN KEY) i adresa : Lokacija stanovanja gosta. Bitno za npr. statistiku.
- drzavljanstvo_id (INT, FOREIGN KEY): Podatak o državljanstvu, prijava stranaca.
- status_id (INT, FOREIGN KEY): Poveznica za tablicu STATUS_GOSTA. Odreduje status gosta.
- ukupna_potrosnja_cache (DECIMAL(12,2), DEFAULT 0.00): Polje za optimizaciju. Zbroj svih računa za specifičnog gost se čuvaju ovdje. Uvid u trenutnu vrijednost gosta. Ažurira se prilikom svakog novog računa.
- (BOOLEAN, DEFAULT 0): Brza oznaka za osoblje da ovaj gost zahtijeva poseban tretman, neovisno o automatskom statusu.

3.2 Definiranje ponude i resursa

3.2.1 Tablica: TIP_SOBE

```sql
CREATE TABLE tip_sobe (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL,
    opis TEXT,
    standardni_kapacitet INT NOT NULL
);
```

Isječak koda 9: Kreiranje tablice TIP_SOBE

Svrha: Tablica TIP_SOBE definira kategorije smještajnih jedinica (npr. jednokrevetna, dvokrevetna, predsjednički apartman). Njena svrha je grupiranje soba prema zajedničkim karakteristikama, što služi kao osnova za definiranje cjenika (različiti tipovi imaju različite cijene) i upravljanje kapacitetima prilikom rezervacije.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator tipa sobe. Služi za povezivanje konkretnih soba i cjenika s njihovim tipom.
- naziv (VARCHAR(50), NOT NULL): Naziv kategorije (npr. "Standardna dvokrevetna"). Obavezno polje koje se prikazuje gostima i recepcionarima.
- opis (TEXT, NULL): Detaljniji opis karakteristika sobe (npr. kvadratura, pogled), opcionalno polje.
- standardni_kapacitet (INT, NOT NULL): Broj osoba za koji je soba predviđena. Ovaj podatak je ključan za logiku rezervacijskog sustava kako bi se spriječilo da se u sobu smjesti više osoba od dozvoljenog broja.

3.2.2 Tablica: SOBA

```sql
CREATE TABLE soba (
    id INT AUTO_INCREMENT PRIMARY KEY,
    broj INT NOT NULL UNIQUE,
    tip_sobe_id INT NOT NULL,
    kapacitet_osoba INT NOT NULL,
    kat INT NOT NULL,
    minibar BOOLEAN NOT NULL DEFAULT 0,
    balkon BOOLEAN NOT NULL DEFAULT 0,
    status VARCHAR(20) NOT NULL DEFAULT 'SLOBODNA',
    CONSTRAINT chk_soba_status CHECK (status IN ('SLOBODNA','
        ZAUZETA','IZVAN_FUNKCIJE', 'CISCENJE')),
    CONSTRAINT fk_soba_tip FOREIGN KEY (tip_sobe_id) REFERENCES
        tip_sobe(id)
);
```

Isječak koda 10: Kreiranje tablice SOBA

Svrha: Tablica SOBA predstavlja fizičke prostorije u hotelu. Svaka soba je jedinstvena jedinica koja se iznajmljuje gostima. Tablica prati statičke karakteristike (broj, kat, oprema)

i dinamički status (čisto/prljavo/zauzeto), što je ključno za rad recepcije i domaćinstva u realnom vremenu.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Tehnički identifikator sobe u bazi.
- broj (INT, NOT NULL, UNIQUE): Stvarni broj sobe na vratima. Mora biti jedinstven kako bi gosti i osoblje znali točnu lokaciju.
- tip_sobe_id (INT, NOT NULL, FOREIGN KEY): Poveznica na tablicu TIP_SOBE. Definira kojoj kategoriji soba pripada.
- kapacitet_osoba (INT, NOT NULL): Maksimalan broj osoba za ovu specifičnu sobu. Iako tip sobe ima standardni kapacitet, neka soba može imati pomoćni ležaj, pa se ovaj podatak ovdje definira preciznije.
- kat (INT, NOT NULL): Informacija o lokaciji sobe unutar zgrade.
- minibar, balkon (BOOLEAN): Oznake (zastavice) ima li soba dodatne sadržaje.
- status (VARCHAR(20), NOT NULL): Trenutno stanje sobe (npr. 'CISCENJE', 'SLOBODNA'). Ovo je ključno za recepciju kako ne bi dodijelili sobu koja još nije očišćena.

3.2.3 Tablica: CJENIK_SOBA

```sql
CREATE TABLE cjenik_soba (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tip_sobe_id INT NOT NULL,
    datum_od DATE NOT NULL,
    datum_do DATE NOT NULL,
    cijena_nocenja NUMERIC(10,2) NOT NULL,
    boravisna_pristojba_po_osobi NUMERIC(10,2) NOT NULL,
    aktivan BOOLEAN NOT NULL DEFAULT 1,
    CONSTRAINT fk_cjenik_soba FOREIGN KEY (tip_sobe_id) REFERENCES
        tip_sobe(id)
);
```

Isječak koda 11: Kreiranje tablice CJENIK_SOBA

Svrha: Tablica CJENIK_SOBA omogućuje definiranje cijena smještaja ovisno o vremenskom periodu (sezonalnost). Umjesto fiksne cijene, hotel može definirati različite cijene za predsezonu, sezonu i postsezonu za svaki tip sobe. Ovo je ključno za maksimizaciju prihoda i prilagodbu tržištu.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator stavke cjenika.
- tip_sobe_id (INT, NOT NULL, FOREIGN KEY): Poveznica na tip sobe za koji se cijena definira.

- datum_od, datum_do (DATE, NOT NULL): Vremenski okvir u kojem vrijedi ova cijena. Sustav prilikom rezervacije provjerava u koji datumski raspon pada željeni termin i povlači odgovarajuću cijenu.
- cijena_nocenja (NUMERIC(10,2), NOT NULL): Osnovna cijena jednog noćenja u definiranom periodu.
- boravisna_pristojba_po_osobi (NUMERIC(10,2), NOT NULL): Iznos zakonske pristojbe koja se naplaćuje povrh cijene sobe. Odvojena je jer se taj novac prosljeđuje državi/turističkoj zajednici.
- aktivan (BOOLEAN, DEFAULT 1): Oznaka koristi li se ovaj cjenik trenutno. Omogućuje povijesni pregled starih cijena bez brisanja podataka.

3.2.4 Tablica: USLUGA

```sql
CREATE TABLE usluga (
    id INT AUTO_INCREMENT PRIMARY KEY,
    kategorija_id INT NULL,
    naziv VARCHAR(50) NOT NULL,
    opis TEXT,
    jedinica_mjere VARCHAR(20) DEFAULT 'kom',
    cijena_trenutna NUMERIC(10,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_usluga_kat FOREIGN KEY (kategorija_id) REFERENCES
        kategorija_usluge(id)
);
```

Isječak koda 12: Kreiranje tablice USLUGA

Svrha: Tablica USLUGA definira sve stavke koje se mogu prodati gostu i pojaviti na računu. To uključuje hranu i piće u restoranu, wellness tretmane, najam bicikala i slično. Ova tablica je cjenik za prodaju, za razliku od tablice ARTIKL koja služi za skladište.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator usluge.
- kategorija_id (INT, NULL, FOREIGN KEY): Poveznica na kategoriju (npr. "Topli napitci").
- naziv (VARCHAR(50), NOT NULL): Komercijalni naziv usluge (npr. "Espresso").
- opis (TEXT): Detaljan opis usluge.
- jedinica_mjere (VARCHAR(20), DEFAULT 'kom'): Mjerna jedinica za prodaju (npr. 'kom', 'sat').
- cijena_trenutna (NUMERIC(10,2), NOT NULL): Trenutna prodajna cijena s porezom. Sustav koristi ovu cijenu prilikom kreiranja novih narudžbi.

3.2.5 Tablica: RESTORAN_STOL

```sql
CREATE TABLE restoran_stol (
    id INT AUTO_INCREMENT PRIMARY KEY,
    broj_stola INT NOT NULL UNIQUE,
```

```
    broj_mjesta INT NOT NULL,
    lokacija VARCHAR(50)
);
```

Isječak koda 13: Kreiranje tablice RESTORAN_STOL

Svrha: Tablica RESTORAN_STOL definira fizički raspored sjedenja u restoranu. Svaki zapis predstavlja jedan stol na koji konobari mogu otvarati narudžbe. Tablica je nužna za vizualizaciju restorana u aplikaciji i upravljanje zauzećem stolova.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Tehnički identifikator stola.
- broj_stola (INT, NOT NULL, UNIQUE): Oznaka stola u restoranu (npr. stol broj 10). Mora biti jedinstven kako ne bi došlo do zabune kod naručivanja.
- broj_mjesta (INT, NOT NULL): Kapacitet stola. Bitno za hostese ili konobare prilikom smještanja većih grupa gostiju.
- lokacija (VARCHAR(50)): Opis gdje se stol nalazi (npr. "Terasa", "Glavna sala", "Uz prozor"). Pomaže u organizaciji rejona za konobare.

3.2.6 Tablica: ARTIKL

```
CREATE TABLE artikl (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL,
    stanje_zaliha DECIMAL(10,2) DEFAULT 0,
    jedinica_mjere VARCHAR(10),
    nabavna_cijena DECIMAL(10,2)
);
```

Isječak koda 14: Kreiranje tablice ARTIKL

Svrha: Tablica ARTIKL služi za vođenje materijalnog knjigovodstva i skladišta. Ovdje se prate sirovine (npr. brašno, meso, pića u bocama) koje se troše pružanjem usluga. Ključna je za praćenje troškova poslovanja i pravovremeno naručivanje nove robe.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator artikla u skladištu.
- naziv (VARCHAR(50), NOT NULL): Naziv sirovine ili proizvoda (npr. "Coca Cola 0.25L - povratna boca").
- stanje_zaliha (DECIMAL(10,2), DEFAULT 0): Trenutna količina na skladištu. Sustav automatski umanjuje ovo stanje kada se proda povezana usluga.
- jedinica_mjere (VARCHAR(10)): Jedinica u kojoj se vodi skladište (npr. 'kg', 'l', 'kom').
- nabavna_cijena (DECIMAL(10,2)): Cijena po kojoj hotel kupuje robu od dobavljača. Služi za izračun marže (razlika između nabavne i prodajne cijene).

3.2.7 Tablica: NORMATIV

```sql
CREATE TABLE normativ (
    usluga_id INT NOT NULL,
    artikl_id INT NOT NULL,
    kolicina_potrosnje DECIMAL(10,4) NOT NULL,
    PRIMARY KEY (usluga_id, artikl_id),
    CONSTRAINT fk_norm_usluga FOREIGN KEY (usluga_id) REFERENCES
       usluga(id),
    CONSTRAINT fk_norm_artikl FOREIGN KEY (artikl_id) REFERENCES
       artikl(id)
);
```

Isječak koda 15: Kreiranje tablice NORMATIV

Svrha: Tablica NORMATIV predstavlja veznu tablicu ("recepturu") između usluga i artikala. Ona definira koliko se kojeg artikla sa skladišta troši za jednu prodanu uslugu. Na primjer, za uslugu "Kava s mlijekom", normativ definira da se troši 0.007 kg kave i 0.1 litra mlijeka. Ovo omogućuje automatizaciju skladišta – prodajom kave sustav sam zna što treba oduzeti sa zaliha.

Atributi:

- usluga_id (INT, NOT NULL, FOREIGN KEY): Poveznica na uslugu koja se prodaje.
- artikl_id (INT, NOT NULL, FOREIGN KEY): Poveznica na artikl (sirovinu) koji se troši.
- kolicina_potrosnje (DECIMAL(10,4), NOT NULL): Točna količina artikla potrebna za jednu jedinicu usluge. Koristi se veća preciznost (4 decimale) jer su količine često vrlo male (npr. začini, gramaža kave).
- PRIMARY KEY (usluga_id, artikl_id): Složeni primarni ključ koji osigurava da se isti artikl ne može dva puta definirati unutar iste usluge.

3.3 Glavni proces: Rezervacije

3.3.1 Tablica: PROMOCIJA

```sql
CREATE TABLE promocija (
    id INT  PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL,
    kod_kupona VARCHAR(20) UNIQUE,
    popust_postotak DECIMAL(5,2),
    datum_pocetka DATE,
    datum_zavrsetka DATE,
    aktivna BOOLEAN DEFAULT 1
);
```

Isječak koda 16: Kreiranje tablice PROMOCIJA

Svrha: Tablica PROMOCIJA služi za upravljanje marketinškim kampanjama i popustima. Omogućuje hotelu definiranje posebnih ponuda (npr. "Black Friday", "Rani booking") ili kodova za popust. Povezivanjem ove tablice s rezervacijama, sustav automatski obračunava nižu cijenu, čime se smanjuje mogućnost greške kod ručnog računanja popusta.

Atributi:

- id (INT, PRIMARY KEY): Jedinstveni identifikator promocije.
- naziv (VARCHAR(50), NOT NULL): Interni naziv kampanje (npr. "Zimski popust 2024").
- kod_kupona (VARCHAR(20), UNIQUE): Kod koji gosti mogu upisati prilikom rezervacije (npr. "ZIMA20"). Jedinstven je kako bi se spriječilo dupliciranje.
- popust_postotak (DECIMAL(5,2)): Iznos popusta u postocima (npr. 15.00 za 15% popusta).
- datum_pocetka, datum_zavrsetka (DATE): Vremenski period u kojem je promocija valjana.
- aktivna (BOOLEAN, DEFAULT 1): Prekidač za brzo uključivanje ili isključivanje promocije bez brisanja iz baze.

3.3.2 Tablica: REZERVACIJA

```sql
CREATE TABLE rezervacija (
    id INT AUTO_INCREMENT PRIMARY KEY,
    gost_nositelj_id INT NOT NULL,
    zaposlenik_id INT NOT NULL,
    soba_id INT NOT NULL,
    promocija_id INT NULL,

    datum_rezervacije TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    pocetak_datum DATE NOT NULL,
    kraj_datum DATE NOT NULL,

    vrijeme_check_in DATETIME NULL,
    vrijeme_check_out DATETIME NULL,
```

```
    broj_osoba INT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'POTVRDJENA',
    napomena TEXT,

    CONSTRAINT chk_rezervacija_status CHECK (status IN ('POTVRDJENA
        ','OTKAZANA','U_TIJEKU','ZAVRSENA')),
    CONSTRAINT fk_rez_gost FOREIGN KEY (gost_nositelj_id)
        REFERENCES gost(id),
    CONSTRAINT fk_rez_zaposlenik FOREIGN KEY (zaposlenik_id)
        REFERENCES zaposlenik(id),
    CONSTRAINT fk_rez_soba FOREIGN KEY (soba_id) REFERENCES soba(id
        ),
    CONSTRAINT fk_rez_promo FOREIGN KEY (promocija_id) REFERENCES
        promocija(id)
);
```

Isječak koda 17: Kreiranje tablice REZERVACIJA

Svrha: Tablica REZERVACIJA je središnja operativna tablica sustava. Ona povezuje gosta (tko?), sobu (gdje?) i vrijeme (kada?). Osim planiranih datuma, ova tablica prati stvarni životni ciklus boravka kroz status i vremena prijave/odjave (Check-in/Check-out). Svi financijski i operativni izvještaji temelje se na podacima iz ove tablice.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni broj rezervacije (Booking ID).
- gost_nositelj_id (INT, NOT NULL, FOREIGN KEY): Poveznica na gosta koji je napravio rezervaciju i koji je odgovoran za plaćanje.
- zaposlenik_id (INT, NOT NULL, FOREIGN KEY): Zaposlenik koji je unio rezervaciju u sustav (za praćenje odgovornosti).
- soba_id (INT, NOT NULL, FOREIGN KEY): Dodijeljena soba.
- promocija_id (INT, NULL, FOREIGN KEY): Poveznica na promociju ako je korišten popust.
- datum_rezervacije (TIMESTAMP): Trenutak kada je rezervacija nastala.
- pocetak_datum, kraj_datum (DATE, NOT NULL): Planirani datumi dolaska i odlaska.
- vrijeme_check_in, vrijeme_check_out (DATETIME, NULL): Točna vremena kada je gost fizički preuzeo i vratio ključ. Bitno za naplatu kasnog odčeka (late check-out).
- status (VARCHAR(20)): Trenutno stanje rezervacije (npr. 'U_TIJEKU' znači da je gost trenutno u hotelu). Definirano CHECK ograničenjem kako bi se spriječili neispravni statusi.

3.3.3 Tablica: REZERVACIJA_GOST

```
CREATE TABLE rezervacija_gost (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    rezervacija_id INT NOT NULL,
    gost_id INT NOT NULL,
    uloga VARCHAR(20) DEFAULT 'GOST',
    CONSTRAINT fk_rezgost_rez FOREIGN KEY (rezervacija_id)
       REFERENCES rezervacija(id),
    CONSTRAINT fk_rezgost_gost FOREIGN KEY (gost_id) REFERENCES
       gost(id),
    UNIQUE (rezervacija_id, gost_id)
);
```

Isječak koda 18: Kreiranje tablice REZERVACIJA_GOST

Svrha: Tablica REZERVACIJA_GOST povezuje rezervaciju sa svim osobama koje stvarno borave u sobi, a ne samo s osobom koja je napravila rezervaciju (nositeljem). Ovo je ključno za zakonsku prijavu gostiju (eVisitor) jer hotel mora imati podatke o svakom pojedincu u sobi, uključujući djecu i suputnike.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Tehnički identifikator zapisa.
- rezervacija_id (INT, NOT NULL, FOREIGN KEY): Poveznica na rezervaciju.
- gost_id (INT, NOT NULL, FOREIGN KEY): Poveznica na profil gosta u bazi.
- uloga (VARCHAR(20), DEFAULT 'GOST'): Opisuje ulogu osobe u toj rezervaciji (npr. 'NOSITELJ', 'PRATNJA', 'DIJETE').
- UNIQUE (rezervacija_id, gost_id): Ograničenje koje sprječava da se isti gost greškom doda dva puta na istu rezervaciju.

3.3.4 Tablica: LOG_REZERVACIJE

```
CREATE TABLE log_rezervacije (
    id INT AUTO_INCREMENT PRIMARY KEY,
    rezervacija_id INT NOT NULL,
    stari_status VARCHAR(20),
    novi_status VARCHAR(20),
    vrijeme_promjene TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    korisnik_db VARCHAR(50),
    CONSTRAINT fk_rezervacija_id FOREIGN KEY (rezervacija_id)
       REFERENCES rezervacija(id)
);
```

Isječak koda 19: Kreiranje tablice LOG_REZERVACIJE

Svrha: Tablica LOG_REZERVACIJE služi za reviziju i sigurnost (Audit Trail). Ona automatski bilježi svaku promjenu statusa rezervacije. Ako dođe do spora (npr. gost tvrdi da je otkazao rezervaciju na vrijeme, a hotel tvrdi da nije), ova tablica pruža točan vremenski slijed događaja. Obično se puni putem okidača (triggera) u bazi.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator log zapisa.

- rezervacija_id (INT, NOT NULL, FOREIGN KEY): Poveznica na rezervaciju koja se mijenjala.
- stari_status, novi_status (VARCHAR(20)): Bilježi stanje "prije" i "poslije" promjene (npr. iz 'POTVRDJENA' u 'OTKAZANA').
- vrijeme_promjene (TIMESTAMP): Točno vrijeme kada se promjena dogodila.
- korisnik_db (VARCHAR(50)): Ime korisnika baze ili aplikacije koji je inicirao promjenu.

3.4 Operativno poslovanje (tokom boravka)

3.4.1 Tablica: RESTORAN_NARUDZBA

```sql
CREATE TABLE restoran_narudzba (
    id INT AUTO_INCREMENT PRIMARY KEY,
    zaposlenik_id INT NOT NULL,
    restoran_stol_id INT NOT NULL,
    datum_otvaranja TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    datum_zatvaranja TIMESTAMP NULL,
    status VARCHAR(20) DEFAULT 'OTVORENA',

    CONSTRAINT fk_rest_konobar FOREIGN KEY (zaposlenik_id)
       REFERENCES zaposlenik(id),
    CONSTRAINT fk_rest_stol FOREIGN KEY (restoran_stol_id)
       REFERENCES restoran_stol(id)
);
```

Isječak koda 20: Kreiranje tablice RESTORAN_NARUDZBA

Svrha: Tablica RESTORAN_NARUDZBA predstavlja "otvoreni račun" na određenom stolu. Ona grupira sve stavke koje gosti naruče tijekom jednog posjeta restoranu. Omogućuje konobarima da dodaju stavke na isti stol tijekom vremena prije nego što se izda finalni račun. Također prati tko poslužuje stol i koliko dugo su gosti boravili.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator narudžbe.
- zaposlenik_id (INT, NOT NULL, FOREIGN KEY): Konobar koji je otvorio stol.
- restoran_stol_id (INT, NOT NULL, FOREIGN KEY): Stol na koji se narudžba odnosi.
- datum_otvaranja (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP): Vrijeme kada su gosti sjeli i naručili prvo piće.
- datum_zatvaranja (TIMESTAMP, NULL): Vrijeme kada je narudžba zaključena (plaćena).
- status (VARCHAR(20)): Trenutno stanje narudžbe (npr. 'OTVORENA', 'PLACENA').

3.4.2 Tablica: RESTORAN_STAVKA

```sql
CREATE TABLE restoran_stavka (
    id INT AUTO_INCREMENT PRIMARY KEY,
    narudzba_id INT NOT NULL,
    usluga_id INT NOT NULL,
    kolicina INT NOT NULL DEFAULT 1,
    cijena_u_trenutku NUMERIC(10,2) NOT NULL,
    status_pripreme VARCHAR(20) DEFAULT 'NARUCENO',

```

```
    CONSTRAINT fk_rest_stavka_nar FOREIGN KEY (narudzba_id)
       REFERENCES restoran_narudzba(id),
    CONSTRAINT fk_rest_stavka_usl FOREIGN KEY (usluga_id)
       REFERENCES usluga(id),

    CONSTRAINT chk_rest_kolicina CHECK (kolicina > 0),
    CONSTRAINT chk_rest_cijena CHECK (cijena_u_trenutku >= 0)
);
```

Isječak koda 21: Kreiranje tablice RESTORAN_STAVKA

Svrha: Tablica RESTORAN_STAVKA sadrži pojedinačne linije narudžbe (npr. "2x Kava", "1x Pizza"). Povezuje narudžbu s uslugom iz cjenika. Ova tablica je ključna za komunikaciju s kuhinjom/šankom jer status pripreme govori osoblju što treba pripremiti, a što je već posluženo.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator stavke.
- narudzba_id (INT, NOT NULL, FOREIGN KEY): Poveznica na glavnu narudžbu (stol).
- usluga_id (INT, NOT NULL, FOREIGN KEY): Poveznica na uslugu (što je naručeno).
- kolicina (INT, NOT NULL): Broj porcija.
- cijena_u_trenutku (NUMERIC(10,2), NOT NULL): Cijena usluge u trenutku naručivanja. Ovo je važno pohraniti jer se cijena u cjeniku (tablica usluga) može promijeniti u budućnosti, ali ova narudžba mora ostati po staroj cijeni.
- status_pripreme (VARCHAR(20)): Status za kuhinju (npr. 'NARUCENO' -> 'PRIPREMA' -> 'POSLUZENO').

);

3.4.3 Tablica: CISCENJE_DNEVNI_NALOG

```
CREATE TABLE ciscenje_dnevni_nalog (
    id INT AUTO_INCREMENT PRIMARY KEY,
    zaposlenik_id INT NOT NULL,
    rezervacija_id INT NOT NULL,
    datum_naloga TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    prijavljena_steta BOOLEAN NOT NULL DEFAULT 0,
    opis_stete TEXT,
    obavljeno BOOLEAN DEFAULT 0,
    CONSTRAINT fk_ciscenje_zapos FOREIGN KEY (zaposlenik_id)
       REFERENCES zaposlenik(id),
    CONSTRAINT fk_ciscenje_rez FOREIGN KEY (rezervacija_id)
       REFERENCES rezervacija(id)
);
```

Isječak koda 22: Kreiranje tablice CISCENJE_DNEVNI_NALOG

3.4.4 Tablica: SERVIS_DNEVNI_NALOG

```sql
CREATE TABLE servis_dnevni_nalog (
    id INT AUTO_INCREMENT PRIMARY KEY,
    zaposlenik_id INT NOT NULL,
    soba_id INT NOT NULL,
    datum_naloga TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    korisnik_placa BOOLEAN NOT NULL DEFAULT 0,
    opis TEXT,
    rijeseno BOOLEAN DEFAULT 0,
    CONSTRAINT fk_servis_zapos FOREIGN KEY (zaposlenik_id)
       REFERENCES zaposlenik(id),
    CONSTRAINT fk_servis_soba FOREIGN KEY (soba_id) REFERENCES soba
       (id)
);
```

Isječak koda 23: Kreiranje tablice SERVIS_DNEVNI_NALOG

3.5 Financije i Zaključenje boravka

3.5.1 Tablica: RACUN

```sql
CREATE TABLE racun (
    id INT AUTO_INCREMENT PRIMARY KEY,
    gost_id INT NULL,
    zaposlenik_id INT NOT NULL,
    rezervacija_id INT NULL,
    datum_izdavanja TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    nacin_placanja VARCHAR(20) NOT NULL,
    ukupni_iznos DECIMAL(10,2) NOT NULL,
    jir VARCHAR(36),
    zki VARCHAR(32),
    CONSTRAINT fk_racun_gost FOREIGN KEY (gost_id) REFERENCES gost(
        id),
    CONSTRAINT fk_racun_zaposlenik FOREIGN KEY (zaposlenik_id)
        REFERENCES zaposlenik(id),
    CONSTRAINT fk_racun_rez FOREIGN KEY (rezervacija_id) REFERENCES
         rezervacija(id)
);
```

Isječak koda 24: Kreiranje tablice RACUN

Svrha: Tablica RACUN predstavlja finalni financijski dokument koji se izdaje gostu prilikom plaćanja. Za razliku od narudžbe koja je privremena, račun je trajni pravni dokument. Sadrži podatke nužne za fiskalizaciju (JIR, ZKI) i zakonsku evidenciju prihoda hotela.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni broj računa.
- gost_id (INT, NULL, FOREIGN KEY): Gost na kojeg račun glasi (može biti NULL za anonimne prolazne goste u baru).
- zaposlenik_id (INT, NOT NULL, FOREIGN KEY): Osoba koja je izdala i naplatila račun.
- rezervacija_id (INT, NULL, FOREIGN KEY): Poveznica ako se račun odnosi na smještaj.
- datum_izdavanja (TIMESTAMP): Točno vrijeme nastanka porezne obveze.
- nacin_placanja (VARCHAR(20), NOT NULL): Metoda plaćanja (npr. 'GOTOVINA', 'KARTICA', 'VIRMAN').
- ukupni_iznos (DECIMAL(10,2), NOT NULL): Finalna suma za naplatu.
- jir, zki (VARCHAR): Jedinstveni identifikatori računa dobiveni od Porezne uprave (fiskalizacija).

3.5.2 Tablica: STAVKA_RACUNA

```sql
CREATE TABLE stavka_racuna (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    racun_id INT NOT NULL,
    usluga_id INT NOT NULL,
    kolicina INT NOT NULL,
    cijena_jedinicna DECIMAL(10,2) NOT NULL,
    popust_postotak DECIMAL(5,2) DEFAULT 0,
    ukupni_iznos_stavke DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_stavka_racun FOREIGN KEY (racun_id) REFERENCES
       racun(id),
    CONSTRAINT fk_stavka_usluga FOREIGN KEY (usluga_id) REFERENCES
       usluga(id)
);
```

Isječak koda 25: Kreiranje tablice STAVKA_RACUNA

Svrha: Tablica STAVKA_RACUNA detaljno specificira što je točno naplaćeno na računu. Svaki redak na isprintanom računu odgovara jednom retku u ovoj tablici. Ovdje se trajno bilježi cijena koja je vrijedila u trenutku prodaje, neovisno o kasnijim promjenama cjenika.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator stavke.
- racun_id (INT, NOT NULL, FOREIGN KEY): Poveznica na zaglavlje računa.
- usluga_id (INT, NOT NULL, FOREIGN KEY): Što je prodano (npr. "Noćenje", "Coca Cola").
- kolicina (INT, NOT NULL): Količina prodane usluge.
- cijena_jedinicna (DECIMAL(10,2), NOT NULL): Cijena jedne jedinice usluge u trenutku izdavanja računa.
- popust_postotak (DECIMAL(5,2)): Primijenjeni popust na ovu stavku (npr. 10%).
- ukupni_iznos_stavke (DECIMAL(10,2), NOT NULL): Izračunata vrijednost: (količina * cijena) umanjeno za popust.

3.5.3 Tablica: RECENZIJA

```
CREATE TABLE recenzija (
    id INT AUTO_INCREMENT PRIMARY KEY,
    gost_id INT NOT NULL,
    soba_id INT NULL,
    ocjena INT NOT NULL,
    komentar TEXT,
    datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_recenzija_gost FOREIGN KEY (gost_id) REFERENCES
       gost(id),
    CONSTRAINT fk_recenzija_soba FOREIGN KEY (soba_id) REFERENCES
       soba(id),
    CONSTRAINT chk_ocjena CHECK (ocjena BETWEEN 1 AND 5)
);
```

Isječak koda 26: Kreiranje tablice RECENZIJA

Svrha: Tablica RECENZIJA služi za prikupljanje povratnih informacija od gostiju. Ovi podaci su ključni za upravljanje kvalitetom (Quality Assurance). Analizom ocjena i komentara menadžment može uočiti probleme u uslugama (npr. "loše čišćenje", "spora posluga") ili nagraditi izvrsne zaposlenike.

Atributi:

- id (INT, AUTO_INCREMENT, PRIMARY KEY): Jedinstveni identifikator recenzije.
- gost_id (INT, NOT NULL, FOREIGN KEY): Gost koji je ostavio recenziju.
- soba_id (INT, NULL, FOREIGN KEY): Opcionalna poveznica na sobu u kojoj je gost boravio.
- ocjena (INT, NOT NULL): Numerička ocjena zadovoljstva (obično od 1 do 5). Osigurano CHECK ograničenjem.
- komentar (TEXT): Tekstualni opis iskustva, pohvale ili pritužbe.
- datum (TIMESTAMP): Vrijeme kada je recenzija napisana.

4. Eugen B.

4.1 Procedure (Eng. Procedures)

4.1.1 Procedura: Otkazivanje rezervacije

```
DELIMITER //
CREATE PROCEDURE otkazi_rezervaciju(IN p_rezervacija_id INT)
BEGIN
    DECLARE v_trenutni_status VARCHAR(20);

    SELECT status INTO v_trenutni_status
    FROM rezervacija
    WHERE id = p_rezervacija_id;

    IF v_trenutni_status = 'POTVRDJENA' THEN
        UPDATE rezervacija
        SET status = 'OTKAZANA'
        WHERE id = p_rezervacija_id;

        SELECT CONCAT('Rezervacija ', p_rezervacija_id, ' je
           otkazana.') AS Msg;
    ELSE
        SELECT CONCAT('Rezervacija ', p_rezervacija_id,
                      ' se ne moze otkazati jer je u statusu: ',
                      IFNULL(v_trenutni_status, 'NEPOSTOJI')) AS
                         Msg;
    END IF;
END //
DELIMITER ;

```

```
CALL otkazi_rezervaciju(40);
SELECT * FROM rezervacija WHERE id = 40;
CALL otkazi_rezervaciju(1);
```

Isječak koda 27: Procedura za otkazivanje rezervacije

Ova procedura nam pruža funkcionalnost otkazivanja rezervacije koja još nije započela te osigurava da se otkazivanje može izvršiti isključivo nad rezervacijama koje su najavljene te trenutno u bazi sa statusom 'POTVRDJENA', čime se sprječava mijenjanja statusa već gotovih, otkazanih ili rezervacija koje su još u trajanju. Procedura prima jedan ulazni parametar:

- 'p_rezervacija_id' - - (IN) ID rezervacije koju korisnik baze želi otkazati

1. Procedura prvo dohvaća trenutni status rezervacije iz tablice 'rezervacija' na temelju prosljeđenog ID-ja te ga pohranjuje u lokalnu varijablu 'v_trenutni_status'.
2. Procedura provodi provjera pomoću 'IF' grananja:

   Ukoliko je trenutni status jednak 'POTVRDJENA' , procedura izvršava naredbu ažuriranja koja mijenja status te pojedine rezervacije u 'OTKAZANA'. Nakon toga ispisuje poruku korisniku baze da je rezervacija uspješno otkazana.

   Ako trenutni status nije 'POTVRDJENA' ili rezervacija sa unesenim ID-jem ne postoji, procedura ne radi izmjenu statusa, već ispisuje poruku korisniku baze koja ga obavještava da se rezervacija ne može otkazati jer je završena, u tijeku, već prethodno otkazana ili uopće ne postoji u bazi.

Cilj procedure:

Cilj ove procedure je automatizirati otkazivanje pojedine rezervacije uz ugrađenu provjeru i poštivanje poslovnih pravila, osiguravajući integritet podataka unutar baze, sprječavajući izmjenu rezervacija koje se ne mogu otkazati ili ne postoje unutar same baze.

4.1.2 Procedura: Brisanje podataka iz loga kojima je prošlo više od 90 dana

```
DELIMITER //

CREATE PROCEDURE ocisti_stare_log_rez(
    IN p_starije_od_dana INT,
    OUT p_obrisano_redaka INT
)
BEGIN
    DECLARE v_granicni_datum DATETIME;

    SET v_granicni_datum = NOW() - INTERVAL p_starije_od_dana DAY;

    SELECT COUNT(*) INTO p_obrisano_redaka
    FROM log_rezervacije
    WHERE vrijeme_promjene < v_granicni_datum;

        SELECT
        id AS 'ID␣Log',
```

```
        rezervacija_id AS 'ID Rez',
        stari_status,
        novi_status,
        korisnik_db,
        vrijeme_promjene AS 'Stari datum'
    FROM log_rezervacije
    WHERE vrijeme_promjene < v_granicni_datum;

    DELETE FROM log_rezervacije
    WHERE vrijeme_promjene < v_granicni_datum;
END //
DELIMITER ;
```

Isječak koda 28: Procedura za čišćenje starih logova

Procedura ‘ocisti_stare_log_rez‘, služi za periodično održavanje baze podataka tako što briše stare zapise iz tablice ‘log_rezervacije‘. Ona briše zapise starije od definiranog broja dana zatim prikazuje izbrisane podatke te vraća povratnu informaciju o broju obrisanih redaka iz arhive. Procedura prima dva parametra (jedan ulazni i jedan izlazni):

- ‘p_starije_od_dana‘ - - (IN): Broj dana koji definira granicu starosti podataka (npr. 90 dana).
- ‘p_obrisano_redaka‘ - - (OUT): Izlazni parametar u koji se pohranjuje ukupan broj zapisa koji su obrisani iz arhive nakon pozivanja procedure

1. Određivanje graničnog datuma:

   Procedura računa točan datum i vrijeme (‘v_granicni_datum‘) oduzimanjem unesenog broja dana ‘p_starije_od_dana‘ od trenutnog timestamp-a‘NOW()‘. Svi zapisi stariji od ovog datuma smatraju se zastarjelima te su kao takvi spremni za brisanje iz arhive.

2. Brojanje zapisa za brisanje:

   Prije samog brisanja, procedura prvo broji koliko zapisa zadovoljava postavljeni uvjet da je datum promjene manji od graničnog datuma te taj broj pohranjuje u izlazni parametar ‘p_obrisano_redaka‘.

3. Prikaz podataka:

   Procedura pri svom izvršavanju radi ‘SELECT‘ upit kako bi prikazala detalje zapisa (ID_Log, ID_Rez, stari status, novi status, korisnika baze i datum izmjene) koji će biti obrisani. Ovo služi kao sigurnosna provjera te pregled podataka koji će biti izbrisani iz baze.

4. Brisanje podataka:

   Na kraju, izvršava se ‘DELETE‘ naredba koja briše sve zapise iz tablice ‘log_rezervacije‘ koji su stariji od izračunatog graničnog datuma.

Cilj procedure:

Cilj procedure je automatizirati čišćenje povijesnih podataka (logova) kako bi se optimizirala sama veličina baze podataka i njene performanse, uz paralelno pružanje povratne informacije o samoj količini I samih obrisanih podataka.

4.2 Okidač (Eng. Trigger)

4.2.1 Okidač: Sprječavanje preklapanja rezervacija

```
DELIMITER //

CREATE TRIGGER trg_dupla_rezervacija_sobe
BEFORE INSERT ON rezervacija
FOR EACH ROW
BEGIN
    DECLARE brojac_rez INT;

    -- Provjera postoje li aktivne rezervacije u tom terminu
    SELECT COUNT(*) INTO brojac_rez
    FROM rezervacija
    WHERE soba_id = NEW.soba_id
      AND status != 'OTKAZANA'
      AND NEW.pocetak_datum < kraj_datum
      AND NEW.kraj_datum > pocetak_datum;

    IF brojac_rez > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Greska: Soba je vec rezervirana u odabranom terminu!';
    END IF;
END //

DELIMITER ;
```

Isječak koda 29: Trigger za sprječavanje preklapanja rezervacija

Ovaj okidač nam pruža blokadu koja sprječava nastanak dvostrukih rezervacija tj. "overbooking" te osigurava da se nova rezervacija može unijeti isključivo ako je odabrana soba u potpunosti slobodna u zatraženom terminu. Time se sprječava preklapanje s već postojećim, aktivnim rezervacijama.

Ovaj okidač se aktivira prije samog unosa novog zapisa u tablicu rezervacija tj. 'BEFORE INSERT'.Funkcionalnost okidačase bazira na provjeri vremenskog preklapanja termina za odabranu sobu u insertu. Cijeli proces se odvija prema sljedećim provjerama:

1. Provjera dostupnosti termina:

   Okidač prvo deklarira lokalnu varijablu 'brojac_rez'. Nakon toga izvršava 'SELECT COUNT' upit nad tablicom rezervacija kako bi utvrdio postoji li već zapis koji zadovoljava tri uvjeta:

   - Da se radi o istoj sobi kao u novom zahtjevu za rezervaciju (NEW.soba_id).

   - Da je status postojeće rezervacije "U_TIJEKU" ili "POTVRĐENA".

   - Da postoji vremensko preklapanje, gdje novi datum dolaska počinje prije završetka postojeće rezervacije, a novi datum odlaska završava nakon početka postojeće rezervacije.Rezultat prebrojavanja pohranjuje se u varijablu 'brojac_rez'.

2. Sprječavanje unosa neispravnih podataka:

U drugom koraku, procedura provodi provjeru pomoću IF grananja nad varijablom 'brojac_rez'.Ukoliko je u usporedbi vrijednost veća od 0 (što znači da je soba zauzeta), okidač aktivira korisnički definiranu SIGNAL SQLSTATE naredbu. Ovime se momentalno prekida unos rezervacije i korisniku baze ispisuje poruka "Greška: Soba je već rezervirana u odabranom terminu!", čime se onemogućuje zapisivanje podatka u bazu koji se krši sa poslovnim pravilom.

Cilj okidača:

Cilj ovog okidača je automatizirati kontrolu dostupnosti smještajnih kapaciteta na najnižoj razini baze, eliminirajući mogućnost ljudske pogreške zaposlenika sa recepcije i osiguravajući da pojedina soba ne može biti prodana dvostruko ukoliko se termini dolaska ili odlaska preklapaju.

4.2.2 Okidač: Promocija se smije unijeti u rezervaciju samo ako je aktivna i vremenski važeća

```
DELIMITER //

CREATE TRIGGER trg_unos_provjera_valjanosti_promocije
BEFORE INSERT ON rezervacija
FOR EACH ROW
BEGIN
    DECLARE v_aktivna INT;
    DECLARE v_promo_vrijedi_od DATE;
    DECLARE v_promo_vrijedi_do DATE;
    DECLARE v_rez_dolazak DATE;
    DECLARE v_rez_odlazak DATE;

    IF NEW.promocija_id IS NOT NULL THEN

        -- Dohvat podataka o promociji
        SELECT aktivna, datum_pocetka, datum_zavrsetka
        INTO v_aktivna, v_promo_vrijedi_od, v_promo_vrijedi_do
        FROM promocija
        WHERE id = NEW.promocija_id;

        SET v_rez_dolazak = NEW.pocetak_datum;
        SET v_rez_odlazak = NEW.kraj_datum;

        IF v_aktivna = 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Odabrana promocija je neaktivna!';
        END IF;

        IF v_rez_dolazak < v_promo_vrijedi_od THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Rezervacija pocinje prije nego sto sama promocija vrijedi!';
        END IF;

```

```
        IF v_rez_odlazak > v_promo_vrijedi_do THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Rezervacija zavrsava nakon
               sto je promocija istekla!';
        END IF;

    END IF;
END //

DELIMITER ;
```

Isječak koda 30: Trigger za provjeru valjanosti promocije pri unosu

Okidačem 'trg_unos_provjera_valjanosti_promocije' kontroliramo primjenu promocija prema sljedećim kriterijima; valjanost same promocije (aktivna ili neaktivna) te provjeru vremenskog perioda njezine primjenjivosti . Njegova svrha je spriječiti gubitak prihoda koji bi nastao primjenom nevažećih, isteklih ili neaktivnih promotivnih akcija prilikom kreiranja nove rezervacije.

Okidač se aktivira prije samog unosa novog zapisa u tablicu rezervacija 'BEFORE INSERT' isključivo uz uvjet da je uz rezervaciju priložen sam ID promocije 'promocija_id IS NOT NULL' , a proces daljnje provjere se odvija kroz sljedeće korake:

1. Dohvat parametara promocije:

   Na temelju unesenog ID-a promocije, okidač iz tablice promocija dohvaća tri ključna podatka:

   - status aktivnosti (aktivna)

   - datume početka

   - završetka važenja promocije.

   Te vrijednosti pohranjuje u lokalne varijable kako bi se mogao usporediti s terminom rezervacije.

2. Provjera valjanosti uvjeta:

   - Provjera aktivnosti promocije:

   Prvo se provjerava je li promocija uopće aktivna (v_aktivna = 0). Ako je promocija u sustavu označena kao neaktivna, okidač odmah blokira unos uz poruku korisniku baze: "Greška: Odabrana promocija je neaktivna!".

   - Provjera datuma početka:

   Zatim se uspoređuje datum dolaska gosta s datumom početka promocije. Ako rezervacija počinje prije nego što promocija službeno vrijedi, sustav također javlja grešku.

   - Provjera datuma završetka:

   Na kraju, provjerava se datum odlaska. Ako rezervacija završava nakon što je promocija istekla, okidač blokira unos jer se prema poslovnom pravilu popust ne može primijeniti na termin izvan promotivnog razdoblja.

Cilj okidača:

Cilj ovog okidača je automatizirati primjenu marketinških I poslovnih pravila bez ručne provjere korisnika baze, osiguravajući da se popusti odobravaju isključivo unutar predefiniranih vremenskih okvira i samo za akcije koje su trenutno odobrene od strane direktora.

4.2.3 Okidač: Provjera valjanosti promocije prilikom ažuriranja rezervacije

```
DELIMITER //

CREATE TRIGGER
   trg_azuriranje_rezervacije_provjera_valjanosti_promocije
BEFORE UPDATE ON rezervacija
FOR EACH ROW
BEGIN
    DECLARE v_aktivna INT;
    DECLARE v_promo_vrijedi_od DATE;
    DECLARE v_promo_vrijedi_do DATE;
    DECLARE v_rez_dolazak DATE;
    DECLARE v_rez_odlazak DATE;

    IF NEW.promocija_id IS NOT NULL THEN
        -- Dohvat podataka o promociji
        SELECT aktivna, datum_pocetka, datum_zavrsetka
        INTO v_aktivna, v_promo_vrijedi_od, v_promo_vrijedi_do
        FROM promocija
        WHERE id = NEW.promocija_id;

        SET v_rez_dolazak = NEW.pocetak_datum;
        SET v_rez_odlazak = NEW.kraj_datum;

        IF v_aktivna = 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Odabrana promocija je
               neaktivna!';
        END IF;

        IF v_rez_dolazak < v_promo_vrijedi_od THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Rezervacija pocinje prije
               nego sto sama promocija pocinje vrijediti!';
        END IF;

        IF v_rez_odlazak > v_promo_vrijedi_do THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Rezervacija zavrsava nakon
               sto je promocija istekla!';
        END IF;
    END IF;
END //

```

```
DELIMITER ;
```

Isječak koda 31: Trigger za provjeru valjanosti promocije pri ažuriranju

Okidač trg_ažuriranje_rezervacije_provjera_valjanosti_promocije je nadogradnja prethodnog okidača. On sprječava situaciju u kojoj bi korisnik baze mogao pokušati zaobići pravila tako da kreira rezervaciju bez promocije, a zatim naknadno preko ažuriranja, doda neaktivnu ili vremenski nevažeću promociju. Ovaj se okidač aktivira prije izmjene postojećeg zapisa u tablici rezervacija – 'BEFORE UPDATE'. Funkcionalnost okidača temelji se na ponovnoj validaciji uvjeta kroz sljedeće korake:

1. Inicijalizacija i dohvat parametara:

   Proces započinje provjerom postoji li uopće promocija u ažuriranim podacima 'NEW.promocija_id IS NOT NULL'. Ukoliko postoji, okidač iz tablice promocija dohvaća njezine ključne informacije: status aktivnosti, datum početka i datum završetka valjanosti. Te vrijednosti, zajedno s novim datumima boravka 'NEW.pocetak_datum' i 'NEW.kraj_datum', pohranjuje u lokalne varijable radi lakše usporedbe.

2. Trostruka provjera (Re-validacija):

   Okidač provodi identičnu provjeru kao i kod unosa, koristeći IF grananja:

   - Status aktivnosti:

   Prvo se provjerava je li promocija uopće aktivna (v_aktivna = 0). Ako je neaktivna, sustav odmah blokira ažuriranje uz poruku korisniku baze: "Greška: Odabrana promocija je neaktivna!".

   - Početak važenja:

   Zatim se uspoređuje datum dolaska gosta s početkom promocije. Ako rezervacija počinje prije nego što promocija vrijedi, okidač također javlja grešku.

   - Istek promocije:

   Na kraju se provjerava datum odlaska. Ako boravak završava nakon što je promocija istekla, okidač isto sprječava izmjenu podataka. U svim navedenim slučajevima koristi se korisnički definiran SIGNAL SQLSTATE za 'custom' porukama za korisnika kao povratni info zašto nije u mogućnosti dodati traženu promociju na postojeću rezervaciju

Cilj okidača:

Cilj ovog okidača je zatvaranje "rupe" koja je ostala nakon što smo blokirali unos nevažećih promocija kod novih rezervacija. Ovom nadogradnjom pokrivamo I slučaj da zaposlenik recepcije pokuša naknadno dodati nevažeću promociju na već postojeću rezervaciju.

4.2.4 Okidač: Kontrola vremena prijave (Check-in) striktno od 14:00

```
DELIMITER //

CREATE TRIGGER trg_provjera_checkin_time
BEFORE UPDATE ON rezervacija
FOR EACH ROW
BEGIN
```

```
    IF NEW.vrijeme_check_in IS NOT NULL AND OLD.vrijeme_check_in IS
        NULL THEN

        IF TIME(NEW.vrijeme_check_in) <= '14:00:00' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Greska: Check-in nije dozvoljen
               prije 14:00 sati!';
        END IF;

    END IF;
END //

DELIMITER ;
```

Isječak koda 32: Trigger za provjeru vremena check-ina

Okidač trg_provjera_checkin_time implementiran je kako bi se na razini baze podataka automatski provodilo poslovno pravilo o vremenu ulaska gosta u svoju sobu.

Ovaj se okidač aktivira prilikom ažuriranja rezervacije 'BEFORE UPDATE', u trenutku kada gost dođe na recepciju te se želi prijaviti u hotel.

Funkcionalnost okidača se temelji na promjeni statusa dolaska i provjeri satnice kroz sljedeće korake:

1. Prepoznavanje trenutka prijave:

   Okidač reagira isključivo na prijavu gosta, te to postižemo provjerom uvjeta: ako je 'OLD.vrijeme_check_in' bilo 'NULL' , a 'NEW.vrijeme_check_in' poprima neku vrijednost proces provjere se pokreće.

2. Vremenska usporedba:

   Sustav koristi funkciju 'HOUR()' kako bi iz unesenog vremenskog 'timestampa' izvukao samo sat. Zatim provodi provjeru je li taj sat manji ili jednak 14. Ovime se osigurava da se prijava ne može realizirati dok se radi čišćenje i priprema soba.

3. Blokada nedozvoljene radnje:

   Ukoliko recepcija pokuša unijeti vrijeme prijave prije 14:00h, okidač aktivira SIGNAL SQLSTATE naredbu. Radnja prijave se odmah prekida, a korisniku baze se ispisuje poruka: "Greška: Check-in nije dozvoljen prije 14:00 sati!", čime se sprječava ulazak u sobu koja nije spremna.

Cilj okidača:

Cilj ovog okidača je podrška odjelu domaćinstva. On digitalno nameće stanku između odlaska prethodnog i dolaska novog gosta, osiguravajući potrebno vrijeme za pripremu sobe te sprječava ljudsku pogrešku recepcije pri uranjenom puštanju gosta u sobu.

4.2.5 Okidač: Automatski unos svih novih rezervacija u "log_rezervacija"

```
DELIMITER //

CREATE TRIGGER trg_unos_log_rezervacija
```

```
AFTER INSERT ON rezervacija
FOR EACH ROW
BEGIN
    INSERT INTO log_rezervacije (rezervacija_id, stari_status,
       novi_status, korisnik_db)
    VALUES (NEW.id, NULL, NEW.status, USER());
END //

DELIMITER ;
```

Isječak koda 33: Trigger za logiranje unosa nove rezervacije

Okidač trg_unos_log_rezervacija je temelj sustava za praćenje povijesti promjena na pojedinog rezervaciji. Njegova svrha je uspostaviti trag promjena za svaku rezervaciju, osiguravajući da imamo točan zapis o trenutku kada je rezervacija nastala i koji korisnik ju je kreirao, prije nego što se bilo kakve dodatne izmjene uopće dogode (ako se dogode).

Ovaj se okidač aktivira neposredno nakon uspješnog spremanja nove rezervacije u bazu 'AFTER INSERT'.

Funkcionalnost je automatizirana i odvija se kroz jedan ključni korak arhiviranja:

1. Kreiranje inicijalnog zapisa u logu:

Odmah po nastanku nove rezervacije, okidač preuzima podatke iz novog retka 'NEW' i stvara zapis u tablici log_rezervacije. Pritom se događaju sljedeće tri stvari:

- Povezivanje:

  Bilježi se ID nove rezervacije 'NEW.id' kako bismo znali na koju rezervaciju se log odnosi.

- Definiranje statusa:

  Budući da je ovo prvi zapis, stari_status se odmah postavlja na NULL (zbog toga što je ovo prvo pojavljivanje rezervacije u log-u dok se 'novi_status' preuzima iz same rezervacije (npr. 'POTVRDJENA').

- Identifikacija korisnika:

  Pomoću funkcije USER(), sustav nam bilježi korisničko ime osobe koja je izvršila unos.

Cilj okidača:

Cilj ovog okidača je osigurati potpuno bilježenje podataka od samog početka pojedine rezervacije. Kreiranjem ovog prvog zapisa omogućujemo analizu i odgovornost pojedinog zaposlenika, jer uvijek možemo nedvojbeno utvrditi tko je i kada unio rezervaciju u sustav.

4.2.6 Okidač: Automatski unos svih ažuriranja postojećih rezervacija u "log_rezervacija"

```
DELIMITER //

CREATE TRIGGER trg_azuriranje_log_rezervacija
AFTER UPDATE ON rezervacija
FOR EACH ROW
```

```
BEGIN
    -- Logiramo promjenu samo ako se status stvarno promijenio
    IF OLD.status != NEW.status THEN
        INSERT INTO log_rezervacije (rezervacija_id, stari_status,
           novi_status, korisnik_db)
        VALUES (NEW.id, OLD.status, NEW.status, USER());
    END IF;
END //

DELIMITER ;
```

Isječak koda 34: Trigger za logiranje promjene statusa rezervacije

Okidač "trg_azuriranje_log_rezervacija" djeluje kao nastavak prethodnog okidača za arhiviranje podataka o rezervacija. Budući da se na rezervacijama mogu dogoditi promjene; one se potvrđuju, gosti se prijavljuju, odjavljuju ili otkazuju.

Ovaj se okidač aktivira nakon svake izmjene statusa na postojećoj rezervaciji – 'AFTER UPDATE'.

Funkcionalnost okidača temelji se na filtriranju promjena kako bi se izbjeglo nepotrebno gomilanje podataka:

1. Filtracija ključnih promjena:

   Prvo provodi provjeru uvjeta 'IF OLD.status != NEW.status'.

   Sustav nam uspoređuje stari status rezervacije s novim. Isključivo samo ako je došlo do promjene statusa (npr. iz 'POTVRDJENA' u 'OTKAZANA'), okidač nastavlja s radom.

2. Arhiviranje proomjene:

   Kada se prepozna promjena statusa, okidač izvršava 'INSERT' naredbu u tablicu 'log_rezervacije'.

Pritom se bilježi:

- ID rezervacije o kojoj se radi. - stari_status - kakva je rezervacija bila prije izmjene. - novi_status - kakva je rezervacija nakon ažuriranja. - korisnika bsur 'USER()' koji je napravio tu promjenu.

Cilj okidača:

Cilj ovog okidača je omogućiti precizno praćenje povijesti događaja. Omogućuje nam da u bilo kojem trenutku možemo arhivirati tko je I kada napravio bitne izmjene na postojećoj rezervacij.

4.2.7 Okidač: Automatski unos svih brisanja postojećih rezervacija u "log_rezervacija"

```
DELIMITER //

CREATE TRIGGER trg_brisanje_log_rezervacija
AFTER DELETE ON rezervacija
FOR EACH ROW
BEGIN
```

```
    INSERT INTO log_rezervacije (rezervacija_id, stari_status,
       novi_status, korisnik_db)
    VALUES (OLD.id, OLD.status, 'OBRISANO', USER());
END //

DELIMITER ;
```

Isječak koda 35: Trigger za logiranje brisanja rezervacije

Okidač "trg_brisanje_log_rezervacija" predstavlja naš alat za bilježenje podataka o izbrisanim rezervacijama. Brisanje je kao takva destruktivna i u pravilu nepovratna radnja, zato ovaj okidač služi kao "safeguard" čija je funkcija da čak i nakon fizičkog uklanjanja rezervacije iz baze, ostane trajni trag o njenom nekadašnjem postojanju i korisniku baze koja ju je uklonio/la.

Preduvjet:

Prije implementacije ovog okidača, potrebno je izvršiti naredbu "ALTER TABLE log_rezervacije DROP FOREIGN KEY fk_rezervacija_id". Ovo je ključan korak jer prema definiranju DDL-a strani ključ neće dozvoliti postojanje zapisa u logu koji pokazuju na nepostojeću rezervaciju. Uklanjanjem tog ograničenja omogućujemo da logovi ostanu sačuvani čak i ako originalni podatak nestane.

Okidač se aktivira nakon izvršenja naredbe brisanja 'AFTER DELETE'.

Funkcionalnost se odvija kroz proces bilježenja posljednjeg stanja u bazi:

1. Dovaćanje podataka prije brisanja:

   U trenutku brisanja, okidač pristupa podacima koji upravo nestaju koristeći prefiks 'OLD'. On dohvaća originalni ID rezervacije i njen zadnji poznati status.

2. Trajno brisanje:

   Okidač izvršava 'INSERT' u tablicu log_rezervacije. Ovdje se bilježi bitna razlika: novi_status se postavlja na vrijednost 'OBRISANO'. Time se u arhivi log-a razlikuju rezervacije koje su samo otkazane (ali postoje u sustavu) od onih koje su trajno izbrisane. Uz to, bilježi se točan korisnik baze 'USER()' koji je izvršio brisanje.

Cilj okidača:

Cilj ovog okidača je sigurnosnost I sprječavanje uklanjanja tragova. Osigurava se da svaka radnja brisanja pojedine rezervacije u bazi ostavi unos u log-u, omogućujući administratoru ili menadžmentu da naknadno utvrdi tko je obrisao podatke, kada se to dogodilo i koji je bio zadnji status te rezervacije.

4.2.8 Okidač: Automatski unos besplatnog čišćenja ako je gost boravio duže do tri dana

```
DELIMITER //

CREATE TRIGGER trg_besplatno_ciscenje_dugi_boravak
BEFORE INSERT ON stavka_racuna
FOR EACH ROW
BEGIN
```

```
    DECLARE v_naziv_usluge VARCHAR(50);
    DECLARE v_rezervacija_id INT;
    DECLARE v_trajanje_dana INT;

    -- Provjera izvodi se samo ako je unesena usluga (nije null)
    IF NEW.usluga_id IS NOT NULL THEN

        -- Dohvat naziva usluge
        SELECT naziv INTO v_naziv_usluge
        FROM usluga
        WHERE id = NEW.usluga_id;

        IF v_naziv_usluge = 'Zavrsno␣ciscenje' THEN

            -- Dohvat ID-a rezervacije preko ra una
            SELECT rezervacija_id INTO v_rezervacija_id
            FROM racun
            WHERE id = NEW.racun_id;

            SELECT DATEDIFF(kraj_datum, pocetak_datum) INTO
               v_trajanje_dana
            FROM rezervacija
            WHERE id = v_rezervacija_id;


            IF v_trajanje_dana > 3 THEN
                SET NEW.cijena_jedinicna = 0.00;
                SET NEW.iznos_ukupno = 0.00;
                SET NEW.opis = CONCAT(IFNULL(NEW.opis, ''), '␣(
                   GRATIS␣-␣Boravak␣du i␣od␣3␣dana)');
            END IF;

        END IF;
    END IF;
END //

DELIMITER ;
```

Isječak koda 36: Trigger za automatski popust na čišćenje

Okidač "trg_besplatno_ciscenje_dugi_boravak" implementiran je kako bi se automatski primijenila promo pogodnost besplatnog završnog čišćenja za goste koji su boravili duže od tri dana. Ovaj se okidač aktivira prije samog unosa novog zapisa u tablicu stavka_racuna 'BEFORE INSERT'. Funkcionalnost okidača temelji se na dvije ključne provjere: koja se točno usluga dodaje na račun i koliko je trajala pojedina rezervacija vezana uz taj račun. Cijeli proces odvija se kroz sljedeće korake:

1. Identifikacija usluge: Okidač prvo provjerava naziv usluge na temelju unesenog usluga_id iz novog zapisa 'NEW'. Ukoliko je naziv te usluge 'Završno čišćenje', proces se nastavlja. U suprotnom, ako se radi o bilo kojoj drugoj usluzi, okidač odmah prestaje s radom kako ne bi bespotrebno trošio resurse baze.

2. Dohvaćanje i izračun trajanja boravka:

   Nakon što je potvrđeno da se radi o završnom čišćenju, okidač kalkulira koliko je dugo gost boravio. To radi provjerom kroz tablice: povezuje stavku s pripadajućim računom - preko 'racun_id', a zatim taj račun s originalnom rezervacijom - preko 'rezervacija_id'. Koristeći funkciju DATEDIFF, računa razliku između datuma odlaska i dolaska te dobiva točan broj dana boravka te pohranjuje vrijednost u varijablu 'v_trajanje_dana'.

3. Primjena popusta: U završnoj 'IF' provjeri se uspoređuje vrijednost iz varijable 'v_trajanje_dana'. Ukoliko je izračunato trajanje boravka duže od 3 dana, okidač automatski mijenja sljedeće podatke:

   - Jediničnu cijenu - 'cijena_jedinicna' postavlja na 0.00.

   - Ukupan iznos - 'iznos_ukupno' također postavlja na 0.00.

   - U opis stavke – 'opis' automatski nadopisuje tekst "(GRATIS - Boravak duži od 3 dana)", tako da gost na računu ima jasnu informaciju zašto mu ta pojedina usluga nije naplaćena

   Ovaj okidač omogućava potpunu automatizaciju poslovnog pravila o nagrađivanju dugih boravaka bez ručnog unosa osobe koja kreira račun. Time olakšavamo posao zaposleniku te eliminiramo mogućnost ljudske pogreške pri ručnom obračunu popusta i omogućavamo da su svi izdani računi točni i transparentni

4.3 Upiti (Eng. Query)

4.3.1 Prateći upit uz okidač - prikaz aktinih rezervacija za provjeru zauzetosti (Eugen B.)

```sql
SELECT
    r.id AS ID_Rezervacije,
    s.broj AS Broj_Sobe,
    CONCAT(g.ime, ' ', g.prezime) AS Gost,
    CONCAT(DATE_FORMAT(r.pocetak_datum, '%d.%m.'), '-',
           DATE_FORMAT(r.kraj_datum, '%d.%m.%Y')) AS Termin,
    r.status
FROM rezervacija r
JOIN soba s ON r.soba_id = s.id
JOIN gost g ON r.gost_nositelj_id = g.id
WHERE r.status IN ('POTVRDJENA', 'U_TIJEKU');
```

Isječak koda 37: Dohvat aktivnih rezervacija

Ovaj upit služi kao osnovna analiza za recepciju koja pruža jasan, ljudski čitljiv pregled svih trenutno bookiranih termina u hotelu. Pruža nam vizualni podataka prije provjere rada samog okidača za dvostruke rezervacije te možemo odabrati rezervaciju koju ćemo pokušati ponovo bookirati. Upit dohvaća podatke o rezervacijama, ali ih ljepše formatira kroz nekoliko sljedećih koraka:

1. Povezivanje podataka (JOIN):

Budući da tablica rezervacija većinom sadrži 'šture' strane ključeve (ID-ove) koji prosječnom korisniku ne znače puno, upit koristi JOIN kako bi te brojeve povezali sa stvarnim informacijama. Povezuje se s tablicom soba kako bi se dobio broj sobe te s tablicom gost kako bi se dobilo ime i prezime gosta koji je bookirao sobu u sklopu rezervacije.

2. Formatiranje prikaza (CONCAT i DATE_FORMAT):

   Upit koristi funkcije za oblikovanje kako bi kreirao stupac Termin. Datumi se formatiraju i spajaju u elegantniji format (npr. "19.12.-22.12.2026"), što nam olakšava brzo iščitavanje trajanja boravka.

3. Filtriranje relevantnih zapisa:

   Ključan dio upita nam je WHERE klauzula koja filtrira samo one rezervacije koje imaju status 'POTVRDJENA' ili 'U_TIJEKU'. Otkazane rezervacije nas ne zanimaju u ovome upitu jer one ne zauzimaju kapacitete niti su bitne za provjeru dostupnosti soba.

4.3.2 Prateći upit uz okidač: Upit za prikaz statusa i statistike korištenja promocija

```sql
SELECT
    p.id AS ID_Promo,
    p.naziv AS Naziv_Promocije,
    DATE_FORMAT(p.datum_pocetka, '%d.%m.%Y') AS Vrijedi_Od,
    DATE_FORMAT(p.datum_zavrsetka, '%d.%m.%Y') AS Vrijedi_Do,
    p.aktivna AS Aktivna_Raw,
    IF(p.aktivna = 1, 'Da', 'Ne') AS Aktivna_Opis,
    COUNT(r.id) AS Ukupno_Iskoristeno,
    GROUP_CONCAT(r.id) AS ID_Rezervacija_Lista
FROM
    promocija p
LEFT JOIN
    rezervacija r ON p.id = r.promocija_id
GROUP BY
    p.id, p.naziv, p.datum_pocetka, p.datum_zavrsetka, p.aktivna
ORDER BY
    p.id;
```

Isječak koda 38: Statistika korištenja promocija

Ovaj SELECT upit nam služi kao alat za analizu te vizualizaciju promocija koje želimo onemogućiti za unos u prethodnom okidaču. Umjesto 'šturog' popisa jednostavnim 'SELECTOM', on uzima sirove podatke u info izvještaj koji nam prikazuje koje promocije imamo ponudi (aktivne ili neaktivne), već i koliko su korištene u praksi.

Upit koristi sljedeće SQL funkcija i kako bi postigao elegantni format prikaza:

- DATE_FORMAT(datum, format) – Ova funkcija se koristi za konverziju standardnog, SQL formata datuma (YYYY-MM-DD) u "ljudski" format (dd.mm.yyyy).
- IF(uvjet, vrijednost_da, vrijednost_ne) – Koristimo ovu logičku funkciju za transformaciju stupca aktivna. U bazi je taj podatak spremljen kao 'TINYINT' jer tako

MYSQL parsira 'BOOL' vrijednosti. Broj 1 ili 0, što prosječnom krajnjem korisniku ne znači puno. Funkcija IF te brojeve 'prevodi' u tekstualne oznake "Da" ili "Ne".

- LEFT JOIN – Koristimo lijevo spajanje jer želimo vidjeti aposlutno sve promocije iz tablice promocija, čak i one koje nitko nije iskoristio.
- COUNT(r.id) – Agregatna funkcija koja nam broji koliko puta se pojavljuje ID rezervacije uz individualnu promociju. To nam daje osnovni podatak o popularnosti, broj realiziranih rezervacija po promociji.
- GROUP_CONCAT(r.id) –Funkcija koja uzima više redaka (ID-ove rezervacija) i spaja ih u jedan dugački tekstualni niz (npr. "15, 22, 48"). Time izbjegavamo dupli prikaz redaka za istu promociju, a zadržavamo informaciju o točno kojim se rezervacijama radi.
- GROUP BY – Nužna zbog korištenja GROUP_CONCAT. Ona nalaže bazi da sve pronađene rezervacije grupira prema jedinstvenoj promociji.

Glavni cilj ovog upita je izvještaj o ponudi promocija (aktivnih ili neaktivnih) te koje pojedine rezervacije su koristile koje promocije.

1. Koliko je gostiju zapravo iskoristilo kupon? (Vidimo kroz Ukupno_Iskoristeno).
2. Jesu li promocije aktivne? (Vidimo kroz stupac Aktivna).

Upit nam isto služi kao kontrola za prethodni okidač; ako u ovom izvještaju vidimo rezervaciju vezanu uz neaktivnu promociju, odmah znamo da okidač nije funkcionalan ili nije bio implementiran prilikom unosa rezervacije.

4.3.3 Prateći upit uz okidač: Prikaz statusa na aktivnim rezervacijama

```sql
SELECT
    r.id AS ID_Rez,
    CONCAT(g.ime, ' ', g.prezime) AS Nositelj,
    DATE_FORMAT(r.pocetak_datum, '%d.%m.%Y') AS Boravak_Od,
    DATE_FORMAT(r.kraj_datum, '%d.%m.%Y') AS Boravak_Do,
    IFNULL(p.naziv, 'Nema promocije') AS Trenutna_Promo
FROM
    rezervacija r
JOIN
    gost g ON r.gost_nositelj_id = g.id
LEFT JOIN
    promocija p ON r.promocija_id = p.id
WHERE
    r.status IN ('POTVRDJENA', 'U_TIJEKU')
ORDER BY
    r.id;
```

Isječak koda 39: Prikaz rezervacija i pripadajućih promocija

Ovaj upit nam služi za prikaz povezanosti rezervacija s promo akcijama. Također nam služi za testiranje okidača za ažuriranje, vidimo termin boravka pojedine rezervacije te samu primjenu promocije pa možemo odabrati pojedinu rezervaciju za testiranje. Upit se temelji na prikazu podataka prilagođenom ljudskom oku, a ostvaruje se kroz nekoliko tehničkih koraka:

1. Povezivanje podataka (LEFT JOIN):

   Upit koristi LEFT JOIN prema tablici promocija. Želimo prikazati sve rezervacije, one sa i bez primjenjenje promocije.

2. Formatiranje praznih vrijednosti (IFNULL):

   Koristi se funkcija 'IFNULL' kako bismo obradili nedostajuće vrijednosti. Umjesto da se u stupcu Trenutna_Promo pojavi 'NULL', sustav nam automatski upisuje tekst "Nema promocije".

3. Formatiranje i filtriranje:

   Kao i u prethodnim upitima, datume formatiramo u lijepši prikaz, a imena I prezime spajamo za lakše čitanje. Prikaz je ograničen 'WHERE' klauzulom samo na rezervacije koje nas trenutno zanimaju ('POTVRDJENA' ili 'U_TIJEKU').

Cilj upita:

Cilj ovog upita je pružiti uvid u trenutno stanje popusta, omogućujući administratoru da brzo potvrdi jesu li naknadne izmjene promocija upisane u bazi ili su blokirane okidačem.

4.3.4 Upit: Prikaz aktivnih rezervacija i statusa prijave gostiju

```sql
SELECT
    r.id AS ID_Rez,
    CONCAT(g.ime, ' ', g.prezime) AS Gost,
    DATE_FORMAT(pocetak_datum, '%d.%m.%Y') AS Dolazak,

    IFNULL(
        DATE_FORMAT(vrijeme_check_in, '%H:%i'),
        'Ceka se dolazak'
    ) AS Vrijeme_prijave,

    status
FROM rezervacija r
JOIN gost g ON r.gost_nositelj_id = g.id
WHERE status IN ('POTVRDJENA', 'U_TIJEKU')
ORDER BY r.id
LIMIT 10;
```

Isječak koda 40: Prikaz aktivnih rezervacija i statusa prijave gostiju

Upit je napravljen kako bi se dobio brz uvid u buduće i trenutne rezervacije, sa naglaskom na to je li gost već stigao u smještaj ili se njegov dolazak tek očekuje. Upit dohvaća ključne podatke o rezervaciji i gostu te ih formatira za lakše čitanje krajnjem korisniku baze.

Funkcionalnost upita temelji se na povezivanju podataka o rezervacijama sa podacima o gostima te formatiranju vremena prijave. Proces se odvija kroz sljedeće korake:

1. Povezivanje i dohvaćanje podataka o gostu:

   Upit prvo dohvaća osnovne podatke iz tablice rezervacija no kako bi ti podaci imali smisla krajnjem korisniku, povezujemo ih s tablicom gost koristeći JOIN. Na taj

način, umjesto samo ‘gost_nositelj_id’, pomoću funkcije CONCAT spajamo ime i prezime kako bismo dobili puno ime gosta ‘Gost’ koje nam je odmah prepoznatljivo.

2. Formatiranje datuma dolaska:

   Datum početka rezervacije u bazi je zapisan u standardnom formatu, no ovdje opet koristimo funkciju DATE_FORMAT kako bismo ga pretvorili u oblik "dan.mjesec.godina" %d,%m,%Y. Time osiguravamo da je datum prikazan u formatu na koji smo navikli npr. ’20.01.2026’.

3. Prikaz vremena prijave:

   Koristeći funkciju IFNULL, provjeravamo polje vrijeme_check_in:

   - Ako gost još nije stigao (vrijednost u bazi nam je NULL), upit umjesto praznog polja ispisuje poruku ’Čeka se dolazak’.

   -Ako nam je gost već stigao, formatiramo točno vrijeme prijave u oblik "sati:minute" ‘%H:%i’. Na ovaj način recepcija I ostalo osoblje odmah vide tko je u hotelu I koga se još čeka za dolazak.

4. Filtriranje i ograničavanje rezultata:

   Upit ne prikazuje sve rezervacije iz povijesti, već se fokusira samo na one koje su relevantne u ovom trenutku. Pomoću WHERE klauzule filtriramo samo one rezervacije koje su u statusu ’POTVRDJENA’ (gost dolazi) ili ’U_TIJEKU’ (gost je trenutno ovdje). Rezultati su sortirani po ID-u rezervacije, a prikaz je limitiran na prvih 10 zapisa radi preglednosti.

Ovaj upit omogućava recepciji I ostalom osoblju jednostavan pregled dnevnih operacija, jasno razdvajajući goste koji su se već prijavili od onih čiji se dolazak tek očekuje.

4.3.5 Upit: Prikaz povijesti promjena statusa rezervacija s detaljima

```sql
SELECT
    l.rezervacija_id AS 'ID Rez',
    l.id AS 'ID Log',
    CONCAT(g.ime, ' ', g.prezime) AS 'Nositelj Rezervacije',
    DATE_FORMAT(l.vrijeme_promjene, '%d.%m.%Y %H:%i:%s') AS 'Vrijeme Promjene',
    CONCAT(
        IFNULL(l.stari_status, 'NOVA'),
        '-->',
        l.novi_status
    ) AS 'Tijek Promjene Statusa',
    IF(l.novi_status = 'OTKAZANA', 'Otkazana',
        IF(l.novi_status = 'U_TIJEKU', 'U tijeku',
            CONCAT(DATEDIFF(r.kraj_datum, r.pocetak_datum), ' dana')
        )
    ) AS 'Trajanje Boravka',
    l.korisnik_db AS 'Izvrsio Korisnik'
FROM
    log_rezervacije l
```

```
JOIN
    rezervacija r ON l.rezervacija_id = r.id
JOIN
    gost g ON r.gost_nositelj_id = g.id
ORDER BY
    l.vrijeme_promjene DESC, l.id DESC;
```

Isječak koda 41: Prikaz povijesti promjena statusa rezervacija s detaljima

Ovaj upit kreiran je kako bismo dobili potpun uvid u životni ciklus svake individualne rezervacije kroz povijest njezinih promjena. Umjesto da gledamo samo trenutno stanje, ovaj upit nam prikazuje detaljnu povijest logova; tko je, kada i kako mijenjao statuse rezervacija, uz formatiranje dodatnih informacija o samom boravku.

Funkcionalnost upita se temelji na povezivanju tablice logova s podacima o rezervacijama i gostima, te na korištenju logike za prikaz informacija ovisno o trenutnom statusu rezervacije.

Proces se odvija kroz sljedeće korake:

1. Povezivanje podataka o nositelju rezervacije:

   Budući da tablica `log_rezervacije` sadrži samo šture ID-ove, upit nam prvo povezuje logove s tablicama 'rezervacija' i 'gost'. To radimo kako bismo umjesto običnog broja rezervacije mogli prikazati puno ime i prezime gosta ("Nositelj Rezervacije"), što nam odmah govori o kojem gostu se radi.

2. Vizualizacija promjene statusa:

   Kako bi bilo jasno koja promjena se točno dogodila, upit koristi funkciju CONCAT za stvaranje ljepšeg prikaza tijeka promjene. Uzima stari status (ukoliko ga nema, onda pretpostavlja da je to 'NOVA') i pomoću strelice ga spaja s novim statusom. Krajnji rezultat izgleda kao npr. "NOVA –> POTVRDJENA", što nam omogućava da brzo shvatimo nastale promjene.

3. Prikaz trajanja boravka:

   Ovaj dio upita koji koristi ugniježđene IF uvjete za kontekstualni prikaz. Upit ne računa slijepo trajanje za svaku rezervaciju, već provjerava novi status:

   - Ako je rezervacija 'OTKAZANA', ispisuje "Otkazana".
   - Ako je rezervacija 'U_TIJEKU', ispisuje status "U tijeku".
   - Tek u ostalim slučajevima (npr. završena ili potvrđena rezervacija), koristi funkciju DATEDIFF kako bi izračunao i prikazao stvarni broj dana između dolaska i odlaska npr. "5 dana".

4. Formatiranje vremena:

   Datum i vrijeme promjene formatiramo u timestamp sljedećeg formata (%d.%m.%Y %H:%i:%s) kako bismo znali u točnu sekundu kada je promjena nastala. Također, prikazujemo podatak 'korisnik_db' kako bismo točno znali koji je korisnik baze izvršio tu radnju.

5. Sortiranje prema vremenu:

Na kraju, rezultate sortiramo silazno prema vremenu promjene. Na vrhu popisa uvijek će se pojaviti one najnovije promjene koje su se zadnje dogodile u sustavu.

Ovaj upit služi nam kao alat za arhiviranje te omogućavajući administratoru da prati povijest bilo koje rezervacije i provjeri tko je i kada upravljao podacima.

4.3.6 Upit: Prikaz zadnje promjene i trenutnog stanja za specifičnu rezervaciju

```sql
SELECT
    l.rezervacija_id AS 'ID',
    CONCAT(g.ime, ' ', g.prezime) AS 'Gost',
    CONCAT(IFNULL(l.stari_status, 'NOVA'), ' --> ', l.novi_status)
       AS 'Povijest Promjene Statusa',
    DATE_FORMAT(l.vrijeme_promjene, '%H:%i:%s') AS 'Vrijeme
       Promjene',
    DATE_FORMAT(r.pocetak_datum, '%d.%m.%Y') AS 'Trenutni Datum
       Dolaska',
    r.napomena AS 'Trenutna Napomena (Zadnja)'
FROM
    log_rezervacije l
JOIN
    rezervacija r ON l.rezervacija_id = r.id
JOIN
    gost g ON r.gost_nositelj_id = g.id
WHERE
    l.rezervacija_id = 25
ORDER BY
    l.id DESC
LIMIT 1;
```

Isječak koda 42: Prikaz zadnje promjene i trenutnog stanja za specifičnu rezervaciju

Ovaj upit je kreiran kako bismo dobili uvid u posljednju aktivnost za pojedinu rezervaciju (ovdje specifično, onu s ID-om 43). Umjesto da izlistamo kompletnu povijest promjena, ovaj upit nam služi za prikaz zadnje promjene koja se dogodila, uz ključne podatke o toj pojedinoj rezervaciji.

Funkcionalnost ovog upita temelji se na filtriranju logova i izdvajanju samo zadnjeg zapisa, uz paralelno povlačenje bitnih podataka iz glavne tablice rezervacija. Proces se odvija kroz sljedeće korake:

1. Fokus na rezervaciju sa specifičnim ID-jem:

   Pomoću `WHERE` klauzule, upit filtrira podatke i traži isključivo zapise vezane uz rezervaciju s brojem 43. Ovo nam pruža specifičnu provjeru točno određenog rezervacije.

2. Zadnja aktivnost:

   Upit sortira te slaže pronađene logove silazno `DESC` prema njihovom ID-u, a zatim pomoću `LIMIT 1` uzima samo prvi zapis s vrha te tako dohvaćamo zadnju izmjenu koja je napravljena, zanemarujući sve one starije.

3. Vizualiziranje promjene statusa:

Slično kao i u prethodnim upitima, koristimo `CONCAT` kako bismo jasno prikazali promjene. Spajamo stari i novi status sa strelicom (npr. "POTVRDJENA –> U_TIJEKU"), što nam odmah daje ljepši prikaz te zadnje promjene.

4. Povezivanje s trenutnim stanjem:

   Osim što gledamo u log, upit istovremeno gleda i u trenutno stanje tablice rezervacija. Dohvaća najavljeni datum dolaska i zadnju upisanu napomenu `r.napomena`. Ovo nam je bitno jer nam daje potpune informacije; vidimo koja je bila zadnja tehnička promjena, ali i koji su trenutno važeći podaci o dolasku i napomenama za tog gosta.

Ovaj upit bi koristili za situacije kada brzo moramo provjeriti informacije o promjenama na rezervaciji ID broja x i koji su trenutni detalji, bez potrebe za ručnim kopanjem kroz desetke povijesnih zapisa.

4.3.7 Upit: Pregled obrisanih rezervacija i uklonjenih podataka

```sql
SELECT
    l.id AS 'ID Log',
    l.rezervacija_id AS 'ID Obrisane Rezervacije',
    DATE_FORMAT(l.vrijeme_promjene, '%d.%m.%Y %H:%i') AS 'Vrijeme brisanja',
    CONCAT(l.stari_status, ' --> ', l.novi_status) AS 'Status promjena',
    IFNULL(
        CONCAT(g.ime, ' ', g.prezime),
        'Podaci trajno uklonjeni'
    ) AS 'Originalni Nositelj',
    l.korisnik_db AS 'Obrisao korisnik',
    CONCAT(
        TIMESTAMPDIFF(MINUTE, l.vrijeme_promjene, NOW()),
        ' min'
    ) AS 'Proteklo od brisanja'
FROM
    log_rezervacije l
LEFT JOIN
    rezervacija r ON l.rezervacija_id = r.id
LEFT JOIN
    gost g ON r.gost_nositelj_id = g.id
WHERE
    l.novi_status = 'OBRISANO'
ORDER BY
    l.vrijeme_promjene DESC;
```

Isječak koda 43: Pregled obrisanih rezervacija i uklonjenih podataka

Ovaj upit je specifično implementiran za prikaz rezervacija koje su obrisane. Njegov glavni cilj je izvući popis svih rezervacija koje su dobile status 'OBRISANO', uz uvjet da prikaže podatke čak i ako je originalna rezervacija ili gost trajno uklonjena iz baze.

Funkcionalnost ovog upita bazira se na povezivanju tablica i rukovanju situacijama gdje podaci nedostaju. Proces se odvija kroz nekoliko ključnih koraka:

1. **Filtriranje samo obrisanih stavki:**

   Preko `WHERE` klauzule, upit odmah limitira izbor samo na one zapise u logu gdje je `novi_status` postavljen na 'OBRISANO'. Ne zanimaju nas izmjene ili potvrde, tražimo isključivo brisane rezervacije.

2. **Povezivanje LEFT JOIN-om:**

   Koristimo `LEFT JOIN` za povezivanje s tablicama rezervacija i gost. Ako je rezervacija doista obrisana iz glavne tablice, standardni `JOIN` ne bi vratio ništa jer ne bi našao par. `LEFT JOIN` nam omogućava da vidimo zapis iz loga čak i ako je originalna rezervacije izbrisana iz baze.

3. **Izbrisani gosti:**

   Koristeći funkciju `IFNULL` u kombinaciji s podacima o gostu, ovaj upit provjerava možemo li pronaći ime gosta:

   - Ako gost i dalje postoji u bazi, prikazuje se njegovo ime i prezime.
   - Ako je gost obrisan zajedno s rezervacijom (što je kod LEFT JOIN-a rezultiralo NULL vrijednostima), upit umjesto praznog polja ispisuje informativnu poruku 'Podaci trajno uklonjeni'. Ovo nam daje jasnu informaciju da gledamo u "siroče" loga bez matičnog zapisa.

4. **Vremenska analiza (Proteklo vrijeme):**

   Upit ne prikazuje isključivo datum brisanja, već pomoću funkcija `TIMESTAMPDIFF` i `NOW()` izračunava koliko je prošlo minuta od trenutka brisanja do sada. To nam pruža brz uvid u to je li se brisanje dogodilo relativno nedavno ili prije nekoliko dana.

5. **Identifikacija:**

   Jasno se prikazuje `korisnik_db` kako bismo znali tko je točno izvršio brisanje. Rezultati su sortirani datumsko silazno tako da su najnovija brisanja na vrhu, što olakšava pregled nedavnih aktivnosti.

Ovaj upit služi kao alat za sigurnost i kontrolu, jer omogućava uvid u tragove obrisanih podataka koji bi inače bili nevidljivi u glavnom pregledu baze.

4.3.8 Upit: Upit za provjeru ispravnosti automatskog popusta na čišćenje

```sql
SELECT
    CONCAT(g.ime, '␣', g.prezime) AS gost_nositelj,
    r.id AS br_racuna,
    DATE_FORMAT(rez.pocetak_datum, '%d.%m.%Y') AS check_in,
    DATE_FORMAT(rez.kraj_datum, '%d.%m.%Y') AS check_out,
    DATEDIFF(rez.kraj_datum, rez.pocetak_datum) AS trajanje_nocenja
        ,
    sr.opis AS stavka_opis,
    CONCAT(FORMAT(u.cijena_trenutna, 2), '␣  ') AS redovna_cijena,
    CONCAT(FORMAT(sr.cijena_jedinicna, 2), '␣  ') AS
        naplacena_cijena,
```

```
    IF(DATEDIFF(rez.kraj_datum, rez.pocetak_datum) > 3 AND sr.cijena_jedinicna = 0,
        'Gratis',
        IF(DATEDIFF(rez.kraj_datum, rez.pocetak_datum) <= 3 AND sr.cijena_jedinicna > 0,
            'Naplaceno',
            'GRESKA: Trigger ne radi!'
        )
    ) AS status
FROM
    stavka_racuna sr
JOIN
    racun r ON sr.racun_id = r.id
JOIN
    rezervacija rez ON r.rezervacija_id = rez.id
JOIN
    gost g ON rez.gost_nositelj_id = g.id
JOIN
    usluga u ON sr.usluga_id = u.id
WHERE
    u.naziv = 'Zavrsno ciscenje';
```

Isječak koda 44: Upit za provjeru ispravnosti automatskog popusta na čišćenje

Ovaj upit nam služi za prikaz podataka, kao i za provjeru implementacije prethodnog okidača. On provjerava Okidač 8. koji bi trebao automatski dodjeljivati gratis čišćenje za boravke duže od tri dana.

Upit nam povezuje podatke iz pet različitih tablica: `stavka_racuna`, `racun`, `rezervacija`, `gost` i `usluga` kako bi prikazao sve podatke.

Upit se bazira na sljedećim koracima:

1. **Izračun trajanja boravka:**

   Korištenjem funkcije `DATEDIFF`, upit u stvarnom vremenu računa koliko je noćenja gost ostvario, neovisno o onome što je ispisano na računu.

2. **Usporedba cijena:**

   Prikazuje redovnu cijenu usluge iz cjenika te stvarno naplaćenu cijenu na stavci računa.

3. **Automatsko dodjeljivanje (Status):**

   Ovaj dio upita nam je ugniježđena IF logika koja poštuje poslovno pravilo i donosi zaključak o ispravnosti svakog retka:

   - **Ispravno stanje (Gratis):** Ako je gost ostao duže od 3 dana, a cijena je 0€, sustav ispisuje 'Gratis'. To znači da je okidač ispravno odradio posao.
   - **Ispravno stanje (Naplaćeno):** Ako je gost ostao 3 ili manje dana, a cijena je veća od 0€ – sustav ispisuje 'Naplaćeno'. To je ispravno redovno ponašanje.
   - **Alarmantno stanje (Greška):** U bilo kojem drugom slučaju (npr. dugi boravak a naplaćeno, ili kratki boravak a dodijeljeno je besplatno čišćenje),

upit ispisuje poruku: 'GREŠKA: Trigger ne radi!'.

- **Cilj upita:**

  Ovaj upit služi administratoru kao testni upit za brzu provjeru funkcionalnosti okidača u praksi. Umjesto ručnog provjeravanja računa, jednim pogledom na stupac 'status' se može vidjeti funkcionira li automatizacija kako je predviđena ili postoji greška koju treba otkloniti prije puštanja gotovog proizvoda u funkciju.

5. Ivona V.

5.1 Procedure (Eng. Procedures)

5.1.1 Procedura 1: SP_OCISCENA_SOBA

```
DELIMITER $$

CREATE PROCEDURE sp_ociscena_soba(IN p_soba_id INT)
BEGIN
    UPDATE soba
    SET status = 'SLOBODNA'
    WHERE id = p_soba_id
      AND status = 'CISCENJE';
END$$

DELIMITER ;
```

Isječak koda 45: Procedura za ažuriranje statusa sobe nakon čišćenja

Opis: Ova procedura omogućava automatsko prebacivanje statusa sobe na "SLOBODNA" nakon što je izvršeno čišćenje. Procedura provjerava da li je soba trenutno u statusu "CISCENJE" i samo u tom slučaju mijenja status, čime se osigurava da se sobe koje su već slobodne, zauzete ili izvan funkcije ne mijenjaju greškom.

Parametri:

- `p_soba_id` - (IN) ID sobe koju je potrebno označiti kao slobodnu.

Logika izvršavanja: Procedura se izvršava kroz sljedeće korake provjere:

1. **Provjera trenutnog statusa:** Sustav provjerava trenutni status sobe u tablici `soba` za navedeni ID.
2. **Uvjetno ažuriranje:** Ako je status jednak "CISCENJE", procedura ga ažurira na "SLOBODNA".
3. **Sigurnosni mehanizam:** Ako soba nije u statusu čišćenja (npr. već je zauzeta ili slobodna), procedura ne vrši nikakvu promjenu kako bi se očuvao integritet podataka.

Cilj procedure: Glavni cilj je automatizirati proces označavanja sobe kao slobodne nakon fizičkog čišćenja, čime se osigurava točnost podataka o dostupnosti soba u realnom vremenu i sprječava pogrešna rezervacija zauzetih ili neaktivnih soba.

5.1.2 Procedura 2: SP_DODAJ_GOSTA_NA_REZERVACIJU

```
DELIMITER $$

CREATE PROCEDURE sp_dodaj_gosta_na_rezervaciju(
    IN p_rezervacija_id INT,
    IN p_gost_id INT
)
BEGIN
```

```
    IF NOT EXISTS (
        SELECT 1 FROM rezervacija_gost
        WHERE rezervacija_id = p_rezervacija_id
          AND gost_id = p_gost_id
    ) THEN

        INSERT INTO rezervacija_gost(rezervacija_id, gost_id)
        VALUES(p_rezervacija_id, p_gost_id);
    END IF;


    UPDATE rezervacija
    SET broj_osoba = broj_osoba + 1
    WHERE id = p_rezervacija_id;
END$$

DELIMITER ;
```

Isječak koda 46: Procedura za dodavanje gosta na rezervaciju

Opis: Ova procedura omogućava dodavanje gosta u postojeću rezervaciju te automatski povećava broj osoba u toj rezervaciji. Procedura provjerava da gost već nije dodan na rezervaciju kako bi se spriječilo dupliciranje unosa u veznu tablicu.

Parametri:

- `p_rezervacija_id` - (IN) ID rezervacije kojoj se dodaje gost.
- `p_gost_id` - (IN) ID gosta koji se dodaje na rezervaciju.

Logika izvršavanja: Procedura slijedi ove korake kako bi osigurala integritet podataka:

1. **Provjera postojanja:** Provjerava postoji li već veza između navedenog gosta i rezervacije u tablici `rezervacija_gost`.
2. **Unos novog gosta:** Ako veza ne postoji (gost nije prethodno dodan), procedura ubacuje novi zapis (par rezervacija-gost) u tablicu `rezervacija_gost`.
3. **Ažuriranje kapaciteta:** Na kraju, procedura ažurira glavnu tablicu `rezervacija` tako što povećava definiranu vrijednost `broj_osoba` za 1, reflektirajući novo stanje grupe.

Cilj procedure: Olakšati upravljanje gostima unutar jedne rezervacije, automatizirati ažuriranje ukupnog broja osoba bez ručnog računanja i osigurati da se isti gost ne može pogreškom dodati više puta na isti termin.

5.1.3 Procedura 3: SP_PRIMIJENI_PROMOCIJU

```
DELIMITER $$

CREATE PROCEDURE sp_primijeni_promociju(
    IN p_rezervacija_id INT,
    IN p_promocija_id INT
)
BEGIN
```

```
    DECLARE v_popust DECIMAL(5,2);
    DECLARE v_racun_id INT;

    SELECT popust_postotak
    INTO v_popust
    FROM promocija
    WHERE id = p_promocija_id
      AND aktivna = 1
      AND CURDATE() BETWEEN datum_pocetka AND datum_zavrsetka
    LIMIT 1;

    IF v_popust IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Promocija␣nije␣aktivna␣ili␣ne␣postoji';
    END IF;


    UPDATE rezervacija
    SET promocija_id = p_promocija_id
    WHERE id = p_rezervacija_id;


    SELECT id
    INTO v_racun_id
    FROM racun
    WHERE rezervacija_id = p_rezervacija_id
      AND status_racuna = 'OTVOREN'
    LIMIT 1;


    IF v_racun_id IS NOT NULL THEN
        INSERT INTO stavka_racuna(
            racun_id, tip_stavke, opis, kolicina,
            cijena_jedinicna, iznos_ukupno
        )
        VALUES (
            v_racun_id,
            'OSTALO',
            CONCAT('Popust␣-␣promocija␣', p_promocija_id),
            1,
            -v_popust,
            -v_popust
        );
    END IF;
END$$

DELIMITER ;
```

Isječak koda 47: Procedura za primjenu promocije na rezervaciju

Opis: Ova procedura omogućava primjenu promocije na rezervaciju i automatsko dodavanje stavke popusta na otvoreni račun. Procedura provjerava da je promocija aktivna i

da je datum promocije unutar valjanog razdoblja prije nego što izvrši bilo kakve promjene.

Parametri:

- `p_rezervacija_id` - (IN) ID rezervacije na koju se primjenjuje promocija.
- `p_promocija_id` - (IN) ID promocije koja se primjenjuje.

Logika izvršavanja: Procedura osigurava ispravnost podataka kroz nekoliko koraka:

1. **Validacija i dohvat podataka:** Dohvaća postotak popusta iz tablice `promocija`, ali isključivo ako je promocija označena kao aktivna i ako današnji datum pada unutar definiranog trajanja promocije.
2. **Sigurnosna provjera:** Ako promocija nije aktivna, ne postoji, ili je istekao datum, procedura baca grešku (`SIGNAL SQLSTATE`) i prekida izvršavanje kako bi se spriječilo neovlašteno davanje popusta.
3. **Povezivanje:** Ako je validacija uspješna, procedura veže ID promocije na samu rezervaciju u bazi.
4. **Financijsko ažuriranje:** Sustav traži otvoreni račun vezan za tu rezervaciju. Ako ga pronađe, automatski ubacuje novu stavku u `stavka_racuna` s **negativnom vrijednošću** (npr. -10.00 €), što rezultira umanjenjem ukupnog iznosa računa.

Cilj procedure: Automatizirati primjenu promotivnih akcija, osigurati da popust bude jasno vidljiv kao stavka na računu (radi transparentnosti prema gostu) te spriječiti ljudske greške pri ručnom izračunu i unosu popusta.

5.2 Pogledi (Eng. Views)

5.2.1 Pogled: TRENUTNO_STANJE_SOBA

```sql
CREATE OR REPLACE VIEW trenutno_stanje_soba AS
SELECT
    s.id AS soba_id,
    s.broj AS broj_sobe,
    CASE

        WHEN EXISTS (
            SELECT 1
            FROM rezervacija r
            WHERE r.soba_id = s.id

              AND CURDATE() BETWEEN r.pocetak_datum AND r.
                 kraj_datum

              AND r.status IN ('POTVRDJENA', 'U_TIJEKU')
        )
        THEN 'ZAUZETA'
        ELSE s.status
    END AS trenutno_stanje
FROM soba s;

```

```
/* Primjer kori tenja pogleda */
SELECT * FROM trenutno_stanje_soba;
```

Isječak koda 48: Pogled za prikaz trenutnog statusa soba

Opis: Ovaj pogled (*View*) prikazuje trenutni status svih soba u hotelu u realnom vremenu. Njegova specifičnost je u tome što kombinira statičke podatke o sobi s dinamičkim podacima o rezervacijama. Ako soba ima aktivnu rezervaciju za današnji datum, status se automatski prikazuje kao "ZAUZETA", bez obzira na to što piše u definiciji sobe. U suprotnom, koristi se status definiran u tablici soba (npr. SLOBODNA, CISCENJE, IZVAN_FUNKCIJE).

Logika: Pogled funkcionira na principu `CASE` uvjetne logike koja provjerava sljedeće:

1. **Provjera aktivnih rezervacija:** Sustav provjerava postoje li rezervacije za tu sobu gdje današnji datum (`CURDATE()`) pada unutar perioda boravka, te je li status rezervacije "POTVRDJENA" ili "U_TIJEKU".
2. **Prioritet zauzetosti:** Ako takva rezervacija postoji, soba dobiva status "ZAUZETA". Ovo osigurava da soba ne može izgledati slobodna ako je gost u njoj.
3. **Izvorni status:** Ako ne postoji aktivna rezervacija za danas, status se prikazuje onako kako je izvorno pohranjen u tablici `soba`.

Cilj: Glavni cilj je omogućiti recepciji i domaćinstvu pregled *stvarnog* stanja soba za današnji dan. To pomaže osoblju hotela u brzoj provjeri zauzetosti, sprječavanju dvostrukih rezervacija (overbookinga) i efikasnijem planiranju čišćenja.

5.2.2 Pogled: RACUN_PO_REZERVACIJI

```
CREATE OR REPLACE VIEW racun_po_rezervaciji AS
SELECT
    r.id AS rezervacija_id,
    r.status AS status_rezervacije,
    g.id AS gost_id,

    CONCAT(g.ime, ' ', g.prezime) AS ime_prezime_gosta,
    s.broj AS broj_sobe,

    COUNT(DISTINCT ra.id) AS broj_racuna,

    COALESCE(SUM(sr.iznos_ukupno), 0) AS ukupni_iznos
FROM rezervacija r
JOIN gost g ON g.id = r.gost_nositelj_id
JOIN soba s ON s.id = r.soba_id
LEFT JOIN racun ra ON ra.rezervacija_id = r.id
LEFT JOIN stavka_racuna sr ON sr.racun_id = ra.id

GROUP BY
    r.id,
    r.status,
    g.id,
    g.ime,
```

```
    g.prezime,
    s.broj;
```

Isječak koda 49: Pogled za financijski pregled po rezervaciji

Opis: Ovaj pogled (*View*) prikazuje konsolidirane financijske informacije o računima vezanim uz svaku pojedinu rezervaciju. Obuhvaća broj izdanih računa i ukupni zbroj svih stavki računa za tu rezervaciju, pružajući jasan financijski presjek.

Polja pogleda:

- `rezervacija_id` - Jedinstveni identifikator rezervacije.
- `status_rezervacije` - Trenutni status rezervacije (npr. U_TIJEKU, POTVRDJENA).
- `gost_id` - ID gosta koji je nositelj rezervacije.
- `ime_prezime_gosta` - Puno ime i prezime gosta.
- `broj_sobe` - Broj sobe koja je vezana uz tu rezervaciju.
- `broj_racuna` - Ukupan broj računa koji su kreirani za ovu rezervaciju.
- `ukupni_iznos` - Suma svih stavki sa svih računa (ako nema stavki, prikazuje se 0).

Logika: Pogled spaja podatke iz više tablica koristeći sljedeću logiku:

1. **Povezivanje entiteta:** Spaja tablice `rezervacija`, `gost`, `soba`, `racun` i `stavka_racuna` kako bi se dobila kompletna slika.
2. **Grupiranje:** Podaci se grupiraju po jedinstvenoj rezervaciji i gostu kako bi se izbjegli duplikati pri zbrajanju.
3. **Rukovanje praznim vrijednostima:** Koristi funkciju `COALESCE` koja osigurava da, u slučaju kada rezervacija nema niti jedan račun ili stavku, sustav ispiše broj 0 umjesto vrijednosti NULL.

Cilj: Pojednostaviti pregled financijskog stanja svake rezervacije i omogućiti recepcionerima i menadžmentu brzo praćenje naplate po gostu bez potrebe za ručnim zbrajanjem pojedinačnih stavki.

5.2.3 Pogled: LOW_STOCK_ALERT

```sql
CREATE OR REPLACE VIEW low_stock_alert AS
SELECT
    a.id AS artikl_id,
    a.naziv AS artikl,
    a.stanje_zaliha,
    a.jedinica_mjere,
    a.nabavna_cijena,
    CASE
        WHEN a.stanje_zaliha = 0 THEN 'NEMA ZALIHE'
        WHEN a.stanje_zaliha <= 5 THEN 'KRITICNO NISKO'
        ELSE 'OK'
    END AS status_zalihe
FROM artikl a
```

```

WHERE a.stanje_zaliha <= 5;

SELECT * FROM low_stock_alert;
```

Isječak koda 50: Pogled za artikle s kritičnom zalihom

Opis: Ovaj pogled (*View*) prikazuje artikle s kritičnom zalihom u skladištu. Pomaže u brzom prepoznavanju robe koju je potrebno hitno naručiti kako bi se osiguralo nesmetano poslovanje hotela.

Polja pogleda:

- `artikl_id` - Jedinstveni identifikator artikla.
- `artikl` - Naziv artikla.
- `stanje_zaliha` - Trenutno fizičko stanje na skladištu.
- `jedinica_mjere` - Mjerna jedinica artikla (npr. kom, kg).
- `nabavna_cijena` - Cijena po kojoj se artikl nabavlja.
- `status_zalihe` - Tekstualni opis stanja generiran na temelju količine.

Logika: Pogled filtrira i kategorizira podatke na sljedeći način:

1. **Filtriranje:** Prikazuje isključivo artikle čija je količina manja ili jednaka 5 (`WHERE a.stanje_zaliha <= 5`).
2. **Kategorizacija statusa:** Koristeći `CASE` izraz, dodjeljuje opisni status:
   - "NEMA ZALIHE" - ako je stanje točno 0.
   - "KRITICNO NISKO" - ako je stanje između 1 i 5.
   - "OK" - za stanja iznad 5 (iako se ona ne prikazuju zbog filtera, logika postoji u definiciji).

Cilj: Olakšati upravljanje skladištem i omogućiti pravovremeno naručivanje artikala prije nego što zalihe potpuno nestanu.

5.2.4 Pogled: NAJBOLJI_GOSTI_POTROSNJA

```
CREATE OR REPLACE VIEW najbolji_gosti_potrosnja AS
SELECT
    g.id AS gost_id,

    CONCAT(g.ime, ' ', g.prezime) AS gost,
    g.vip_status,

    COUNT(DISTINCT r.id) AS broj_rezervacija,

    COALESCE(SUM(sr.iznos_ukupno), 0) AS ukupna_potrosnja
FROM gost g
JOIN rezervacija r ON r.gost_nositelj_id = g.id
JOIN racun ra ON ra.rezervacija_id = r.id
```

```sql
JOIN stavka_racuna sr ON sr.racun_id = ra.id

WHERE r.status = 'ZAVRSENA'

GROUP BY g.id, g.vip_status;

SELECT * FROM najbolji_gosti_potrosnja
ORDER BY ukupna_potrosnja DESC
LIMIT 10;
```

Isječak koda 51: Pogled za identifikaciju gostiju s najvećom potrošnjom

Opis: Ovaj pogled (*View*) fokusiran je na analizu financijske vrijednosti gostiju. Prikazuje goste s najvećom ukupnom potrošnjom, uzimajući u obzir isključivo završene rezervacije kako bi se dobila stvarna slika prihoda. Uključuje i status VIP-a te frekvenciju dolazaka.

Polja pogleda:

- `gost_id` - Jedinstveni identifikator gosta.
- `gost` - Puno ime i prezime gosta.
- `vip_status` - Indikator je li gost VIP (TRUE/FALSE).
- `broj_rezervacija` - Ukupan broj uspješno završenih rezervacija.
- `ukupna_potrosnja` - Zbroj svih naplaćenih stavki sa svih računa vezanih uz tog gosta.

Logika: Pogled konsolidira podatke kroz nekoliko koraka:

1. **Spajanje tablica:** Povezuje tablice `gost`, `rezervacija`, `racun` i `stavka_racuna` kako bi se povezala osoba s financijskim transakcijama.
2. **Filtriranje:** Isključuje otkazane ili trenutne rezervacije; u obzir se uzimaju samo one sa statusom "ZAVRSENA" (`WHERE r.status = 'ZAVRSENA'`).
3. **Agregacija:** Grupira podatke po jedinstvenom gostu.
4. **Izračun:** Zbraja sve stavke računa. Funkcija `COALESCE` koristi se kao sigurnosna mjera da se u slučaju nedostatka stavki prikaže iznos 0 umjesto NULL vrijednosti.

Cilj: Glavni cilj je omogućiti menadžmentu identifikaciju najvrijednijih gostiju. Ovi podaci su ključni za CRM (Customer Relationship Management), ciljani marketing i dodjelu nagrada ili statusa lojalnosti onima koji najviše troše.

5.2.5 Pogled: EVIDENCIJA_CIJENA_SOBA

```sql
CREATE OR REPLACE VIEW evidencija_cijena_soba AS
SELECT
    cs.id AS cjenik_id,
    ts.naziv AS tip_sobe,
    cs.datum_od,
    cs.datum_do,
    cs.cijena_nocenja,
```

```sql
    cs.boravisna_pristojba_po_osobi,
    cs.aktivan
FROM cjenik_soba cs
JOIN tip_sobe ts ON ts.id = cs.tip_sobe_id
ORDER BY ts.naziv, cs.datum_od DESC;


SELECT * FROM evidencija_cijena_soba;
```

Isječak koda 52: Pogled za povijest i evidenciju cijena soba

Opis: Ovaj pogled (*View*) prikazuje povijest cijena soba grupiranu po tipu sobe. Osim osnovne cijene noćenja, prikazuje i iznose boravišnih pristojbi te status aktivnosti cjenika, omogućujući uvid u to kako su se cijene mijenjale kroz vrijeme.

Polja pogleda:

- `cjenik_id` - Jedinstveni identifikator zapisa u cjeniku.
- `tip_sobe` - Naziv kategorije sobe (npr. Jednokrevetna, Apartman).
- `datum_od / datum_do` - Vremenski period u kojem je navedena cijena važeća.
- `cijena_nocenja` - Iznos cijene po jednom noćenju.
- `boravisna_pristojba_po_osobi` - Dodatna zakonska pristojba koja se naplaćuje po gostu.
- `aktivan` - Indikator je li cjenik trenutno u upotrebi (1) ili je arhiviran (0).

Logika: Pogled strukturira podatke na sljedeći način:

1. **Spajanje podataka:** Povezuje tablicu `cjenik_soba` s šifrarnikom `tip_sobe` kako bi se umjesto ID-a prikazao razumljiv naziv tipa sobe.
2. **Sortiranje:** Podaci su primarno sortirani po nazivu sobe, a sekundarno po datumu početka važenja silazno (`DESC`). To znači da će se za svaki tip sobe na vrhu liste uvijek nalaziti najnoviji (trenutni) cjenik.

Cilj: Omogućiti menadžmentu praćenje fluktuacija cijena kroz sezone i jednostavnu evidenciju povijesnih podataka, što je ključno za formiranje budućih cjenovnih strategija.

5.2.6 Pogled: NEPLACENI_RACUNI

```sql
CREATE OR REPLACE VIEW neplaceni_racuni AS
SELECT
    r.id AS racun_id,
    r.tip_racuna,
    r.rezervacija_id,
    r.datum_izdavanja,
    r.iznos_ukupno
FROM racun r
WHERE r.status_racuna = 'OTVOREN'
ORDER BY r.datum_izdavanja DESC;


```

```
SELECT * FROM neplaceni_racuni;
```

Isječak koda 53: Pogled za prikaz neplaćenih računa

Opis: Ovaj pogled (*View*) prikazuje sve račune koji još nisu plaćeni. Služi kao središnji alat za financijsko praćenje otvorenih obaveza gostiju prema hotelu, omogućujući brzu identifikaciju dugovanja.

Polja pogleda:

- `racun_id` - Jedinstveni identifikator računa.
- `tip_racuna` - Kategorizacija računa (npr. HOTEL, RESTORAN) koja ukazuje na izvor troška.
- `rezervacija_id` - Veza na rezervaciju na koju se račun odnosi.
- `datum_izdavanja` - Datum kada je račun kreiran.
- `iznos_ukupno` - Ukupna suma koju je potrebno naplatiti.

Logika: Pogled selektira podatke na temelju statusa i vremena:

1. **Filtriranje:** Uključuje isključivo račune gdje je `status_racuna` postavljen na "OTVOREN", čime se eliminiraju već plaćeni ili stornirani računi.
2. **Sortiranje:** Rezultati su poredani po datumu izdavanja silazno (`DESC`), tako da su najnoviji neplaćeni računi prikazani na vrhu liste.

Cilj: Olakšati upravljanje naplatom (recepciji i računovodstvu), osigurati da gosti ne napuste objekt s otvorenim dugom te pružiti jasan uvid u trenutna potraživanja hotela.

6. Marko T.

6.1 Okidači

6.1.1 Okidač 1: Osiguravanje ispravnog kronološkog slijeda rezervacije

```
DELIMITER //

CREATE TRIGGER trg_check_datumi_rezervacije
BEFORE INSERT ON rezervacija
FOR EACH ROW
BEGIN
    -- Provjera: Datum odlaska mora biti nakon datuma dolaska
    IF NEW.kraj_datum <= NEW.pocetak_datum THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Gre ka: Datum odlaska mora biti nakon datuma dolaska!';
    END IF;
END //

DELIMITER ;
```

Isječak koda 54: Okidač za provjeru datuma rezervacije

Opis: Ovaj okidač, `trg_check_datumi_rezervacije`, služi kao osnovna logička kontrola prilikom kreiranja nove rezervacije. Njegova svrha je spriječiti unos podataka koji nemaju smisla u stvarnom svijetu – situaciju u kojoj bi gost pokušao otići iz hotela prije nego što je uopće došao. Okidač se aktivira prije samog unosa novog zapisa u tablicu rezervacija (`BEFORE INSERT`).

Logika izvršavanja: Proces provjere odvija se kroz sljedeće korake:

1. **Usporedba datuma:** Okidač uspoređuje uneseni datum odlaska (`NEW.kraj_datum`) s datumom dolaska (`NEW.pocetak_datum`).
2. **Detekcija greške:** Koristeći IF grananje, sustav provjerava je li datum odlaska manji ili jednak datumu dolaska. Ako je taj uvjet zadovoljen, to znači da je rezervacija neispravna (boravak traje 0 ili negativan broj dana).
3. **Blokada unosa:** U slučaju greške, okidač aktivira `SIGNAL SQLSTATE` naredbu koja prekida unos i ispisuje poruku: "Greška: Datum odlaska mora biti nakon datuma dolaska!".

Cilj okidača: Cilj ovog okidača je očuvanje integriteta podataka sprječavanjem logičkih pogrešaka pri unosu datuma, osiguravajući da svaka rezervacija ima valjan vremenski okvir trajanja.

6.1.2 Okidač 2: Zabrana prijevremenog Check-ina prije datuma rezervacije

```sql
DELIMITER //

CREATE TRIGGER trg_sprijeci_rani_checkin
BEFORE UPDATE ON rezervacija
FOR EACH ROW
BEGIN
    IF NEW.status = 'U_TIJEKU' AND OLD.status != 'U_TIJEKU' THEN
        IF CURDATE() < NEW.pocetak_datum THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Gre ka: Check-in nije mogu prije datuma po etka rezervacije!';
        END IF;
    END IF;
END //

DELIMITER ;
```

Isječak koda 55: Okidač za sprječavanje ranog check-ina

Opis: Okidač `trg_sprijeci_rani_checkin` implementiran je kako bi se osiguralo da gosti ne mogu biti prijavljeni u hotel (Check-in) na datum koji je raniji od onog definiranog u njihovoj rezervaciji. Ovo je različito od provjere satnice (14:00h); ovdje se radi o pokušaju prijave na pogrešan dan. Ovaj se okidač aktivira prilikom ažuriranja rezervacije (`BEFORE UPDATE`).

Logika izvršavanja: Funkcionalnost se temelji na sljedećim provjerama:

1. **Prepoznavanje radnje Check-ina:** Okidač prvo provjerava mijenja li se status rezervacije u 'U_TIJEKU'. Ako je novi status 'U_TIJEKU', a stari to nije bio,

sustav zna da se radi o procesu prijave gosta.

2. **Provjera današnjeg datuma:** Sustav koristi funkciju `CURDATE()` kako bi dobio točan današnji datum i uspoređuje ga s planiranim datumom početka rezervacije (`NEW.pocetak_datum`).

3. **Blokada nedozvoljene radnje:** Ukoliko je današnji datum manji od datuma početka rezervacije (gost je došao dan ili više ranije), okidač prekida radnju i ispisuje poruku: "Greška: Check-in nije moguć prije datuma početka rezervacije!".

Cilj okidača: Cilj okidača je sprječavanje administrativnih pogrešaka recepcije i neovlaštenog korištenja soba izvan plaćenih termina, osiguravajući da se status 'U_TIJEKU' može dodijeliti samo na ispravan datum dolaska.

6.1.3 Okidač 3: Automatizacija statusa sobe nakon odlaska gosta

```
DELIMITER //

CREATE TRIGGER trg_auto_ciscenje_nakon_checkouta
AFTER UPDATE ON rezervacija
FOR EACH ROW
BEGIN
    IF NEW.status = 'ZAVRSENA' AND OLD.status != 'ZAVRSENA' THEN
        UPDATE soba
        SET status = 'CISCENJE'
        WHERE id = NEW.soba_id;
    END IF;
END //

DELIMITER ;
```

Isječak koda 56: Okidač za automatsko postavljanje sobe u čišćenje

Opis: Okidač `trg_auto_ciscenje_nakon_checkouta` služi za automatsku komunikaciju između recepcije i odjela domaćinstva. U trenutku kada gost napusti hotel, soba se mora označiti kao prljava kako bi čistačice znale da je spremna za čišćenje. Okidač se aktivira nakon ažuriranja statusa rezervacije (`AFTER UPDATE`).

Logika izvršavanja: Proces se odvija kroz sljedeće korake:

1. **Detekcija odlaska (Check-out):** Sustav provjerava je li novi status rezervacije postavljen na 'ZAVRSENA', dok prethodni status to nije bio.

2. **Ažuriranje statusa sobe:** Ukoliko je gost odjavljen, okidač automatski izvršava `UPDATE` naredbu nad tablicom `soba`. Mijenja status te konkretne sobe (`NEW.soba_id`) u 'CISCENJE'.

Cilj okidača: Cilj procedure je ubrzati protok informacija unutar hotela, eliminirajući potrebu da recepcioner ručno mijenja status sobe nakon odjave gosta, čime se osigurava brža priprema sobe za idućeg gosta.

6.1.4 Okidač 4: Blokada prodaje soba koje su u kvaru ili čišćenju

```
DELIMITER //

CREATE TRIGGER trg_sprijeci_rezervaciju_kvar
BEFORE INSERT ON rezervacija
FOR EACH ROW
BEGIN
    DECLARE v_status_sobe VARCHAR(20);

    SELECT status INTO v_status_sobe
    FROM soba
    WHERE id = NEW.soba_id;

    IF v_status_sobe IN ('IZVAN_FUNKCIJE', 'CISCENJE') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Greska: Soba nije dostupna (Status je
           CISCENJE ili IZVAN FUNKCIJE)!';
    END IF;
END //

DELIMITER ;
```

Isječak koda 57: Okidač za sprječavanje rezervacije nedostupnih soba

Opis: Okidač `trg_sprijeci_rezervaciju_kvar` djeluje kao sigurnosna barijera koja sprječava prodaju soba koje fizički nisu spremne za prihvat gostiju. Ovaj okidač se aktivira prije unosa nove rezervacije (`BEFORE INSERT`).

Logika izvršavanja: Cijeli proces se odvija prema sljedećim provjerama:

1. **Dohvat statusa sobe:** Okidač deklarira varijablu `v_status_sobe` te u nju pomoću `SELECT` upita sprema trenutni status sobe za koju se pokušava napraviti rezervacija.
2. **Provjera dostupnosti:** Koristeći `IF` grananje i operator `IN`, provjerava se je li status sobe 'IZVAN_FUNKCIJE' (kvar) ili 'CISCENJE'.
3. **Sprječavanje unosa:** Ukoliko je soba u jednom od tih statusa, aktivira se `SIGNAL SQLSTATE` koji prekida rezervaciju i javlja grešku: "Greška: Soba nije dostupna (Status je ČIŠĆENJE ili IZVAN FUNKCIJE)!".

Cilj okidača: Cilj ovog okidača je spriječiti "overbooking" na neispravne sobe, čime se izbjegavaju neugodne situacije s gostima i osigurava da se prodaju isključivo sobe koje su spremne za korištenje.

6.2 Procedure

6.2.1 Procedura 1: PROC_OTVORI_NARUDZBU

```
DELIMITER //

CREATE PROCEDURE proc_otvori_narudzbu(
    IN p_zaposlenik_id INT,
    IN p_stol_id INT,
    OUT p_nova_narudzba_id INT
)
BEGIN
    DECLARE v_postoji_otvorena INT DEFAULT 0;


    SELECT COUNT(*) INTO v_postoji_otvorena
    FROM restoran_narudzba
    WHERE restoran_stol_id = p_stol_id AND status = 'OTVORENA';

    IF v_postoji_otvorena > 0 THEN
        SIGNAL SQLSTATE '45001'
        SET MESSAGE_TEXT = 'Greska: Stol vec ima otvorenu narudzbu!
            ';
    ELSE

        INSERT INTO restoran_narudzba (zaposlenik_id,
            restoran_stol_id, datum_otvaranja, status)
        VALUES (p_zaposlenik_id, p_stol_id, NOW(), 'OTVORENA');


        SET p_nova_narudzba_id = LAST_INSERT_ID();
    END IF;
END //

DELIMITER ;
```

Isječak koda 58: Procedura za otvaranje nove narudžbe

Opis: Procedura `proc_otvori_narudzbu` služi za inicijalizaciju novog računa (narudžbe) u restoranu. Ključna funkcionalnost ove procedure je sigurnosna provjera koja osigurava da se na jednom stolu ne mogu paralelno voditi dvije otvorene narudžbe, čime se sprječava kaos u poslovanju i naplati.

Parametri:

- `p_zaposlenik_id` - (IN) ID zaposlenika (konobara) koji otvara narudžbu.
- `p_stol_id` - (IN) ID stola za koji se otvara narudžba.
- `p_nova_narudzba_id` - (OUT) Izlazni parametar u koji sustav vraća jedinstveni broj (ID) upravo kreirane narudžbe.

Logika izvršavanja: Procedura slijedi strogi slijed provjera:

1. **Provjera dostupnosti stola:** Procedura izvršava prebrojavanje aktivnih narudžbi za zadani stol. Traži zapise koji zadovoljavaju dva uvjeta: pripadaju traženom stolu i status im je 'OTVORENA'.
2. **Validacija poslovnog pravila:** Ako brojač pokaže vrijednost veću od 0, procedura blokira radnju i aktivira grešku (`SIGNAL SQLSTATE`) s porukom da je stol zauzet.
3. **Unos nove narudžbe:** Ako je stol slobodan, izvršava se `INSERT` naredba s trenutnim vremenom (`NOW()`) i statusom 'OTVORENA'.
4. **Dohvat generiranog ID-a:** Na kraju, procedura koristi `LAST_INSERT_ID()` kako bi dohvatila sistemski generirani ključ i vratila ga aplikaciji putem izlaznog parametra.

Cilj procedure: Osigurati integritet procesa naručivanja sprječavanjem dupliciranja otvorenih stolova te olakšati rad konobarima automatskim generiranjem zaglavlja narudžbe.

6.2.2 Procedura 2: PROC_DODAJ_STAVKU

```
DELIMITER //

CREATE PROCEDURE proc_dodaj_stavku(
    IN p_narudzba_id INT,
    IN p_usluga_id INT,
    IN p_kolicina INT
)
BEGIN
    DECLARE v_cijena_u_trenutku DECIMAL(10,2);
    DECLARE v_status_narudzbe VARCHAR(20);


    SELECT status INTO v_status_narudzbe
    FROM restoran_narudzba WHERE id = p_narudzba_id;

    IF v_status_narudzbe != 'OTVORENA' THEN
        SIGNAL SQLSTATE '45002'
        SET MESSAGE_TEXT = 'Greska: ne mogu dodati stavku na zatvorenu narudzbu!';
    END IF;


    SELECT cijena_trenutna INTO v_cijena_u_trenutku
    FROM usluga WHERE id = p_usluga_id;


    INSERT INTO restoran_stavka (narudzba_id, usluga_id, kolicina, cijena_u_trenutku, status_pripreme)
    VALUES (p_narudzba_id, p_usluga_id, p_kolicina, v_cijena_u_trenutku, 'NARUCENO');
END //

```

```
DELIMITER ;
```

Isječak koda 59: Procedura za dodavanje stavke na narudžbu

Opis: Procedura `proc_dodaj_stavku` ključna je komponenta operativnog rada koja omogućuje konobarima dodavanje jela ili pića na postojeću narudžbu. Posebnost procedure je "zamrzavanje" cijene usluge u trenutku naručivanja, neovisno o budućim promjenama cjenika.

Parametri:

- `p_narudzba_id` - (IN) ID narudžbe na koju se stavka dodaje.
- `p_usluga_id` - (IN) ID usluge (jela ili pića) koja se naručuje.
- `p_kolicina` - (IN) Količina naručenih usluga.

Logika izvršavanja: Procedura osigurava ispravnost podataka kroz tri koraka:

1. **Provjera valjanosti narudžbe:** Prije upisa, provjerava se status narudžbe. Ako status nije 'OTVORENA', procedura momentalno prekida rad i baca grešku, sprječavajući izmjene na već naplaćenim računima.
2. **Fiksiranje cijene:** Iz tablice `usluga` dohvaća se trenutna cijena artikla. Ovaj iznos se koristi za unos, čime se osigurava da gost plati točno onoliko koliko je artikl koštao u trenutku narudžbe.
3. **Unos stavke:** Kreira se novi zapis u tablici `restoran_stavka` s fiksiranom cijenom i inicijalnim statusom 'NARUCENO', što služi kao signal kuhinji ili šanku za početak pripreme.

Cilj procedure: Automatizirati proces naručivanja uz strogu kontrolu poslovnih pravila – onemogućavanje izmjena na zatvorenim računima i osiguravanje transparentnosti cijena prema gostu.

6.2.3 Procedura 3: PROC_ZAPOCNI_PRIPREMU

```
DELIMITER //

CREATE PROCEDURE proc_zapocni_pripremu(
    IN p_stavka_id INT,
    OUT p_poruka VARCHAR (100)
)
BEGIN
    DECLARE v_usluga_id INT;
    DECLARE v_kolicina_naruceno INT;
    DECLARE v_trenutni_status VARCHAR(20);
    DECLARE v_nedovoljno_zaliha INT DEFAULT 0;


    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_poruka = 'Greska: Doslo je do problema u bazi.';
    END;
```

```
19
20      START TRANSACTION;
21
22      SELECT usluga_id, kolicina, status_pripreme
23      INTO v_usluga_id, v_kolicina_naruceno, v_trenutni_status
24      FROM restoran_stavka
25      WHERE id = p_stavka_id;
26
27      IF v_usluga_id IS NULL THEN
28          SET p_poruka = 'Greska:␣Stavka␣ne␣postoji!';
29          ROLLBACK;
30      ELSEIF v_trenutni_status != 'NARUCENO' THEN
31          SET p_poruka = CONCAT('Greska:␣Ne␣mogu␣pripremiti␣stavku␣
               koja␣je␣u␣statusu␣', v_trenutni_status);
32          ROLLBACK;
33      ELSE
34          SELECT COUNT(*) INTO v_nedovoljno_zaliha
35          FROM normativ n
36          JOIN artikl a ON n.artikl_id = a.id
37          WHERE n.usluga_id = v_usluga_id
38            AND (a.stanje_zaliha - (n.kolicina_potrosnje *
               v_kolicina_naruceno)) < 0;
39
40          IF v_nedovoljno_zaliha > 0 THEN
41              SET p_poruka = 'Greska:␣Nema␣dovoljno␣namirnica␣na␣
                   skladistu␣za␣ovu␣narudzbu!';
42              ROLLBACK;
43          ELSE
44              UPDATE artikl a
45              JOIN normativ n ON a.id = n.artikl_id
46              SET a.stanje_zaliha = a.stanje_zaliha - (n.
                   kolicina_potrosnje * v_kolicina_naruceno)
47              WHERE n.usluga_id = v_usluga_id;
48
49              UPDATE restoran_stavka
50              SET status_pripreme = 'PRIPREMA'
51              WHERE id = p_stavka_id;
52
53              SET p_poruka = 'Namirnice␣rezervirane,␣priprema␣
                   zapoceta.';
54              COMMIT;
55          END IF;
56      END IF;
57  END //
58
59  DELIMITER ;
```

Isječak koda 60: Procedura za započinjanje pripreme i kontrolu zaliha

Opis: Procedura `proc_zapocni_pripremu` predstavlja jednu od najvažnijih logičkih cjelina u sustavu. Ona upravlja ključnim trenutkom kada kuhinja ili šank preuzimaju narudžbu. Osim same promjene statusa, ova procedura osigurava da se namirnice automat-

ski skidaju sa stanja (prema normativima), ali tek nakon što sustav provjeri ima li ih uopće dovoljno na skladištu. Zbog važnosti usklađivanja stanja zaliha i statusa narudžbe, cijela procedura se izvodi unutar transakcije.

Parametri:

- `p_stavka_id` - (IN) ID stavke narudžbe koju osoblje želi početi pripremati.
- `p_poruka` - (OUT) Izlazni parametar koji vraća tekstualnu poruku o ishodu operacije (uspjeh ili opis greške).

Logika izvršavanja: Procedura osigurava integritet podataka kroz nekoliko sigurnosnih razina:

1. **Sigurnost i upravljanje greškama:** Procedura definira `EXIT HANDLER` za `SQLEXCEPTION`. Ako dođe do bilo kakve nepredviđene greške u bazi, automatski se izvršava `ROLLBACK` (poništavanje promjena) kako bi se spriječilo parcijalno ažuriranje podataka.
2. **Dohvat i validacija statusa:** Unutar transakcije, sustav provjerava postoji li stavka i je li njen status 'NARUCENO'. Ako je status već promijenjen (npr. u 'PRIPREMA' ili 'OTKAZANO'), radnja se odbija.
3. **Provjera stanja zaliha (Normativi):** Sustav ne skida namirnice "na slijepo". Spajanjem tablica `normativ` i `artikl`, simulira se potrošnja. Ako bi priprema ovog jela dovela stanje bilo koje potrebne namirnice ispod nule, procedura radi `ROLLBACK` i javlja da nema dovoljno zaliha.
4. **Ažuriranje i realizacija:** Samo ako su svi uvjeti zadovoljeni, procedura trajno umanjuje stanje zaliha u tablici `artikl` i mijenja status stavke u 'PRIPREMA', nakon čega potvrđuje transakciju (`COMMIT`).

Cilj procedure: Potpuna automatizacija skladišnog poslovanja u realnom vremenu. Sprječavanje ljudskih grešaka i osiguravanje da je stanje u skladištu uvijek sinkronizirano s fizičkim procesima u kuhinji.

6.2.4 Procedura 4: PROC_POSLUZI_STAVKU

```
DELIMITER //

CREATE PROCEDURE proc_posluzi_stavku(
    IN p_stavka_id INT,
    OUT p_poruka VARCHAR(100)
)
BEGIN
    DECLARE v_usluga_id INT;
    DECLARE v_kolicina_naruceno INT;
    DECLARE v_trenutni_status VARCHAR(20);
    DECLARE v_nedovoljno_zaliha INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_poruka = 'Greska: Doslo je do problema u bazi.';
    END;
```

```

    START TRANSACTION;

    SELECT usluga_id, kolicina, status_pripreme
    INTO v_usluga_id, v_kolicina_naruceno, v_trenutni_status
    FROM restoran_stavka
    WHERE id = p_stavka_id;

    IF v_trenutni_status = 'PRIPREMA' THEN
        UPDATE restoran_stavka SET status_pripreme = 'POSLUZENO'
           WHERE id = p_stavka_id;
        SET p_poruka = 'Jelo posluzeno (zalihe su skinute ranije).'
           ;
        COMMIT;
    ELSEIF v_trenutni_status = 'NARUCENO' THEN
        SELECT COUNT(*) INTO v_nedovoljno_zaliha
        FROM normativ n
        JOIN artikl a ON n.artikl_id = a.id
        WHERE n.usluga_id = v_usluga_id
          AND (a.stanje_zaliha - (n.kolicina_potrosnje *
             v_kolicina_naruceno)) < 0;

        IF v_nedovoljno_zaliha > 0 THEN
            SET p_poruka = 'Greska: Nema dovoljno artikala na
               zalihi za posluzivanje!';
            ROLLBACK;
        ELSE
            UPDATE artikl a
            JOIN normativ n ON a.id = n.artikl_id
            SET a.stanje_zaliha = a.stanje_zaliha - (n.
               kolicina_potrosnje * v_kolicina_naruceno)
            WHERE n.usluga_id = v_usluga_id;

            UPDATE restoran_stavka SET status_pripreme = 'POSLUZENO
               ' WHERE id = p_stavka_id;
            SET p_poruka = 'Posluzeno i skinuto sa zaliha (direktna
               narudzba).';
            COMMIT;
        END IF;
    ELSEIF v_trenutni_status = 'POSLUZENO' THEN
        SET p_poruka = 'Info: Vec je posluzeno.';
        ROLLBACK;
    ELSE
        SET p_poruka = 'Greska: Nepoznat status.';
        ROLLBACK;
    END IF;
END //

DELIMITER ;
```

Isječak koda 61: Procedura za posluživanje stavke i finalizaciju zaliha

Opis: Procedura `proc_posluzi_stavku` zatvara operativni ciklus jedne narudžbe. Njezina glavna uloga je evidentirati da je gost dobio svoju narudžbu, ali je pritom dizajnirana da prepozna razliku između dva različita tijeka rada: posluživanje jela koje je pripremljeno u kuhinji (gdje su zalihe već riješene) i posluživanje pića ili gotovih proizvoda (gdje zalihe tek treba skinuti).

Parametri:

- `p_stavka_id` - (IN) ID stavke narudžbe koja se poslužuje.
- `p_poruka` - (OUT) Tekstualna poruka o rezultatu operacije.

Logika izvršavanja: Cijeli proces se odvija unutar transakcije, a logika je podijeljena na tri ključna scenarija:

1. **Scenarij 1: Završetak pripreme (Kuhinja):** Ako je trenutni status stavke 'PRIPREMA', to znači da su namirnice već skinute kroz prethodnu proceduru. Sustav samo ažurira status u 'POSLUZENO' i potvrđuje transakciju.
2. **Scenarij 2: Direktna narudžba (Šank/Hladni pogon):** Ako je status još uvijek 'NARUCENO' (npr. kava ili sok), procedura mora odraditi skidanje zaliha.
   - *Provjera:* Sustav provjerava ima li dovoljno robe na skladištu. Ako nema, vraća grešku.
   - *Realizacija:* Ako robe ima, umanjuje stanje zaliha prema normativu i postavlja status na 'POSLUZENO'.
3. **Scenarij 3: Sigurnosne provjere:** Procedura pokriva rubne slučajeve poput pokušaja posluživanja već posluženog jela (vraća info poruku) ili grešaka u bazi (aktivira `ROLLBACK` mehanizam).

Cilj procedure: Osigurati fleksibilnost sustava omogućavanjem jedinstvenog načina rada za konobare (jedan gumb "Posluži"), dok sustav u pozadini automatski odlučuje o trenutku skidanja zaliha, garantirajući točnost skladišnog stanja.

6.2.5 Procedura 5: PROC_NAPLATI_NARUDZBU

```
DELIMITER //

CREATE PROCEDURE proc_naplati_narudzbu(
    IN p_narudzba_id INT,
    IN p_nacin_placanja VARCHAR(20),
    IN p_broj_sobe INT,
    OUT p_poruka VARCHAR(100)
)
main: BEGIN
    DECLARE v_status_narudzbe VARCHAR(20);
    DECLARE v_rezervacija_id INT DEFAULT NULL;
    DECLARE v_racun_id INT;
    DECLARE v_gost_ime VARCHAR(100);

    DECLARE v_naziv_usluge VARCHAR(50);
    DECLARE v_kolicina INT;
    DECLARE v_cijena DECIMAL(10,2);
```

```
    DECLARE v_stavka_ukupno DECIMAL(10,2);
    DECLARE done INT DEFAULT FALSE;

    DECLARE cur_stavke CURSOR FOR
        SELECT u.naziv, rs.kolicina, rs.cijena_u_trenutku
        FROM restoran_stavka rs
        JOIN usluga u ON rs.usluga_id = u.id
        WHERE rs.narudzba_id = p_narudzba_id
          AND rs.status_pripreme != 'STORNIRANO';

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_poruka = 'Greska: Neuspjela naplata (SQL Error).';
    END;

    START TRANSACTION;

    SELECT status INTO v_status_narudzbe FROM restoran_narudzba
       WHERE id = p_narudzba_id;

    IF v_status_narudzbe IS NULL OR v_status_narudzbe != 'OTVORENA'
        THEN
        SET p_poruka = 'Greska: Narudzba nije otvorena ili ne
           postoji.';
        ROLLBACK;
        LEAVE main;
    END IF;

    IF p_broj_sobe IS NOT NULL THEN
        SELECT r.id, CONCAT(g.ime, ' ', g.prezime)
        INTO v_rezervacija_id, v_gost_ime
        FROM rezervacija r
        JOIN soba s ON r.soba_id = s.id
        JOIN gost g ON r.gost_nositelj_id = g.id
        WHERE s.broj = p_broj_sobe AND r.status = 'U_TIJEKU';

        IF v_rezervacija_id IS NULL THEN
            SET p_poruka = CONCAT('Greska: U sobi ', p_broj_sobe, '
               nema prijavljenih gostiju!');
            ROLLBACK;
            LEAVE main;
        END IF;
    END IF;

    IF v_rezervacija_id IS NOT NULL THEN
        SELECT id INTO v_racun_id
        FROM racun
        WHERE rezervacija_id = v_rezervacija_id
```

```
65          AND status_racuna = 'OTVOREN'
66          AND tip_racuna = 'HOTEL'
67        LIMIT 1;
68
69        IF v_racun_id IS NULL THEN
70            INSERT INTO racun (tip_racuna, rezervacija_id,
                datum_izdavanja, nacin_placanja, iznos_ukupno,
                status_racuna)
71            VALUES ('HOTEL', v_rezervacija_id, NOW(), 'VIRMANSKI',
                0.00, 'OTVOREN');
72            SET v_racun_id = LAST_INSERT_ID();
73        END IF;
74    ELSE
75        INSERT INTO racun (tip_racuna, rezervacija_id,
            datum_izdavanja, nacin_placanja, iznos_ukupno,
            status_racuna)
76        VALUES ('RESTORAN', NULL, NOW(), p_nacin_placanja, 0.00, '
            PLACENO');
77        SET v_racun_id = LAST_INSERT_ID();
78    END IF;
79
80    OPEN cur_stavke;
81
82    read_loop: LOOP
83        FETCH cur_stavke INTO v_naziv_usluge, v_kolicina, v_cijena;
84        IF done THEN
85            LEAVE read_loop;
86        END IF;
87
88        SET v_stavka_ukupno = v_kolicina * v_cijena;
89
90        INSERT INTO stavka_racuna (racun_id, tip_stavke, opis,
            kolicina, cijena_jedinicna, iznos_ukupno)
91        VALUES (v_racun_id, 'USLUGA', v_naziv_usluge, v_kolicina,
            v_cijena, v_stavka_ukupno);
92    END LOOP;
93
94    CLOSE cur_stavke;
95
96    UPDATE racun
97    SET iznos_ukupno = (SELECT SUM(iznos_ukupno) FROM stavka_racuna
        WHERE racun_id = v_racun_id)
98    WHERE id = v_racun_id;
99
100   UPDATE restoran_narudzba
101   SET status = 'PLACENO', datum_zatvaranja = NOW()
102   WHERE id = p_narudzba_id;
103
104   SET p_poruka = CONCAT('Uspjeh: Racun naplacen (ID Racuna: ',
        v_racun_id, ')');
105   COMMIT;
```

```
END //

DELIMITER ;
```

Isječak koda 62: Procedura za naplatu narudžbe i prijenos na račun (Kursor)

Opis: Procedura `proc_naplati_narudzbu` predstavlja financijsko srce restoranskog modula. Ona služi za finalizaciju narudžbe, generiranje fiskalnog računa i prijenos stavki iz privremene tablice narudžbi u trajnu financijsku evidenciju. Posebnost ove procedure je što podržava dva različita načina poslovanja: direktnu naplatu (za vanjske goste) i "stavljanje na sobu" (za goste hotela).

Parametri:

- `p_narudzba_id` - (IN) ID narudžbe koja se naplaćuje.
- `p_nacin_placanja` - (IN) Metoda plaćanja (npr. 'GOTOVINA', 'KARTICA').
- `p_broj_sobe` - (IN) Opcionalni parametar. Ako je unesen, trošak se veže na sobu.
- `p_poruka` - (OUT) Povratna informacija o uspjehu ili grešci.

Logika izvršavanja: Cijeli proces je omotan u transakciju zbog financijske osjetljivosti, a logika se odvija kroz nekoliko faza:

1. **Validacija i provjera statusa:** Sustav provjerava je li narudžba otvorena. Ako je unesen broj sobe, dodatno se provjerava postoji li u toj sobi aktivna rezervacija kako bi se spriječilo terećenje pogrešnog gosta.
2. **Strategija kreiranja računa:**
   - *Gost hotela:* Sustav traži postojeći otvoreni račun za tu rezervaciju. Ako ne postoji, otvara novi.
   - *Vanjski gost:* Odmah se kreira novi račun sa statusom 'PLACENO', jer se transakcija zatvara na licu mjesta.
3. **Obrada stavki pomoću Kursora:** Umjesto jednostavnog kopiranja, procedura koristi KURSOR (`cur_stavke`) za preciznu iteraciju. Kursor prolazi redak po redak, dohvaća podatke i fizički ih upisuje u tablicu `stavka_racuna`.
4. **Finalizacija:** Nakon prijenosa svih stavki, procedura zbraja ukupan iznos, ažurira zaglavlje računa i mijenja status narudžbe u 'PLACENO'.

Cilj procedure: Sigurna i točna financijska transakcija koja integrira restoranski sustav s recepcijskim i knjigovodstvenim modulom, osiguravajući da niti jedna stavka ne ostane nenaplaćena.

6.2.6 Procedura 6: PROC_KREIRAJ_REZERVACIJU

```
DELIMITER //

CREATE PROCEDURE proc_kreiraj_rezervaciju(
    IN p_gost_id INT,
    IN p_soba_id INT,
    IN p_promocija_id INT,
```

```
    IN p_datum_dolaska DATE,
    IN p_datum_odlaska DATE,
    IN p_broj_osoba INT,
    IN p_napomena TEXT
)
BEGIN
    INSERT INTO rezervacija
    (gost_nositelj_id, zaposlenik_id, soba_id, promocija_id,
       pocetak_datum, kraj_datum, broj_osoba, status, napomena)
    VALUES
    (p_gost_id, 1, p_soba_id, p_promocija_id, p_datum_dolaska,
       p_datum_odlaska, p_broj_osoba, 'POTVRDJENA', p_napomena);
END //

DELIMITER ;
```

Isječak koda 63: Procedura za kreiranje nove rezervacije

Opis: Procedura `proc_kreiraj_rezervaciju` služi kao osnovna točka ulaza za unos novih rezervacija u sustav. Dizajnirana je da bude jednostavna i direktna, preuzimajući sve nužne podatke od korisnika (recepcije) i pohranjujući ih u bazu podataka u jednom koraku.

Parametri:

- `p_gost_id` - (IN) ID gosta nositelja rezervacije.
- `p_soba_id` - (IN) ID sobe koja se rezervira.
- `p_promocija_id` - (IN) ID promocije (može biti NULL ako nema popusta).
- `p_datum_dolaska` - (IN) Datum početka boravka.
- `p_datum_odlaska` - (IN) Datum završetka boravka.
- `p_broj_osoba` - (IN) Broj ljudi u sobi.
- `p_napomena` - (IN) Tekstualna napomena za posebne zahtjeve.

Logika izvršavanja: Procedura je namjerno lišena složene logike provjere (poput dostupnosti sobe ili ispravnosti datuma) jer se taj dio posla oslanja na prethodno definirane **okidače** (Triggers). Umjesto toga, ona izvršava direktan `INSERT` sa specifičnim pravilima:

1. **Automatski status:** Svaka rezervacija kreirana ovom procedurom automatski dobiva status 'POTVRDJENA', pod pretpostavkom da je gost već prihvatio uvjete.
2. **Fiksni zaposlenik:** Polje `zaposlenik_id` je u ovoj verziji fiksno postavljeno na vrijednost 1. To predstavlja sistemskog administratora ili generičkog korisnika recepcije, čime se pojednostavljuje unos jer se ne mora pratiti ID trenutnog smjenskog radnika.

Cilj procedure: Standardizacija i ubrzanje unosa. Umjesto pisanja dugačkih SQL naredbi u aplikacijskom kodu, ova procedura nudi čisto sučelje za kreiranje rezervacije, osiguravajući da su sva obavezna polja popunjena.

6.2.7 Procedura 7: PROC_KREIRAJ_GOSTA

```
DELIMITER //

CREATE PROCEDURE proc_kreiraj_gosta(
    IN p_ime VARCHAR(50),
    IN p_prezime VARCHAR(50),
    IN p_vrsta_dok_id INT,
    IN p_broj_dok VARCHAR(30),
    IN p_grad_id INT,
    IN p_adresa VARCHAR(100),
    IN p_drzava_id INT
)
BEGIN
    INSERT INTO gost
    (ime, prezime, vrsta_dokumenta_id, broj_dokumenta,
       prebivaliste_grad_id, prebivaliste_adresa, drzavljanstvo_id)
    VALUES
    (p_ime, p_prezime, p_vrsta_dok_id, p_broj_dok, p_grad_id,
       p_adresa, p_drzava_id);
END //

DELIMITER ;
```

Isječak koda 64: Procedura za registraciju novog gosta

Opis: Procedura `proc_kreiraj_gosta` standardizira proces unosa novih gostiju u bazu podataka. Koristi se na recepciji prilikom prvog dolaska gosta, kada je potrebno evidentirati njegove osobne podatke i identifikacijske dokumente u sustav.

Parametri:

- `p_ime` - (IN) Ime gosta.
- `p_prezime` - (IN) Prezime gosta.
- `p_vrsta_dok_id` - (IN) ID vrste dokumenta (npr. osobna iskaznica, putovnica) iz šifrarnika.
- `p_broj_dok` - (IN) Broj identifikacijskog dokumenta.
- `p_grad_id` - (IN) ID grada prebivališta.
- `p_adresa` - (IN) Ulica i kućni broj.
- `p_drzava_id` - (IN) ID državljanstva.

Logika izvršavanja: Funkcionalnost se temelji na sljedećim principima:

1. **Centralizirani unos podataka:** Slično kao i kod kreiranja rezervacije, ova procedura pojednostavljuje rad aplikacije. Umjesto da aplikacija "ručno" formira SQL upite, procedura prihvaća sirove podatke i mapira ih na odgovarajuće stupce u tablici `gost`.
2. **Povezivanje sa šifrarnicima:** Procedura osigurava strukturiranost podataka. Umjesto upisivanja slobodnog teksta za grad ili državu, procedura zahtijeva ID-ove

(p_grad_id, p_drzava_id). Time se osigurava referencijalni integritet i omogućuje točna statistička obrada podataka (npr. izvještaji o broju gostiju po državama).

Cilj procedure: Cilj ove procedure je ubrzati proces "Check-ina". Ona omogućuje recepciji brzo spremanje profila gosta kroz jednostavno sučelje, osiguravajući da su svi nužni podaci za zakonsku prijavu (poput broja dokumenta i državljanstva) ispravno proslijeđeni u bazu.

6.2.8 Procedura 8: PROC_REZERVACIJA_CHECK_IN

```
DELIMITER //

CREATE PROCEDURE proc_rezervacija_check_in(
    IN p_rezervacija_id INT
)
BEGIN
    UPDATE rezervacija
    SET status = 'U_TIJEKU',
        vrijeme_check_in = NOW()
    WHERE id = p_rezervacija_id;
END //

DELIMITER ;
```

Isječak koda 65: Procedura za prijavu gosta (Check-in)

Opis: Procedura `proc_rezervacija_check_in` operativna je procedura koju recepcija koristi u trenutku kada gost fizički stigne u hotel i preuzme ključeve sobe. Ona formalno pretvara rezervaciju iz statusa "najave" u status "aktivni boravak".

Parametri:

- `p_rezervacija_id` - (IN) ID rezervacije za koju se vrši prijava.

Logika izvršavanja: Iako jednostavna, procedura obavlja dvije ključne funkcije u jednom koraku:

1. **Ažuriranje statusa boravka:** Izvršava se `UPDATE` naredba koja mijenja status rezervacije u 'U_TIJEKU'. Ova promjena služi kao signal ostatku sustava (restoranu, domaćinstvu) da je soba zauzeta i da je gost prisutan u objektu.
2. **Bilježenje vremena dolaska:** Istovremeno s promjenom statusa, u stupac `vrijeme_check_in` upisuje se trenutni datum i vrijeme koristeći funkciju `NOW()`. Ovo je ključno za točnu evidenciju, jer se stvarno vrijeme dolaska često razlikuje od planiranog.

Cilj procedure: Cilj ove procedure je jednim klikom evidentirati dolazak gosta. Time se osigurava da u sustavu postoji točan podatak tko je trenutno u hotelu (za potrebe policije i sigurnosti) te se aktiviraju ostali procesi vezani uz boravak gosta, poput mogućnosti stavljanja restoranskih troškova na teret sobe.

6.2.9 Procedura 9: PROC_REZERVACIJA_CHECK_OUT

```
DELIMITER //

```

```
CREATE PROCEDURE proc_rezervacija_check_out(
    IN p_rezervacija_id INT
)
BEGIN
    UPDATE rezervacija
    SET status = 'ZAVRSENA',
        vrijeme_check_out = NOW()
    WHERE id = p_rezervacija_id;
END //

DELIMITER ;
```

Isječak koda 66: Procedura za odjavu gosta (Check-out)

Opis: Procedura `proc_rezervacija_check_out` koristi se na recepciji prilikom odlaska gosta iz hotela. Ovom procedurom se formalno završava pružanje usluge smještaja, zatvara se poslovni ciklus boravka i oslobađa se termin u sustavu.

Parametri:

- `p_rezervacija_id` - (IN) ID rezervacije koja se zaključuje (odjavljuje).

Logika izvršavanja: Procedura obavlja dvije radnje koje pokreću lančanu reakciju u sustavu:

1. **Finalizacija boravka:** Izvršava se `UPDATE` naredba kojom se status rezervacije mijenja iz 'U_TIJEKU' u 'ZAVRSENA'. *Napomena:* Ova promjena statusa je ključna jer (kako je definirano u ranijim okidačima) automatski aktivira okidač `trg_auto_ciscenje_nakon_checkouta`, koji sobu momentalno prebacuje u status 'CISCENJE'.
2. **Vremenski pečat odlaska:** Paralelno s promjenom statusa, u stupac `vrijeme_check_out` upisuje se točan trenutak odjave koristeći funkciju `NOW()`. Ovaj podatak je bitan za povijest boravka, statistiku i eventualnu naplatu naknada za kasni odlazak (late check-out).

Cilj procedure: Brzo i točno evidentiranje odlaska gosta. Procedura osigurava da se boravak službeno zatvori u točno vrijeme, čime se automatski signalizira domaćinstvu da sobu treba pripremiti za idućeg gosta.

6.2.10 Procedura 10: PROC_DODAJ_USLUGU_NA_SOBU

```
DELIMITER //

CREATE PROCEDURE proc_dodaj_uslugu_na_sobu(
    IN p_rezervacija_id INT,
    IN p_usluga_id INT,
    IN p_kolicina INT,
    IN p_napomena VARCHAR(100)
)
BEGIN
    DECLARE v_racun_id INT;
    DECLARE v_cijena DECIMAL(10,2);
    DECLARE v_naziv_usluge VARCHAR(50);
```

```
    DECLARE v_konacni_opis VARCHAR(100);

    SELECT cijena_trenutna, naziv INTO v_cijena, v_naziv_usluge
    FROM usluga WHERE id = p_usluga_id;

    SET v_konacni_opis = COALESCE(NULLIF(p_napomena, ''),
       v_naziv_usluge);

    SELECT id INTO v_racun_id
    FROM racun
    WHERE rezervacija_id = p_rezervacija_id
      AND tip_racuna = 'HOTEL'
      AND status_racuna = 'OTVOREN'
    LIMIT 1;

    IF v_racun_id IS NULL THEN
        INSERT INTO racun (tip_racuna, rezervacija_id,
           nacin_placanja, iznos_ukupno, status_racuna)
        VALUES ('HOTEL', p_rezervacija_id, 'VIRMANSKI', 0.00, '
           OTVOREN');
        SET v_racun_id = LAST_INSERT_ID();
    END IF;

    INSERT INTO stavka_racuna (racun_id, usluga_id, tip_stavke,
       opis, kolicina, cijena_jedinicna, iznos_ukupno)
    VALUES (v_racun_id, p_usluga_id, 'USLUGA', v_konacni_opis,
       p_kolicina, v_cijena, v_cijena * p_kolicina);

    UPDATE racun
    SET iznos_ukupno = (SELECT SUM(iznos_ukupno) FROM stavka_racuna
        WHERE racun_id = v_racun_id)
    WHERE id = v_racun_id;
END //

DELIMITER ;
```

Isječak koda 67: Procedura za dodavanje dodatne usluge na sobu

Opis: Procedura `proc_dodaj_uslugu_na_sobu` koristi se za terećenje sobe za razne dodatne troškove koji nastanu tijekom boravka gosta (npr. korištenje minibara, usluga pranja rublja, wellness tretmani ili room service). Njezina ključna prednost je automatizacija upravljanja računima te inteligentna obrada tekstualnih opisa stavki.

Parametri:

- `p_rezervacija_id` - (IN) ID rezervacije na koju se trošak veže.
- `p_usluga_id` - (IN) ID usluge iz šifrarnika.
- `p_kolicina` - (IN) Količina iskorištenih usluga.
- `p_napomena` - (IN) Opcionalni tekst koji definira detalje usluge.

Logika izvršavanja: Procedura osigurava integritet podataka kroz nekoliko koraka:

1. **Dohvat trenutne cijene:** Sustav uvijek dohvaća cijenu iz baze u trenutku izvršavanja, osiguravajući da se gostu naplati aktualni iznos.
2. **Pametni odabir opisa:** Implementirana je logika koja koristi funkcije `NULLIF` i `COALESCE`. Ako je parametar napomene prazan, sustav automatski koristi službeni naziv usluge iz šifrarnika. Time se sprječava pojava stavki bez opisa na finalnom računu.
3. **Pametno upravljanje računom:** Procedura provjerava postoji li već otvoreni račun za tu rezervaciju. Ako ne postoji (npr. ovo je prvi trošak), procedura ga automatski kreira. Time se recepcija rasterećuje od ručnog otvaranja računa.
4. **Ažuriranje salda:** Nakon unosa stavke, procedura automatski rekalkulira ukupan iznos računa.

Cilj procedure: Centralizacija svih troškova gosta na jednom mjestu uz poboljšani integritet podataka i sprječavanje administrativnih propusta (npr. zaboravljenih opisa stavki).

6.2.11 Procedura 11: PROC_PRIJAVI_KVAR

```
DELIMITER //

CREATE PROCEDURE proc_prijavi_kvar(
    IN p_zaposlenik_id INT,
    IN p_soba_id INT,
    IN p_opis_kvara TEXT,
    IN p_korisnik_kriv INT
)
BEGIN
    INSERT INTO servis_dnevni_nalog (zaposlenik_id, soba_id, opis,
       korisnik_placa, datum_naloga, rijeseno)
    VALUES (p_zaposlenik_id, p_soba_id, p_opis_kvara,
       p_korisnik_kriv, NOW(), 0);

    UPDATE soba
    SET status = 'IZVAN_FUNKCIJE'
    WHERE id = p_soba_id;
END //

DELIMITER ;
```

Isječak koda 68: Procedura za prijavu kvara i blokadu sobe

Opis: Procedura `proc_prijavi_kvar` kritična je komponenta sustava održavanja kvalitete hotela. Koristi se kada osoblje (domaćinstvo ili recepcija) uoči tehnički problem u sobi. Njena funkcionalnost nadilazi običnо bilježenje problema; ona djeluje kao sigurnosni mehanizam koji automatski povlači sobu iz prodaje.

Parametri:

- `p_zaposlenik_id` - (IN) ID zaposlenika koji prijavljuje kvar.
- `p_soba_id` - (IN) ID sobe u kojoj je nastao problem.

- `p_opis_kvara` - (IN) Tekstualni opis problema.
- `p_korisnik_kriv` - (IN) Zastavica (0 ili 1) koja definira hoće li se trošak popravka teretiti gostu.

Logika izvršavanja: Procedura sinkronizira rad službe održavanja i recepcije kroz dva koraka:

1. **Kreiranje radnog naloga:** Procedura izvršava `INSERT` naredbu u tablicu `servis_dnevni_nalog`. Sustav bilježi točno vrijeme prijave, a status naloga (`rijeseno`) postavlja se na 0. Time kvar postaje odmah vidljiv kućnom majstoru kroz pogled `view_aktivni_kvarovi`.
2. **Automatska blokada sobe:** Odmah nakon unosa naloga, status sobe mijenja se u 'IZVAN_FUNKCIJE'. *Važno:* Ova promjena statusa je ključna jer, u kombinaciji s okidačem `trg_sprijeci_rezervaciju_kvar`, sustav fizički onemogućuje recepcionerima kreiranje novih rezervacija za tu sobu sve dok se kvar ne otkloni.

Cilj procedure: Prevencija nezadovoljstva gostiju. Procedura osigurava trenutnu sinkronizaciju podataka, garantirajući da se soba s tehničkim nedostatkom ne može iznajmiti gostu, dok istovremeno generira službeni nalog za popravak.

6.3 Funkcije

6.3.1 Funkcija 1: IZRACUNAJ_MARZU_JELA

```
DELIMITER //

CREATE FUNCTION izracunaj_marzu_jela(p_usluga_id INT) RETURNS
   DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_prodajna DECIMAL(10,2);
    DECLARE v_trosak_nabave DECIMAL(10,2);

    SELECT cijena_trenutna INTO v_prodajna
    FROM usluga WHERE id = p_usluga_id;

    SELECT COALESCE(SUM(n.kolicina_potrosnje * a.nabavna_cijena),
       0)
    INTO v_trosak_nabave
    FROM normativ n
    JOIN artikl a ON n.artikl_id = a.id
    WHERE n.usluga_id = p_usluga_id;

    RETURN (v_prodajna - v_trosak_nabave);
END //

DELIMITER ;
```

Isječak koda 69: Funkcija za izračun marže jela

Opis: Funkcija `izracunaj_marzu_jela` predstavlja analitički alat unutar baze podataka. Za razliku od procedura koje izvršavaju transakcijske radnje, ova funkcija služi za izračun financijske isplativosti pojedinog artikla. Ona automatski uspoređuje prodajnu cijenu jela s nabavnom cijenom svih njegovih sastojaka kako bi generirala točan iznos zarade (marže) po jednoj porciji.

Parametri:

- `p_usluga_id` - (IN) ID usluge (jela ili pića) za koju se računa profitabilnost.

Povratna vrijednost:

- `DECIMAL(10,2)` - Iznos marže u valuti.

Logika izvršavanja: Proces izračuna odvija se u tri koraka:

1. **Dohvat prodajne cijene:** Funkcija iz tablice `usluga` dohvaća trenutnu prodajnu cijenu (`cijena_trenutna`) koju plaća gost i pohranjuje je u varijablu `v_prodajna`.
2. **Izračun troška namirnica (Food Cost):** Sustav računa koliko košta priprema jela:
   - *Povezivanje podataka:* Koristi se `JOIN` između tablica `normativ` i `artikl`.
   - *Matematički izračun:* Množi se količina potrošnje s nabavnom cijenom za svaki pojedinačni sastojak, a zatim se sve zbraja funkcijom `SUM`.

- *Sigurnosna provjera:* Funkcija koristi `COALESCE(..., 0)` kako bi osigurala da trošak bude 0 (a ne NULL) u slučajevima kada jelo nema definiranih normativa (npr. usluga rezanja torte), čime se sprječava rušenje izračuna.

3. **Finalni izračun:** Funkcija vraća rezultat operacije: Prodajna cijena - Trošak nabave.

Cilj funkcije: Omogućiti menadžmentu restorana brz i precizan uvid u profitabilnost jelovnika. Funkcija eliminira potrebu za ručnim kalkulacijama i omogućuje trenutnu identifikaciju jela s najvećom zaradom, kao i onih koja generiraju gubitke zbog visokih nabavnih cijena namirnica.

6.4 Pogledi

6.5 Pogledi (Views)

6.5.1 Pogled 1: PREGLED_SVIH_RACUNA

```sql
CREATE OR REPLACE VIEW pregled_svih_racuna AS
SELECT
    r.id AS racun_id,
    r.tip_racuna,
    r.status_racuna,
    r.iznos_ukupno,
    r.nacin_placanja,
    r.datum_izdavanja,
    r.rezervacija_id,

    COALESCE(CONCAT(g.ime, ' ', g.prezime), 'Vanjski Gost') AS
        gost_nositelj
FROM racun r
LEFT JOIN rezervacija rez ON r.rezervacija_id = rez.id
LEFT JOIN gost g ON rez.gost_nositelj_id = g.id;
```

Isječak koda 70: Centralizirani pregled svih računa

Opis: Pogled `pregled_svih_racuna` služi kao glavni financijski izvještaj za menadžment i računovodstvo. On apstrahira kompleksnost povezivanja više tablica i nudi jedinstvenu, pročišćenu listu svih izdanih računa unutar sustava, bez obzira na to jesu li nastali na recepciji (za goste hotela) ili u restoranu (za vanjske posjetitelje).

Logika izvršavanja: Dizajn ovog pogleda rješava problem fragmentiranih podataka kroz dva ključna mehanizma:

1. **Povezivanje podataka (LEFT JOIN):** Ključna karakteristika je korištenje `LEFT JOIN` naredbe.
   - Budući da nisu svi računi vezani uz rezervaciju (npr. vanjski gost na večeri), korištenje običnog `JOIN`-a sakrilo bi restoranske račune.
   - `LEFT JOIN` osigurava da se prikažu *svi* računi, a podaci o rezervaciji se dodaju samo ako postoje.
2. **Pametna identifikacija (COALESCE):** Pogled koristi funkciju `COALESCE` u kombinaciji s `CONCAT`.
   - Sustav pokušava spojiti ime i prezime gosta. Ako je rezultat `NULL` (račun nije vezan uz sobu), funkcija automatski zamjenjuje praznu vrijednost tekstom 'Vanjski Gost'.
   - Time se osigurava da izvještaj nema praznih polja u stupcu nositelja troška.

Cilj pogleda: Pojednostaviti izvještavanje servirsanjem gotove, ljudski čitljive tablice analitičarima, eliminirajući potrebu za pisanjem složenih upita svaki put kada je potreban uvid u promet.

6.5.2 Pogled 2: PREGLED_ZALIHA_SKLADISTE

```
CREATE OR REPLACE VIEW pregled_zaliha_skladiste AS
SELECT
    a.id AS artikl_id,
    a.naziv,
    a.stanje_zaliha,
    a.jedinica_mjere,
    a.nabavna_cijena,
    (a.stanje_zaliha * a.nabavna_cijena) AS ukupna_vrijednost
FROM artikl a;
```

Isječak koda 71: Pregled zaliha skladišta

Opis: Pogled `pregled_zaliha_skladiste` ključan je alat za inventuru i financijsko praćenje materijalne imovine hotela i restorana. On pruža trenutni uvid u stanje skladišta, ne samo količinski, već i financijski.

Logika izvršavanja: Pogled transformira sirove podatke u financijske informacije:

1. **Izračun vrijednosti imovine:** Glavna snaga ovog pogleda je generirani stupac `ukupna_vrijednost`.
2. **Automatizacija:** Umjesto ručnog množenja, pogled automatski provodi operaciju:

   $$Stanjezaliha \times Nabavnacijena$$

   Time se dobiva točan podatak o "zarobljenom" kapitalu u zalihama (npr. ukupna vrijednost kave na skladištu).
3. **Efikasnost:** Budući da nema složenih spajanja (JOIN), pogled je izuzetno brz i pogodan za često osvježavanje u aplikaciji.

Cilj pogleda: Financijska transparentnost skladišta. Omogućuje menadžmentu uvid u ukupnu vrijednost zaliha u realnom vremenu, što je nužno za planiranje budžeta nabave.

6.5.3 Pogled 3: PREGLED_SVIH_REZERVACIJA

```
CREATE OR REPLACE VIEW pregled_svih_rezervacija AS
SELECT
    r.id AS rezervacija_id,
    CONCAT(g.ime, ' ', g.prezime) AS gost,
    s.broj AS soba,
    r.pocetak_datum,
    r.kraj_datum,
    r.status,
    r.broj_osoba,
    r.napomena
FROM rezervacija r
JOIN gost g ON r.gost_nositelj_id = g.id
JOIN soba s ON r.soba_id = s.id
ORDER BY r.pocetak_datum DESC;
```

Isječak koda 72: Pregled svih rezervacija

Opis: Pogled `pregled_svih_rezervacija` služi kao glavni operativni izvještaj za recepciju. On transformira tehničke podatke iz tablice rezervacija (koja primarno sadrži ID-ove) u ljudski čitljiv format, omogućujući osoblju brz uvid u raspored dolazaka i odlazaka.

Logika izvršavanja: Pogled formira cjelovitu sliku kroz dva mehanizma:

1. **Povezivanje entiteta (JOIN):** Spajaju se tri ključne tablice:
   - *Gost:* Prikazuje se puno ime i prezime umjesto šifre gosta (`CONCAT`).
   - *Soba:* Prikazuje se fizički broj sobe (npr. "101") umjesto sistemskog ID-a.
2. **Kronološko sortiranje:** Rezultati su sortirani prema datumu početka rezervacije silazno (`ORDER BY ... DESC`). Time se na vrhu popisa uvijek prikazuju najnovije ili buduće rezervacije, olakšavajući recepcionerima fokus na nadolazeće goste.

Cilj pogleda: Pružiti recepciji jasan "master" popis. Eliminira se potreba za ručnim pretraživanjem i omogućuje brzo odgovaranje na upite o zauzetosti kapaciteta.

6.5.4 Pogled 4: VIEW_MENI_RESTORAN

```sql
CREATE OR REPLACE VIEW view_meni_restoran AS
SELECT id, naziv, cijena_trenutna, kategorija_id
FROM usluga
WHERE kategorija_id IN (1, 2);
```

Isječak koda 73: Restoranski meni (Hrana i Piće)

Opis: Pogled `view_meni_restoran` služi kao digitalni cjenik prilagođen operativnom radu restorana. Iako centralna tablica usluga sadrži sve stavke hotela (uključujući noćenja i wellness), ovaj pogled izolira samo ono što je relevantno za konobare.

Logika izvršavanja: Optimizacija prikaza postiže se filtriranjem:

1. **Filtriranje sadržaja (WHERE):** Ključna je uvjetna naredba `WHERE kategorija_id IN (1, 2)`.
2. **Kontekst:** U ovom modelu, kategorija 1 označava hranu, a kategorija 2 piće. Ovim filterom se automatski uklanjaju sve ostale vrste usluga (npr. smještaj, parking), čime se dobiva "čista" lista artikala koji se fizički poslužuju.
3. **Selekcija stupaca:** Prikazuju se samo ID, naziv i cijena, skrivajući nepotrebne tehničke detalje.

Cilj pogleda: Optimizacija POS (Point of Sale) sustava. Osigurava se da konobari prilikom unosa narudžbe vide isključivo relevantnu ponudu hrane i pića, čime se ubrzava rad i smanjuje mogućnost pogreške.

6.5.5 Pogled 5: VIEW_KUHINJA_DISPLAY

```sql
CREATE OR REPLACE VIEW view_kuhinja_display AS
SELECT
    rs.id AS stavka_id,
    rs.narudzba_id,
    u.naziv AS naziv_jela,
    rs.kolicina,
```

```
    rs.status_pripreme,
    u.kategorija_id
FROM restoran_stavka rs
JOIN usluga u ON rs.usluga_id = u.id
WHERE rs.status_pripreme IN ('NARUCENO', 'PRIPREMA');
```

Isječak koda 74: Kuhinjski monitor (KDS)

Opis: Pogled `view_kuhinja_display` simulira digitalni zaslon u kuhinji (Kitchen Display System - KDS). Njegova svrha je zamijeniti papirnate bonove i omogućiti kuharima i šankerima jasan, digitalni pregled onoga što trenutno trebaju pripremiti.

Logika izvršavanja: Fokus je na trenutnoj aktivnosti:

1. **Filtriranje aktivnih narudžbi:** Ključna logika nalazi se u `WHERE` klauzuli koja propušta samo statuse 'NARUCENO' i 'PRIPREMA'.
2. **Isključivanje završenog:** Pogled automatski uklanja stavke koje su 'POSLUZENO' ili 'STORNIRANO'. Time se osigurava da se na ekranu ne gomilaju stare narudžbe, već je fokus isključivo na tekućim zadacima.
3. **Kategorizacija:** Dohvaća se `kategorija_id`, što omogućuje aplikaciji da filtrira prikaz (npr. ekran na šanku prikazuje samo pića, a ekran u kuhinji samo hranu).

Cilj pogleda: Organizacija rada u kuhinji u realnom vremenu. Smanjuje se mogućnost greške, gubitka papirnatih narudžbi i ubrzava komunikacija između konobara i kuhara.

6.5.6 Pogled 6: VIEW_OTVORENE_NARUDZBE_TOTAL

```
CREATE OR REPLACE VIEW view_otvorene_narudzbe_total AS
SELECT
    rn.id AS narudzba_id,
    rs.broj_stola,
    rs.lokacija,
    COALESCE(SUM(rst.kolicina * rst.cijena_u_trenutku), 0) AS
       total_iznos
FROM restoran_narudzba rn
JOIN restoran_stol rs ON rn.restoran_stol_id = rs.id
LEFT JOIN restoran_stavka rst ON rn.id = rst.narudzba_id
WHERE rn.status = 'OTVORENA'
GROUP BY rn.id, rs.broj_stola, rs.lokacija;
```

Isječak koda 75: Ukupni zbroj otvorenih stolova

Opis: Pogled `view_otvorene_narudzbe_total` pruža konobarima i voditeljima smjene brzi financijski i prostorni pregled sale. On prikazuje koliko novca trenutno "leži" na otvorenim stolovima koji još nisu naplaćeni, uz preciznu informaciju o lokaciji stola.

Logika izvršavanja: Pogled vrši agregaciju podataka u realnom vremenu:

1. **Agregacija i izračun:** Koristi se funkcija `SUM` za zbrajanje vrijednosti svih stavki na narudžbi (`kolicina * cijena`). Podaci su grupirani tako da daju jedinstveni iznos za svaki aktivni stol.

2. **Rukovanje praznim vrijednostima:** Koristi se `LEFT JOIN` i `COALESCE(..., 0)`. Ovo je bitno za slučajeve kada je stol otvoren u sustavu (gosti su sjeli), ali još nisu ništa naručili. Umjesto greške ili `NULL` vrijednosti, sustav uredno prikazuje iznos 0.00.

3. **Prostorni kontekst:** Stupac `lokacija` omogućuje filtriranje prihoda po zonama (npr. 'Terasa', 'Glavna sala'), što olakšava raspodjelu konobara po reonima.

Cilj pogleda: Monitoring prihoda i stanja u sali. Voditelj smjene može jednim pogledom procijeniti opterećenje restorana i financijsko stanje otvorenih računa prije zatvaranja dana.

6.5.7 Pogled 7: VIEW_SOBE_ZA_CISCENJE

```sql
CREATE OR REPLACE VIEW view_sobe_za_ciscenje AS
SELECT s.id, s.broj, ts.naziv AS tip_sobe, s.status
FROM soba s
JOIN tip_sobe ts ON s.tip_sobe_id = ts.id
WHERE s.status = 'CISCENJE';
```

Isječak koda 76: Lista soba za čišćenje

Opis: Pogled `view_sobe_za_ciscenje` osnovni je alat za odjel domaćinstva. On filtrira cjelokupni popis soba u hotelu i prikazuje isključivo one koje zahtijevaju pažnju čistačica/čistača. Prikazuje se ID, fizički broj sobe, te tip sobe (apartman vs. standard) što je bitno za procjenu potrebnog vremena za rad.

Logika izvršavanja:

1. **Filtriranje statusa:** Pogled je optimiziran za brzinu jer koristi direktan filter: `WHERE s.status = 'CISCENJE'`.

2. **Automatizacija:** Ovaj status se ne postavlja ručno, već je rezultat okidača `trg_auto_ciscenje_r` koji se aktivira čim gost napusti sobu.

Cilj pogleda: Optimizacija rada domaćinstva. Umjesto provjere svake sobe, osoblje na svojim uređajima vidi filtriranu "To-Do" listu, što značajno ubrzava proces pripreme hotela za nove goste.

6.5.8 Pogled 8: VIEW_AKTIVNE_REZERVACIJE

```sql
CREATE OR REPLACE VIEW view_aktivne_rezervacije AS
SELECT
    r.id AS rezervacija_id,
    s.broj AS broj_sobe,
    g.ime,
    g.prezime
FROM rezervacija r
JOIN soba s ON r.soba_id = s.id
JOIN gost g ON r.gost_nositelj_id = g.id
WHERE r.status = 'U_TIJEKU';
```

Isječak koda 77: Gosti trenutno u hotelu (In-House)

Opis: Pogled `view_aktivne_rezervacije` ključan je za recepciju i službu sigurnosti. Za razliku od pregleda svih rezervacija (koji uključuje povijesne i buduće podatke), ovaj pogled fokusira se isključivo na goste koji se u ovom trenutku nalaze u hotelu (tzv. "In-House" lista).

Logika izvršavanja:

1. **Fokus na aktivni boravak:** Filter `WHERE r.status = 'U_TIJEKU'` je srž ovog pogleda. On eliminira sve najave dolazaka i arhivirane rezervacije.
2. **Povezivanje entiteta:** Spajanjem tablica soba i gostiju dobiva se jasna slika tko boravi u kojoj sobi.

Cilj pogleda: Brza identifikacija i sigurnost. Koristi se za provjeru identiteta gosta (npr. zaboravljen ključ) te kao primarni popis u hitnim slučajevima (evakuacijska lista).

6.5.9 Pogled 9: VIEW_DODATNE_USLUGE

```sql
CREATE OR REPLACE VIEW view_dodatne_usluge AS
SELECT id, naziv, cijena_trenutna
FROM usluga
WHERE kategorija_id IN (3, 4);
```

Isječak koda 78: Cjenik dodatnih usluga (Wellness i Ostalo)

Opis: Pogled `view_dodatne_usluge` komplementaran je pogledu za restoranski meni. Dok restoran koristi popis hrane i pića, recepciju i wellness zanima cjenik ostalih usluga koje se mogu staviti na račun sobe (npr. masaže, najam bicikla, parking).

Logika izvršavanja:

1. **Filtriranje kategorija:** Pogled koristi filter `WHERE kategorija_id IN (3, 4)`. U ovom modelu podataka, to označava 'Wellness' i 'Ostale usluge'.
2. **Segregacija:** Ovime se izbjegava miješanje s tisućama artikala hrane i pića, čineći listu preglednom.

Cilj pogleda: Segregacija ponude radi efikasnosti. Odjeljivanje restoranske ponude od ostalih usluga smanjuje mogućnost ljudske pogreške pri odabiru stavke za naplatu i ubrzava rad recepcionera.

6.5.10 Pogled 10: VIEW_AKTIVNI_KVAROVI

```sql
CREATE OR REPLACE VIEW view_aktivni_kvarovi AS
SELECT
    n.id AS nalog_id,
    CONCAT(z.ime, ' ', z.prezime) AS serviser,
    s.broj AS broj_sobe,
    n.soba_id,
    n.opis AS opis_kvara,
    n.datum_naloga,
    n.rijeseno
FROM servis_dnevni_nalog n
JOIN zaposlenik z ON n.zaposlenik_id = z.id
JOIN soba s ON n.soba_id = s.id
```

```
WHERE n.rijeseno = 0;
```

Isječak koda 79: Nerealizirani servisni nalozi

Opis: Pogled `view_aktivni_kvarovi` predstavlja digitalnu "To-Do" listu za kućnog majstora. On centralizira sve prijavljene probleme u hotelu koji još nisu riješeni, omogućujući brz pregled prioriteta održavanja.

Logika izvršavanja:

1. **Status rješenja:** Ključan filter je `WHERE n.rijeseno = 0`. Pogled automatski skriva sve popravke koji su završeni, držeći listu čistom i fokusiranom na tekuće probleme.
2. **Povezivanje podataka:** Koriste se `JOIN` operacije prema tablici `zaposlenik` (tko je zadužen) i `soba` (lokacija kvara).
3. **Tehnički detalj:** Pogled eksplicitno vraća `soba_id`. Iako korisniku treba broj sobe, aplikaciji treba ID kako bi mogla programski pristupiti toj sobi (npr. automatski je blokirati za prodaju dok je kvar aktivan).

Cilj pogleda: Efikasno održavanje objekta. Ovaj pogled osigurava da niti jedan prijavljeni kvar (npr. "ne radi klima u sobi 202") ne bude zaboravljen, već ostaje vidljiv u sustavu sve dok ga majstor ne označi kao riješenog.

6.5.11 Pogled 11: VIEW_OSOBLJE_ODRZAVANJA

```
CREATE OR REPLACE VIEW view_osoblje_odrzavanja AS
SELECT
    z.id,
    z.ime,
    z.prezime,
    CONCAT(z.ime, ' ', z.prezime) AS puno_ime
FROM zaposlenik z
JOIN odjel o ON z.odjel_id = o.id
WHERE o.naziv = 'Odrzavanje';
```

Isječak koda 80: Zaposlenici odjela održavanja

Opis: Pogled `view_osoblje_odrzavanja` filtrira bazu zaposlenika kako bi prikazao isključivo osoblje koje je kvalificirano za rješavanje tehničkih kvarova.

Logika izvršavanja:

1. **Filtriranje po odjelu:** Ključna logika leži u `JOIN` operaciji s tablicom `odjel` uz uvjet `WHERE o.naziv = 'Odrzavanje'`.
2. **Segregacija:** Ovime se iz popisa za dodjelu servisnih naloga automatski isključuju recepcioneri, konobari i kuhari, sprječavajući pogrešnu alokaciju zadataka.

Cilj pogleda: Integritet korisničkog sučelja. Cilj ovog pogleda je popunjavanje padajućih izbornika (dropdown) u aplikaciji. Kada recepcioner prijavljuje kvar, sustav mu nudi da odabere majstora isključivo iz ovog filtriranog popisa, čime se eliminira mogućnost ljudske pogreške.

6.5.12 Pogled 12: VIEW_STATUS_SOBA_BOJE

```sql
CREATE OR REPLACE VIEW view_status_soba_boje AS
SELECT
    s.id,
    s.broj,
    ts.trenutno_stanje AS stanje,
    CASE
        WHEN ts.trenutno_stanje = 'ZAUZETA' THEN '#ffcdd2'
        WHEN ts.trenutno_stanje = 'SLOBODNA' THEN '#c8e6c9'
        WHEN ts.trenutno_stanje = 'CISCENJE' THEN '#fff9c4'
        WHEN ts.trenutno_stanje = 'IZVAN_FUNKCIJE' THEN '#e0e0e0'
        ELSE '#ffffff'
    END AS hex_boja
FROM soba s
JOIN trenutno_stanje_soba ts ON s.id = ts.soba_id;
```

Isječak koda 81: Vizualizacija statusa soba (Boje)

Opis: Pogled `view_status_soba_boje` djeluje kao "prevoditelj" podataka za grafičko sučelje aplikacije. Umjesto da frontend aplikacija sadrži logiku o bojama, baza podataka centralizirano definira vizualni identitet svakog statusa.

Logika izvršavanja:

1. **Logika mapiranja boja (CASE):** Pogled koristi `CASE` naredbu za dodjelu HEX kodova boja:
   - **#ffcdd2 (Crvena):** Soba je zauzeta.
   - **#c8e6c9 (Zelena):** Soba je slobodna.
   - **#fff9c4 (Žuta):** Soba je u procesu čišćenja.
   - **#e0e0e0 (Siva):** Soba je izvan funkcije (kvar).

Cilj pogleda: Kreiranje "Dashboarda" (nadzorne ploče). Aplikacija iscrtava tlocrt hotela koristeći boje koje vrati ovaj pogled, čime se recepcionerima omogućuje trenutna vizualna procjena zauzetosti hotela bez čitanja teksta.

6.5.13 Pogled 13: VIEW_SVE_SOBE_DROPDOWN

```sql
CREATE OR REPLACE VIEW view_sve_sobe_dropdown AS
SELECT
    s.id,
    s.broj,
    t.naziv AS tip_sobe,
    s.status
FROM soba s
JOIN tip_sobe t ON s.tip_sobe_id = t.id
ORDER BY s.broj;
```

Isječak koda 82: Detaljni popis soba za odabir

Opis: Pogled `view_sve_sobe_dropdown` koristi se u formama kada korisnik mora odabrati sobu. Sama informacija o broju sobe često nije dovoljna za donošenje ispravne odluke prilikom rezervacije.

Logika izvršavanja:

1. **Obogaćivanje informacija:** Spajanjem s tablicom `tip_sobe`, ovaj pogled daje nužan kontekst broju sobe.
2. **Sortiranje:** Podaci su sortirani po broju sobe radi lakšeg snalaženja.

Cilj pogleda: Poboljšanje korisničkog iskustva (UX). Recepcioner u padajućem izborniku vidi npr. "101 - Jednokrevetna" umjesto samo "101". To sprječava operativne greške, poput dodjeljivanja premale sobe obitelji s djecom ili dodjeljivanja apartmana gostu koji je platio standardnu sobu.

6.5.14 Pogled 14: VIEW_STAVKE_NARUDZBE

```sql
CREATE OR REPLACE VIEW view_stavke_narudzbe AS
SELECT
    rs.narudzba_id,
    u.naziv AS naziv_artikla,
    rs.kolicina,
    rs.cijena_u_trenutku AS cijena_jedinicna,
    (rs.kolicina * rs.cijena_u_trenutku) AS ukupno_stavka,
    rs.status_pripreme
FROM restoran_stavka rs
JOIN usluga u ON rs.usluga_id = u.id;
```

Isječak koda 83: Stavke unutar narudžbe

Opis: Pogled `view_stavke_narudzbe` prikazuje detaljan sadržaj pojedine narudžbe prije nego što ona postane finalni račun. Služi kao operativni prozor u trenutno stanje konzumacije na stolovima.

Logika izvršavanja:

1. **Obogaćivanje podataka:** Putem `JOIN` operacije s tablicom `usluga`, ID-ovi artikala zamjenjuju se čitljivim nazivima (npr. "Espresso").
2. **Kalkulacija u realnom vremenu:** Pogled automatski računa vrijednost svake stavke formulom:

$$Ukupno = Koliina \times Cijenautrenutku$$

3. **Status pripreme:** Prikazuje se status (npr. 'NARUCENO' vs 'POSLUZENO'), što konobarima omogućuje praćenje toka servisa.

Cilj pogleda: Prikaz detalja na ekranu blagajne (POS). Konobar može u svakom trenutku vidjeti rekapitulaciju narudžbe stola i trenutni iznos zaduženja prije samog izdavanja računa.

6.5.15 Pogled 15: VIEW_DETALJI_RACUNA

```
CREATE OR REPLACE VIEW view_detalji_racuna AS
SELECT
    racun_id,
    tip_stavke,
    opis,
    kolicina,
    cijena_jedinicna,
    iznos_ukupno
FROM stavka_racuna;
```

Isječak koda 84: Detalji finaliziranog računa

Opis: Pogled `view_detalji_racuna` služi za pregled stavki na računima koji su već službeno kreirani i zaključeni.

Logika izvršavanja:

1. **Izvor podataka:** Za razliku od prethodnog pogleda koji čita iz "žive" tablice restorana, ovaj pogled čita iz tablice `stavka_racuna`.
2. **Imutabilnost:** Ovo je ključna razlika – podaci u ovom pogledu predstavljaju trajni zapis. Čak i ako se cijena kave u cjeniku promijeni sutra, ovaj pogled će prikazivati cijenu koja je vrijedila u trenutku izdavanja računa.

Cilj pogleda: Arhivski pregled i revizija. Koristi se za ispis kopija računa, rješavanje reklamacija i financijsku inspekciju, osiguravajući da povijesni podaci ostanu nepromijenjeni.

6.5.16 Pogled 16: VIEW_RECEPTURE_DETALJNO

```
CREATE OR REPLACE VIEW view_recepture_detaljno AS
SELECT
    u.id AS jelo_id,
    u.naziv AS naziv_jela,
    a.naziv AS namirnica,
    n.kolicina_potrosnje,
    a.jedinica_mjere,
    a.nabavna_cijena AS cijena_po_jedinici,
    ROUND(n.kolicina_potrosnje * a.nabavna_cijena, 2) AS
        trosak_sastojka
FROM usluga u
JOIN normativ n ON u.id = n.usluga_id
JOIN artikl a ON n.artikl_id = a.id
ORDER BY u.naziv, a.naziv;
```

Isječak koda 85: Detaljna analiza normativa (Recepture)

Opis: Pogled `view_recepture_detaljno` je moćan analitički alat za kontrolu troškova (Food Cost). On "razbija" svako jelo na sastavne dijelove, prikazujući točnu strukturu normativa i cijenu svakog pojedinog sastojka.

Logika izvršavanja:

1. **Trostruko povezivanje:** Da bi se dobila potpuna slika, pogled spaja tri tablice:

$$Usluga(Jelo) \rightarrow Normativ(Veza) \rightarrow Artikl(Sastojak)$$

2. **Izračun troška:** Funkcija `ROUND` koristi se za automatski izračun troška specifičnog sastojka u jelu formulom:

$$Troak = Koliinapotronje \times Nabavnacijena$$

Cilj pogleda: Menadžerska analiza profitabilnosti. Voditelj restorana može vidjeti koliko točno novca odlazi na meso, a koliko na prilog u određenom jelu, te na temelju toga korigirati prodajne cijene kako bi osigurao željenu maržu.

6.5.17 Pogled 17: VIEW_ZAPOSLENICI_RECEPCIJA

```sql
CREATE OR REPLACE VIEW view_zaposlenici_recepcija AS
SELECT
    id,
    ime,
    prezime,
    CONCAT(ime, ' ', prezime) AS puno_ime,
    korisnicko_ime
FROM zaposlenik
WHERE odjel_id = 1;
```

Isječak koda 86: Zaposlenici recepcije

Opis: Pogled `view_zaposlenici_recepcija` filtrira bazu zaposlenika kako bi izdvojio samo one djelatnike koji rade na recepciji. Ovo je bitno jer recepcioneri imaju specifične ovlasti u aplikaciji, poput pristupa sustavu rezervacija.

Logika izvršavanja:

1. **Filtriranje po odjelu:** Ključna logika je u uvjetu `WHERE odjel_id = 1`. Sustav koristi ID 1 za identifikaciju odjela "Recepcija".
2. **Sigurnosni podaci:** Za razliku od ostalih pogleda, ovaj vraća i stupac `korisnicko_ime`, što omogućuje korištenje ovog pogleda za autentifikaciju (Login) i auditiranje (bilježenje tko je napravio rezervaciju).

Cilj pogleda: Sigurnost i organizacija. Osigurava da se na mjestima gdje se traži "Operater sustava" nude samo kvalificirani recepcioneri, sprječavajući neovlašteni pristup ili pogrešnu dodjelu odgovornosti.

6.5.18 Pogled 18: VIEW_ZAPOSLENICI_KONOBARI

```sql
CREATE OR REPLACE VIEW view_zaposlenici_konobari AS
SELECT
    id,
    ime,
    prezime,
    CONCAT(ime, ' ', prezime) AS puno_ime
FROM zaposlenik
```

```
WHERE odjel_id = 5;
```

Isječak koda 87: Zaposlenici restorana (Konobari)

Opis: Pogled `view_zaposlenici_konobari` služi kao izvor podataka za POS (Point of Sale) sustav restorana.

Logika izvršavanja:

1. **Filtriranje osoblja:** Uvjet `WHERE odjel_id = 5` osigurava da se dohvaćaju isključivo djelatnici restorana.
2. **Formatiranje:** Kreira se `puno_ime` spremno za ispis na dnu fiskalnog računa ("Račun izdao: Ivan Ivić").

Cilj pogleda: Ubrzanje rada i smanjenje grešaka. Kada konobar otvara novu narudžbu, aplikacija mu nudi izbor imena samo s ove liste. Time se eliminira mogućnost da se narudžba greškom pripiše recepcioneru, sobarici ili domaru.

6.5.19 Pogled 19: VIEW_ZAPOSLENICI_DOMACINSTVO

```
CREATE OR REPLACE VIEW view_zaposlenici_domacinstvo AS
SELECT
    id,
    ime,
    prezime,
    CONCAT(ime, ' ', prezime) AS puno_ime
FROM zaposlenik
WHERE odjel_id = 2;
```

Isječak koda 88: Zaposlenici domaćinstva

Opis: Pogled `view_zaposlenici_domacinstvo` koristi se kao temeljni šifrarnik u modulu za održavanje čistoće soba. Njegova svrha je izolirati popis osoblja zaduženog za higijenu hotela.

Logika izvršavanja:

1. **Specifičnost odjela:** Sustav filtrira zaposlenike prema `WHERE odjel_id = 2`, što u ovoj strukturi baze predstavlja odjel "Domaćinstvo".
2. **Formatiranje:** Ime i prezime se spajaju radi lakšeg prikaza na malim ekranima (poput tableta koje koriste čistačice).

Cilj pogleda: Točna evidencija rada i odgovornosti. Prilikom izvršavanja procedure za evidenciju čišćenja (`proc_evidentiraj_ciscenje`), sustav mora zabilježiti tko je točno očistio sobu. Ovaj pogled osigurava da se u padajućem izborniku aplikacije nalaze isključivo imena čistačica/čistača, što ubrzava unos podataka na terenu i smanjuje mogućnost greške.

6.5.20 Pogled 20: VIEW_ZAPOSLENICI_ODRZAVANJE

```
CREATE OR REPLACE VIEW view_zaposlenici_odrzavanje AS
SELECT
    id,
```

```
    ime,
    prezime,
    CONCAT(ime, ' ', prezime) AS puno_ime
FROM zaposlenik
WHERE odjel_id = 3;
```

Isječak koda 89: Zaposlenici tehničke službe (Održavanje)

Opis: Pogled `view_zaposlenici_odrzavanje` služi za filtriranje kućnih majstora i tehničara. Iako sličan prethodnim pogledima za zaposlenike, njegova uloga je ključna u procesu upravljanja incidentima i kvarovima.

Logika izvršavanja:

1. **Jednostavno filtriranje:** Umjesto spajanja (`JOIN`) s tablicom odjela radi provjere naziva, ovaj pogled koristi direktan filter `WHERE odjel_id = 3`. Ovo je optimizacija koja osigurava brže izvođenje upita u aplikaciji.
2. **Selekcija:** Prikazuju se samo ID (za bazu) i puno ime (za korisnika).

Cilj pogleda: Efikasna dodjela radnih naloga. Kada se u sustavu prijavljuje kvar (bilo putem procedure ili kroz `view_aktivni_kvarovi`), voditelj hotela mora dodijeliti zadatak konkretnoj stručnoj osobi. Ovaj pogled eliminira informacijski "šum" i u izborniku za dodjelu zadataka prikazuje samo relevantno tehničko osoblje, sprječavajući da se popravak klime greškom dodijeli konobaru.

7. Alma

7.1 A. Upravljanje Recenzijama

```sql
ALTER TABLE recenzija ADD COLUMN odgovor_hotela TEXT DEFAULT NULL;

-- View koji prikazuje recenzije zajedno sa odgovorima hotela
CREATE OR REPLACE VIEW pregled_recenzija_za_menadzera AS
SELECT
    recenzija.rezervacija_id,
    CONCAT(gost.ime, ' ', gost.prezime) AS 'Ime gosta',
    recenzija.ocjena AS 'Ocjena',
    recenzija.komentar AS 'Komentar gosta',
    COALESCE(recenzija.odgovor_hotela, 'Nema odgovora') AS 'Odgovor hotela',
    CASE
        WHEN recenzija.odgovor_hotela IS NULL THEN 'Potrebna akcija'
        ELSE 'Rijeseno'
    END AS 'Status'
FROM recenzija
JOIN rezervacija ON recenzija.rezervacija_id = rezervacija.id
JOIN gost ON rezervacija.gost_nositelj_id = gost.id
ORDER BY recenzija.odgovor_hotela IS NOT NULL, recenzija.ocjena ASC;
```

Isječak koda 90: Dodavanje odgovora hotela i pogled za menadžere

1. Pogled (View): pregled_recenzija_za_menadzera

Opis: Pogled `pregled_recenzija_za_menadzera` predstavlja centralizirani nadzorni alat (virtualnu tablicu) namijenjen menadžmentu hotela za učinkovito praćenje zadovoljstva gostiju. Njegova svrha je transformacija sirovih podataka u organiziranu cjelinu koja omogućuje brzo operativno djelovanje.

Logika i funkcionalnost: Dizajn ovog pogleda oslanja se na tri ključna elementa:

1. **Agregacija podataka (JOIN):** Pogled izvodi operaciju spajanja nad tablicama `recenzija`, `rezervacija` i `gost`. Time se izolirana brojčana ocjena povezuje s konkretnim imenom gosta i detaljima njegovog boravka, dajući nužan kontekst svakoj kritici ili pohvali.
2. **Pametna logika statusa (Business Logic):** Unutar pogleda implementirana je `CASE` logika koja automatski evaluira stupac odgovora:
   - Sustav provjerava je li polje `odgovor_hotela` prazno (`NULL`).
   - Ukoliko odgovor nedostaje, sustav generira status **'Potrebna akcija'**, jasno signalizirajući menadžeru prioritetni zadatak.
   - U suprotnom, status se označava kao **'Riješeno'**.
3. **Prioritizacija prikaza (ORDER BY):** Rezultat je automatski sortiran tako da se recenzije koje zahtijevaju hitnu reakciju (neodgovorene) nalaze na samom vrhu

liste, sprječavajući da se izgube među starijim, riješenim predmetima.

Cilj pogleda: Eliminirati potrebu za ručnim pretraživanjem. Pogled služi kao dinamična "To-Do" lista za odjel odnosa s gostima, osiguravajući da niti jedna recenzija ne ostane zanemarena, čime se direktno utječe na online reputaciju hotela.

2. Procedura: odgovori_na_recenziju

```
DROP PROCEDURE IF EXISTS odgovori_na_recenziju;

DELIMITER //
CREATE PROCEDURE odgovori_na_recenziju(IN p_rezervacija_id INT, IN
    p_tekst_odgovor TEXT)
BEGIN
    DECLARE v_postoji INT;

    -- provjera da li recenzija postoji
    SELECT COUNT(*) INTO v_postoji
    FROM recenzija
    WHERE rezervacija_id = p_rezervacija_id;

    IF v_postoji = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Recenzija za ovu rezervaciju ne postoji
            .';
    END IF;

    -- upisi odgovor
    UPDATE recenzija
    SET odgovor_hotela = p_tekst_odgovor
    WHERE rezervacija_id = p_rezervacija_id;

    -- prikazi rezultat da vidis kako izgleda
    SELECT
        rezervacija_id,
        komentar AS "Komentar gosta",
        odgovor_hotela AS "Odgovor hotela"
    FROM recenzija
    WHERE rezervacija_id = p_rezervacija_id;

END //
DELIMITER ;
```

Isječak koda 91: Procedura za odgovaranje na recenzije

Opis: Procedura `odgovori_na_recenziju` ključni je alat za upravljanje reputacijom hotela. Ona omogućuje menadžmentu slanje službenog odgovora na recenziju gosta, uz ugrađeni validacijski mehanizam koji čuva integritet podataka sprječavanjem manipulacije nepostojećim zapisima.

Ulazni parametri:

- `p_rezervacija_id (IN)`: Jedinstveni identifikator rezervacije na koju se odnosi

recenzija.

- `p_tekst_odgovor (IN)`: Tekstualni sadržaj službenog odgovora hotela.

Koraci izvršavanja:

1. **Provjera postojanja recenzije:** Procedura započinje sigurnosnom provjerom. Izvršava se upit nad tablicom `recenzija` kako bi se utvrdilo postoji li zapis vezan za proslijeđeni ID.
   - Ukoliko recenzija ne postoji (npr. gost još nije napisao komentar, a sustav pokušava odgovoriti), procedura aktivira `SIGNAL SQLSTATE` naredbu. Ovo trenutno zaustavlja proces i aplikaciji vraća jasnu poruku o grešci, sprječavajući "ažuriranje u prazno".
2. **Ažuriranje zapisa (UPDATE):** Nakon uspješne validacije, izvršava se `UPDATE` naredba. U bazu se upisuje tekst odgovora, čime se status te recenzije (vidljiv u pogledu `pregled_recenzija_za_menadzera`) automatski mijenja iz 'Potrebna akcija' u 'Riješeno'.
3. **Potvrda i prikaz (Feedback):** Na samom kraju, procedura vraća ažurirani par podataka (izvorni komentar gosta i novi odgovor hotela). Ovo omogućava frontend aplikaciji da menadžeru odmah prikaže kako njegov odgovor izgleda "uživo", bez potrebe za osvježavanjem stranice.

Cilj procedure: Osigurati kontrolirani i sigurni kanal komunikacije. Procedura tehnički onemogućava logičke pogreške i osigurava da je interakcija s gostom odmah evidentirana i vidljiva u sustavu.

7.2 B. Rezervacije i Provjera Dostupnosti

3. Funkcija: provjeri_dostupnost_sobe

```
DROP FUNCTION IF EXISTS provjeri_dostupnost_sobe;

DELIMITER //

CREATE FUNCTION provjeri_dostupnost_sobe(
    p_soba_id INT,
    p_datum_dolaska DATE,
    p_datum_odlaska DATE
) RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE v_zauzetost_broj INT DEFAULT 0;

    -- Logika presjeka vremenskih intervala
    SELECT COUNT(*) INTO v_zauzetost_broj
    FROM rezervacija
    WHERE soba_id = p_soba_id
        AND status != 'OTKAZANA'
        AND pocetak_datum < p_datum_odlaska
        AND kraj_datum > p_datum_dolaska;
```

```
    -- Vraca 1 (TRUE) ako je soba slobodna, 0 (FALSE) ako je
       zauzeta
    RETURN v_zauzetost_broj = 0;
END //

DELIMITER ;
```

Isječak koda 92: Funkcija za provjeru dostupnosti sobe

Opis: Funkcija `provjeri_dostupnost_sobe` je fundamentalna sigurnosna komponenta sustava rezervacija (tzv. "low-level check"). Njezina isključiva zadaća je osigurati integritet kalendara zauzetosti i spriječiti pojavu dvostrukih rezervacija (overbooking).

Logika i funkcionalnost:

1. **Detekcija preklapanja termina:** Funkcija prima ID konkretne sobe te željeni datum dolaska i odlaska. Zatim pretražuje tablicu rezervacije tražeći bilo koji zapis koji se vremenski preklapa s traženim terminom.
2. **Filtriranje statusa:** Prilikom provjere, funkcija inteligentno ignorira rezervacije koje imaju status 'OTKAZANA'. To znači da, ako je netko otkazao boravak, termin se odmah tretira kao slobodan bez potrebe za brisanjem zapisa.
3. **Binarni rezultat:** Funkcija vraća TRUE (1) isključivo ako nema niti jednog preklapanja s aktivnim rezervacijama. U suprotnom vraća FALSE (0).

Cilj funkcije: Služiti kao atomarna provjera koju koriste sve ostale, složenije procedure sustava prije bilo kakvog pokušaja upisa nove rezervacije.

4. Funkcija: provjeri_dostupnost_kapaciteta

```
DELIMITER //

CREATE FUNCTION provjeri_dostupnost_kapaciteta(
    p_broj_osoba INT,
    p_datum_dolaska DATE,
    p_datum_odlaska DATE
) RETURNS BOOLEAN
DETERMINISTIC
BEGIN
    DECLARE v_broj_slobodnih_soba INT DEFAULT 0;

    -- Tra imo broj soba koje su:
    -- 1. Dovoljno velike (kapacitet)
    -- 2. Slobodne u tra enom terminu (poziv druge funkcije)
    SELECT COUNT(*) INTO v_broj_slobodnih_soba
    FROM soba
    WHERE kapacitet_osoba >= p_broj_osoba
      AND provjeri_dostupnost_sobe(soba.id, p_datum_dolaska,
         p_datum_odlaska) = 1;

    -- Vra a TRUE ako postoji barem jedna takva soba
    RETURN v_broj_slobodnih_soba >= 1;
```

```
END //

DELIMITER ;
```

Isječak koda 93: Funkcija za provjeru dostupnosti odgovarajućeg kapaciteta

Opis: Ova funkcija služi kao prvi filtar u procesu pretrage smještaja. Dok prethodna funkcija gleda konkretnu sobu, `provjeri_dostupnost_kapaciteta` odgovara na općenito pitanje: "Imamo li mi uopće išta slobodno za X ljudi u tom terminu?"

Logika i funkcionalnost:

1. **Validacija kapaciteta:** Funkcija analizira sve sobe u hotelu i filtrira samo one čiji kapacitet zadovoljava traženi broj osoba (ulazni parametar).
2. **Iterativna provjera:** Za svaku sobu koja zadovoljava uvjet kapaciteta, ova funkcija poziva prethodno opisanu funkciju `provjeri_dostupnost_sobe`.
3. **Brzi odgovor:** Čim pronađe barem jednu sobu koja je i dovoljno velika i slobodna, funkcija vraća TRUE.

Cilj funkcije: Optimizacija procesa. Omogućuje sustavu da brzo obavijesti korisnika "Nažalost, nemamo slobodnih soba za vaše potrebe" prije nego što se pokrenu složeni procesi izračuna cijena i detaljnog prikaza.

5. Procedura: dohvati_slobodne_sobe

```
DELIMITER //

CREATE PROCEDURE dohvati_slobodne_sobe(
    IN p_broj_osoba INT,
    IN p_datum_dolaska DATE,
    IN p_datum_odlaska DATE
)
BEGIN
    SELECT
        broj,
        kapacitet_osoba,
        naziv AS tip_sobe,
        CONCAT(izracunaj_cijenu_smjestaja(soba.id, p_datum_dolaska,
            p_datum_odlaska), ' ') AS ukupna_cijena
    FROM soba
    LEFT JOIN tip_sobe ON soba.tip_sobe_id = tip_sobe.id
    WHERE provjeri_dostupnost_sobe(soba.id, p_datum_dolaska,
        p_datum_odlaska) = 1
      AND kapacitet_osoba >= p_broj_osoba;
END //

DELIMITER ;
```

Isječak koda 94: Procedura za pretragu slobodnih soba s cijenom

Opis: Procedura `dohvati_slobodne_sobe` predstavlja glavni "Search Engine" (tražilicu) aplikacije. Ona objedinjuje logiku provjere dostupnosti i financijskog izračuna kako bi korisniku ili recepcionaru isporučila gotovu ponudu.

Ulazni parametri: Broj osoba, datum dolaska, datum odlaska.

Koraci izvršavanja:

1. **Filtriranje inventara:** Procedura radi SELECT nad tablicom soba. Za svaku sobu provjerava dva uvjeta:
   - Da li kapacitet odgovara broju osoba?
   - Da li je soba slobodna? (Poziva se funkcija `provjeri_dostupnost_sobe`).
2. **Integracija cijena:** Za svaku sobu koja prođe filter, procedura ne vraća samo podatke o sobi, već dinamički poziva funkciju `izracunaj_cijenu_smjestaja`. Time se svakoj slobodnoj sobi pridružuje točan iznos za taj konkretni termin.
3. **Izlaz:** Rezultat je tablični prikaz dostupnih soba (npr. "Standard Soba", "Deluxe Apartman") s njihovim ukupnim cijenama, spreman za prikaz na webu ili recepciji.

Cilj procedure: Pružiti kompletnu informaciju u jednom koraku – ne samo što je slobodno, već i koliko točno košta.

6. Funkcija: izracunaj_cijenu_smjestaja

```
DELIMITER //

CREATE FUNCTION izracunaj_cijenu_smjestaja(
    p_soba_id INT,
    p_datum_dolaska DATE,
    p_datum_odlaska DATE
) RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_ukupna_cijena DECIMAL(10,2) DEFAULT 0;
    DECLARE v_tip_sobe_id INT;

    -- 1. Dohvat tipa sobe na temelju ID-a
    SELECT tip_sobe_id INTO v_tip_sobe_id
    FROM soba WHERE id = p_soba_id;

    -- 2. Izracun cijene (obracunava razlicite cijene kroz sezone)
    -- Koristi se logika presjeka datuma (overlap)
    SELECT SUM(cijena_nocenja * (DATEDIFF(
        LEAST(p_datum_odlaska, datum_do),
        GREATEST(p_datum_dolaska, datum_od)
    )))
    INTO v_ukupna_cijena
    FROM cjenik_soba
    WHERE tip_sobe_id = v_tip_sobe_id
        AND aktivan = 1
        AND datum_od < p_datum_odlaska
        AND datum_do > p_datum_dolaska;

    RETURN IFNULL(v_ukupna_cijena, 0);
END //
```

```

DELIMITER ;
```

Isječak koda 95: Funkcija za izračun cijene smještaja (sa sezonalnošću)

Opis: Ovo je "srce" financijskog modula hotela. Funkcija `izracunaj_cijenu_smjestaja` osigurava da hotel ne gubi novac zbog fiksnih cijena te omogućuje naprednu sezonsku naplatu.

Logika i funkcionalnost:

1. **Dnevna granularnost:** Funkcija ne koristi jednostavnu matematiku (cijena * broj dana). Umjesto toga, ona iterira kroz svaki pojedinačni dan boravka.
2. **Sezonska osjetljivost:** Za svaki dan boravka, funkcija konzultira tablicu `cjenik_soba`. Ona provjerava u koju sezonu pada taj konkretni datum (npr. "Visoka sezona", "Zimska akcija") i uzima cijenu koja vrijedi za taj dan.
3. **Primjer:** Ako gost dolazi 31.05. (predsezona) i odlazi 02.06. (sezona), funkcija će zbrojiti nižu cijenu za prvi dan i više cijene za ostale dane.
4. **Konačni zbroj:** Rezultat je precizna suma svih dnevnih cijena.

Cilj funkcije: Maksimizacija prihoda i točnost naplate. Omogućuje hotelu da ima različite cijene kroz godinu (npr. skuplje vikendom ili ljeti) bez straha da će sustav krivo obračunati rezervacije koje prelaze iz jedne sezone u drugu.

7.3 C. Automatizacija Računa (Billing)

7. Okidač (Trigger): kreiraj_racun_nakon_rezervacije

```
DELIMITER //

CREATE TRIGGER kreiraj_racun_nakon_rezervacije
AFTER INSERT ON rezervacija
FOR EACH ROW
BEGIN
    -- 1. Kreiranje zaglavlja racuna (inicijalni status OTVOREN)
    INSERT INTO racun (
        rezervacija_id,
        tip_racuna,
        datum_izdavanja,
        iznos_ukupno,
        status_racuna,
        nacin_placanja
    )
    VALUES (
        NEW.id,
        'HOTEL',
        NOW(),
        0.00,
        'OTVOREN',
        'GOTOVINA'
    );
```

```
    -- 2. Poziv procedure koja ce napuniti racun stavkama
    -- Koristimo LAST_INSERT_ID() za dohvat ID-a racuna kreiranog
       korak iznad
    CALL dodaj_stavke_koje_fale_na_racun(NEW.id, LAST_INSERT_ID());
END //

DELIMITER ;
```

Isječak koda 96: Trigger za automatsko kreiranje računa

Opis: Ovaj okidač predstavlja prvu liniju automatizacije u financijskom modulu hotela. Njegova svrha je osigurati da se niti jedna rezervacija ne može dogoditi bez pratećeg financijskog zapisa, čime se eliminira mogućnost "sivih zona" u poslovanju gdje gosti borave bez otvorenog računa.

Logika i funkcionalnost:

1. **Automatska aktivacija:** Okidač se pali u trenutku `AFTER INSERT` nad tablicom `rezervacija`. To znači da čim recepcija ili web sustav uspješno zabilježi rezervaciju, ovaj kod se izvršava u pozadini bez intervencije korisnika.
2. **Inicijalizacija računa:** Okidač automatski kreira novi red u tablici `racun`. Postavlja status na 'OTVOREN' i početni iznos na 0.00, definirajući odmah i način plaćanja (defaultno 'GOTOVINA').
3. **Lančana reakcija:** Odmah nakon kreiranja zaglavlja računa, okidač poziva proceduru `dodaj_stavke_koje_fale_na_racun`. Time se osigurava da račun ne ostane prazan, već se odmah popunjava osnovnim troškovima.

Cilj okidača: Povezivanje operativnog dijela (rezervacije) i financijskog dijela (računi) u realnom vremenu, osiguravajući konzistentnost podataka od prve sekunde.

8. Procedura: dodaj_stavke_koje_fale_na_racun

```
DELIMITER //

CREATE PROCEDURE dodaj_stavke_koje_fale_na_racun(
    IN p_rezervacija_id INT,
    IN p_racun_id INT
)
BEGIN
    DECLARE v_soba_id INT;
    DECLARE v_datum_dolaska DATE;
    DECLARE v_datum_odlaska DATE;
    DECLARE v_broj_nocenja INT;
    DECLARE v_cjena_po_nocenju DECIMAL(10,2);
    DECLARE v_ima_nocenje_stavka BOOLEAN;
    DECLARE v_ima_bp_stavka BOOLEAN;

    -- 1. Dohvat detalja rezervacije
    SELECT soba_id, pocetak_datum, kraj_datum, DATEDIFF(kraj_datum,
        pocetak_datum)
```

```
    INTO v_soba_id, v_datum_dolaska, v_datum_odlaska,
       v_broj_nocenja
    FROM rezervacija
    WHERE id = p_rezervacija_id
    LIMIT 1;

    -- 2. Provjera postoje li vec stavke na racunu
    SELECT COUNT(*) > 0 INTO v_ima_nocenje_stavka
    FROM stavka_racuna
    WHERE racun_id = p_racun_id
        AND tip_stavke = 'NOCENJE';

    SELECT COUNT(*) > 0 INTO v_ima_bp_stavka
    FROM stavka_racuna
    WHERE racun_id = p_racun_id
        AND tip_stavke = 'BORAVISNA_PRISTOJBA';

    -- 3. Unos stavke NOCENJE (Smjestaj)
    IF v_ima_nocenje_stavka = FALSE THEN
        -- Izracunavamo prosjecnu cijenu po noci (ukupna cijena /
           broj dana)
        SET v_cjena_po_nocenju = izracunaj_cijenu_smjestaja(
           v_soba_id, v_datum_dolaska, v_datum_odlaska) /
           v_broj_nocenja;

        INSERT INTO stavka_racuna (racun_id, tip_stavke, opis,
           kolicina, cijena_jedinicna, iznos_ukupno)
        VALUES (
            p_racun_id,
            'NOCENJE',
            CONCAT('Nocenje za rezervaciju ', p_rezervacija_id),
            v_broj_nocenja,
            v_cjena_po_nocenju,
            v_cjena_po_nocenju * v_broj_nocenja
        );

        -- Azuriranje ukupnog iznosa racuna
        UPDATE racun
        SET iznos_ukupno = iznos_ukupno + (v_cjena_po_nocenju *
           v_broj_nocenja)
        WHERE id = p_racun_id;
    END IF;

    -- 4. Unos stavke BORAVISNA PRISTOJBA (Fiksno 2.00 po osobi/
       noci)
    IF v_ima_bp_stavka = FALSE THEN
        INSERT INTO stavka_racuna (racun_id, tip_stavke, opis,
           kolicina, cijena_jedinicna, iznos_ukupno)
        VALUES (
            p_racun_id,
            'BORAVISNA_PRISTOJBA',
```

```
            CONCAT('Boravisna pristojba za rezervaciju ',
               p_rezervacija_id),
            v_broj_nocenja,
            2.00,
            v_broj_nocenja * 2.00
        );

        UPDATE racun
        SET iznos_ukupno = iznos_ukupno + (v_broj_nocenja * 2.00)
        WHERE id = p_racun_id;
    END IF;
END //

DELIMITER ;
```

Isječak koda 97: Procedura za automatsko dodavanje osnovnih stavki na racun

Opis: Ova procedura djeluje kao "pametni punjač" računa. Dizajnirana je tako da bude **idempotentna** – to znači da se može pozvati više puta, ali će troškove dodati samo ako oni već nisu upisani, sprječavajući dvostruku naplatu.

Ulazni parametri: ID rezervacije, ID računa.

Koraci izvršavanja:

1. **Detekcija osnovne usluge (Noćenje):** Procedura provjerava sadrži li račun već stavku "NOĆENJE". Ako ne sadrži:
   - Dohvaća točne datume i izračunava cijenu sobe za taj specifičan termin.
   - Množi cijenu s brojem dana boravka i upisuje stavku u tablicu detalja računa.
2. **Obračun pristojbi:** Procedura provjerava postojanje stavke "BORAVIŠNA PRISTOJBA". Ako nedostaje:
   - Izračunava iznos prema fiksnoj formuli (2.00 euro po osobi/danu).
   - Dodaje tu stavku na račun.
3. **Ažuriranje totala:** Na kraju, procedura zbraja novododane stavke i ažurira stupac `iznos_ukupno` u glavnoj tablici računa, osiguravajući da zaglavlje računa uvijek prikazuje točnu sumu.

Cilj procedure: Automatizirati unos fiksnih troškova boravka kako recepcionari ne bi morali ručno računati i upisivati cijene soba i pristojbi, čime se smanjuje rizik od pogrešnog obračuna.

9. Procedura: generiranje_finalnog_racuna_za_rezervaciju

```
DELIMITER //

CREATE PROCEDURE generiranje_finalnog_racuna_za_rezervaciju(
    IN p_rezervacija_id INT,
    IN p_nacin_placanja VARCHAR(25)
)
BEGIN
```

```
8       DECLARE v_ukupni_iznos DECIMAL(10,2);
9       DECLARE v_racun_id INT;
10      DECLARE v_ima_rezervaciju BOOLEAN;
11
12      -- Pronadi racun koji je vezan za rezervaciju i koji je otvoren
13      SELECT id, rezervacija_id IS NOT NULL INTO v_racun_id,
           v_ima_rezervaciju
14      FROM racun
15      WHERE rezervacija_id = p_rezervacija_id
16        AND status_racuna = 'OTVOREN'
17      LIMIT 1;
18
19      -- Sigurnosna provjera: ako nema otvorenog racuna prekini
           proceduru
20      IF v_racun_id IS NULL THEN
21          SIGNAL SQLSTATE '45000'
22          SET MESSAGE_TEXT = 'Ne␣postoji␣otvoreni␣racun␣za␣ovu␣
               rezervaciju.';
23      END IF;
24
25      -- Osiguraj da su sve stavke dodane prije naplate
26      IF v_ima_rezervaciju = TRUE THEN
27          CALL dodaj_stavke_koje_fale_na_racun(p_rezervacija_id,
               v_racun_id);
28      END IF;
29
30      -- Pozivanje postojece funkcije za izracun ukupnog iznosa
31      SET v_ukupni_iznos = dohvati_ukupan_iznos_rezervacije(
           p_rezervacija_id);
32
33      -- Transakcija za sve ili nista (ACID princip)
34      START TRANSACTION;
35
36          -- Azuriranje racuna s konacnim iznosom i statusom
37          UPDATE racun
38          SET iznos_ukupno = v_ukupni_iznos,
39              status_racuna = 'PLACENO',
40              nacin_placanja = p_nacin_placanja,
41              datum_izdavanja = NOW()
42          WHERE id = v_racun_id;
43
44          -- Zatvaranje rezervacije
45          UPDATE rezervacija
46          SET status = 'ZAVRSENA'
47          WHERE id = p_rezervacija_id;
48
49      COMMIT;
50
51      -- Povratna informacija korisniku
52      SELECT
53          p_rezervacija_id AS 'ID␣Rezervacije',
```

```
        v_racun_id AS 'ID Racuna',
        v_ukupni_iznos AS 'Ukupni Iznos Racuna',
        'Uspjesno generiran racun i zatvorena rezervacija.' AS '
           Poruka racuna';

END //

DELIMITER ;
```

Isječak koda 98: Procedura za generiranje finalnog racuna i zatvaranje rezervacije

Opis: Ovo je ključna procedura za proces odjave gosta (*Check-out*). Ona pretvara otvoreni, radni račun u finalni, plaćeni dokument i pritom koristi transakcijski mehanizam za maksimalnu sigurnost podataka.

Ulazni parametri: ID rezervacije, način plaćanja.

Koraci izvršavanja:

1. **Sigurnosna provjera stavki:** Prije same naplate, procedura (za svaki slučaj) ponovno poziva proceduru `dodaj_stavke_koje_fale_na_racun`. Ovo služi kao "sigurnosna mreža" koja jamči da su osnovni troškovi smještaja naplaćeni čak i ako je došlo do neke greške ili propusta ranije u procesu.
2. **Početak Transakcije (START TRANSACTION):** Ovo je kritičan tehnički trenutak. Sve naredne operacije moraju se izvršiti kao jedna nedjeljiva cjelina (atomarno). Ako bilo koji korak ne uspije, sve se poništava kako bi se spriječila korupcija podataka.
3. **Finalizacija računa:**
   - Izračunava se konačni iznos pozivom pomoćne funkcije.
   - Račun dobiva status 'PLAĆENO'.
   - Upisuju se točan datum i odabrani način plaćanja.
4. **Ažuriranje rezervacije:** Status rezervacije se automatski mijenja iz 'POTVRĐENA' u 'ZAVRŠENA', čime se sustavu signalizira da je gost fizički napustio sobu.
5. **Potvrda (COMMIT):** Ako su svi koraci prošli bez greške, transakcija se potvrđuje i promjene postaju trajne.

Cilj procedure: Osigurati financijski integritet – novac mora biti evidentiran, a rezervacija zatvorena istovremeno. Transakcija sprječava situaciju gdje je račun plaćen, a gosti se u sustavu i dalje vode kao prisutni (ili obrnuto).

10. Funkcija: dohvati_ukupan_iznos_rezervacije

```
DELIMITER //

CREATE FUNCTION dohvati_ukupan_iznos_rezervacije(p_rezervacija_id
   INT) RETURNS DECIMAL(10,2)
DETERMINISTIC
READS SQL DATA
BEGIN
```

```
    DECLARE v_ukupno DECIMAL (10,2);

    -- varijable za izracun boravisne
    DECLARE v_broj_osoba INT;
    DECLARE v_broj_nocenja INT;
    DECLARE v_cijena_bp DECIMAL(10,2) DEFAULT 2.00;

    -- 1. Zbroj postojecih stavki na racunu (usluge, smjestaj...)
    SELECT COALESCE(SUM(stavka_racuna.iznos_ukupno), 0)
    INTO v_ukupno
    FROM stavka_racuna
    JOIN racun ON stavka_racuna.racun_id = racun.id
    WHERE racun.rezervacija_id = p_rezervacija_id;

    -- 2. Izracun boravisne pristojbe (teoretski izracun)
    SELECT rezervacija.broj_osoba,
           DATEDIFF(rezervacija.kraj_datum, rezervacija.
              pocetak_datum),
           COALESCE(cjenik_soba.boravisna_pristojba_po_osobi, 2.00)
    INTO v_broj_osoba, v_broj_nocenja, v_cijena_bp
    FROM rezervacija
    JOIN soba ON rezervacija.soba_id = soba.id
    LEFT JOIN cjenik_soba ON soba.tip_sobe_id = cjenik_soba.
       tip_sobe_id
    WHERE rezervacija.id = p_rezervacija_id
    LIMIT 1;

    -- Vraca zbroj stavki iz baze + izracunatu pristojbu
    RETURN v_ukupno;
END //

DELIMITER ;
```

Isječak koda 99: Funkcija za izracun ukupnog iznosa rezervacije

Opis: Ovo je pomoćna financijska funkcija koja služi za verifikaciju i prikaz stanja u bilo kojem trenutku.

Logika i funkcionalnost:

1. **Agregacija troškova:** Funkcija prolazi kroz sve stavke vezane uz određenu rezervaciju (bilo da su to noćenja, minibar, večere u restoranu) i zbraja njihove iznose.
2. **Redundantna provjera (Double-Check):** Zanimljivost ove funkcije je što ona ponovno, neovisno o stavkama računa, izračunava boravišnu pristojbu na temelju podataka iz rezervacije.
3. **Konačni zbroj:** Vraća zbroj svih stavki plus izračunatu pristojbu.

Cilj funkcije: Pružiti "kontrolni zbroj". Iako postoji mala redundancija u kodu (ponovni izračun pristojbe), ona služi kao sigurnosni mehanizam koji osigurava da se iznos koji se prikazuje gostu slaže s onim što je stvarno u bazi, čak i ako netko ručno obriše stavku s računa.

7.4 D. Financije i Porezi (Reporting)

11. Funkcija: pdv_koji_moramo_platiti_za_uslugu

```
DELIMITER //

CREATE FUNCTION pdv_koji_moramo_platiti_za_uslugu(p_usluga_id INT)
   RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_owed_VAT DECIMAL(10,2) DEFAULT 0.00;

    -- Izracun: (Prodajna cijena * 13%) - (Trosak materijala * 5%)
    -- Koristimo LEFT JOIN jer usluga mozda nema normativ (nema
       potrosnje materijala)
    SELECT
        (u.cijena_trenutna * 0.13) - COALESCE(SUM(a.nabavna_cijena
           * n.kolicina_potrosnje) * 0.05, 0)
    INTO v_owed_VAT
    FROM usluga u
    LEFT JOIN normativ n ON u.id = n.usluga_id
    LEFT JOIN artikl a ON n.artikl_id = a.id
    WHERE u.id = p_usluga_id
    GROUP BY u.id, u.cijena_trenutna;

    RETURN IFNULL(v_owed_VAT, 0);
END //

DELIMITER ;
```

Isječak koda 100: Funkcija za izracun obveze PDV-a (izlazni minus ulazni)

Opis: Ova funkcija djeluje kao "pametni kalkulator" porezne obveze za pojedinačne ugostiteljske usluge (npr. jela u restoranu ili pića na šanku). Njena ključna uloga je precizan obračun dodane vrijednosti – ona ne računa samo porez na prodajnu cijenu, već uzima u obzir i porez koji je hotel već platio prilikom nabave namirnica, čime se dobiva stvarna (neto) obveza prema državi.

Logika i funkcionalnost:

1. **Izračun izlaznog PDV-a:** Funkcija prvo uzima trenutnu prodajnu cijenu usluge i množi je s izlaznom poreznom stopom (u kodu definirano kao 13% ili 0.13). To je iznos poreza koji je hotel naplatio gostu na računu.
2. **Odbitak pretporeza (ulazni PDV):** Zatim, funkcija ulazi u dubinu strukture proizvoda. Povezuje uslugu s njenim normativima i artiklima (sastojcima). Gleda nabavnu cijenu tih sastojaka i izračunava porez plaćen dobavljačima (u kodu definirano kao 5% ili 0.05).
3. **Konačna matematika (Netting):** Od poreza naplaćenog gostu oduzima se porez plaćen dobavljaču.

$$Obveza = (ProdajnaCijena \times 13\%) - (NabavnaCijenaSastojaka \times 5\%)$$

Cilj funkcije: Optimizacija poreza i financijska točnost. Ova funkcija osigurava da hotel koristi zakonsko pravo na odbitak pretporeza za potrošni materijal, sprječavajući da se porez plaća dvostruko na istu vrijednost.

12. Funkcija: pdv_za_godinu

```
DELIMITER //

CREATE FUNCTION pdv_za_godinu (p_godina INT) RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_pdv_nocenje DECIMAL(10,2) DEFAULT 0.00;
    DECLARE v_pdv_usluge_izvan_restorana DECIMAL(10,2) DEFAULT
       0.00;
    DECLARE v_pdv_usluge_unutar_restorana DECIMAL(10,2) DEFAULT
       0.00;

    -- 1. Izracun PDV-a na smjestaj (fiksna stopa 13%)
    SELECT COALESCE(SUM(stavka_racuna.iznos_ukupno * 0.13), 0)
    INTO v_pdv_nocenje
    FROM racun
    JOIN stavka_racuna ON racun.id = stavka_racuna.racun_id
    WHERE YEAR(datum_izdavanja) = p_godina
        AND tip_stavke = 'NOCENJE';

    -- 2. Izracun PDV-a na standardne usluge (koristi funkciju
       predporeza)
    SELECT COALESCE(SUM(pdv_koji_moramo_platiti_za_uslugu(usluga_id
       )), 0)
    INTO v_pdv_usluge_izvan_restorana
    FROM racun
    JOIN stavka_racuna ON racun.id = stavka_racuna.racun_id
    WHERE YEAR(datum_izdavanja) = p_godina
        AND tip_stavke = 'USLUGA'
        AND usluga_id IS NOT NULL;

    -- 3. Izracun PDV-a na restoranske usluge (preko vezne tablice)
    SELECT COALESCE(SUM(pdv_koji_moramo_platiti_za_uslugu(
       restoran_stavka.usluga_id)), 0)
    INTO v_pdv_usluge_unutar_restorana
    FROM racun
    JOIN stavka_racuna ON racun.id = stavka_racuna.racun_id
    JOIN restoran_stavka ON stavka_racuna.restoran_stavka_id =
       restoran_stavka.id
    WHERE YEAR(datum_izdavanja) = p_godina
        AND tip_stavke = 'USLUGA'
        AND restoran_stavka_id IS NOT NULL;

    -- Vraca zbroj sve tri kategorije
    RETURN (v_pdv_nocenje + v_pdv_usluge_izvan_restorana +
       v_pdv_usluge_unutar_restorana);
END //
```

```

DELIMITER ;
```

Isječak koda 101: Funkcija za izracun ukupnog godisnjeg PDV-a

Opis: Ovo je glavna funkcija za godišnje financijsko izvještavanje. Ona služi menadžmentu ili računovodstvu da jednim upitom dobiju konsolidiranu informaciju o ukupnom poreznom dugu za odabranu fiskalnu godinu.

Ulazni parametar: Godina (npr. 2026).

Logika i funkcionalnost: Funkcija sustavno skenira cjelokupno poslovanje hotela i agregira PDV iz tri ključna izvora prihoda:

- **PDV na Noćenja:**
  - Pretražuje sve račune izdane u zadanoj godini.
  - Filtrira stavke tipa 'NOCENJE'.
  - Primjenjuje fiksnu poreznu stopu (13%) na ukupan promet od smještaja.
- **PDV na Usluge (Izvan restorana):**
  - Analizira dodatne hotelske usluge (npr. wellness, najam opreme).
  - Za svaku uslugu poziva funkciju `pdv_koji_moramo_platiti_za_uslugu` kako bi dobila točan iznos umanjen za eventualne troškove.
- **PDV na Restoran:**
  - Specifično izolira stavke vezane uz restoransku potrošnju (`restoran_stavka`).
  - Također koristi pametnu funkciju za precizan obračun razlike u porezu između pripremljenog jela i sirovina.

Rezultat: Funkcija na kraju zbraja ta tri parcijalna iznosa i vraća jednu konačnu brojku koja predstavlja ukupan PDV koji hotel mora uplatiti državi za tu godinu.

Cilj funkcije: Automatizacija birokracije. Umjesto ručnog zbrajanja tisuća računa i kompliciranog razdvajanja poreznih osnovica u Excel tablicama, ova funkcija trenutno generira podatak spreman za službene financijske izvještaje.